

CIS Oracle Cloud Infrastructure Container Engine for Kubernetes(OKE) Benchmark

v2.0.0 - 07-02-2026

Terms of Use

Please see the below link for our current terms of use:

<https://www.cisecurity.org/cis-securesuite/cis-securesuite-membership-terms-of-use/>

For information on referencing and/or citing CIS Benchmarks in 3rd party documentation (including using portions of Benchmark Recommendations) please contact CIS Legal (legalnotices@cisecurity.org) and request guidance on copyright usage.

NOTE: It is **NEVER** acceptable to host a CIS Benchmark in **ANY** format (PDF, etc.) on a 3rd party (non-CIS owned) site.

Table of Contents

Terms of Use	1
Table of Contents.....	2
Overview	5
Important Usage Information	5
Key Stakeholders	5
Apply the Correct Version of a Benchmark	6
Exceptions	6
Remediation.....	7
Summary	7
Target Technology Details.....	8
Intended Audience	8
Consensus Guidance	9
Typographical Conventions	10
Recommendation Definitions	11
Title	11
Assessment Status	11
Automated.....	11
Manual	11
Profile	11
Description.....	11
Rationale Statement	11
Impact Statement.....	12
Audit Procedure.....	12
Remediation Procedure	12
Default Value.....	12
References	12
CIS Critical Security Controls® (CIS Controls®).....	12
Additional Information	12
Profile Definitions	13
Acknowledgements.....	Error! Bookmark not defined.
Recommendations	15
1 Control Plane Components	15
2 Control Plane Configuration.....	16
2.1 Authentication and Authorization.....	17
2.1.1 Use OCI IAM or OIDC tokens instead of user client certificates (Manual).....	18
2.2 Logging	21

2.2.1 Ensure authorized access to OCI Audit logs for OKE activity (Manual).....	22
3 Worker Nodes	24
3.1 Worker Node Configuration Files	25
3.1.1 Ensure that the kubelet.conf file permissions are set to 644 or more restrictive (Automated).....	26
3.1.2 Ensure that the kubelet.conf file ownership is set to root:root (Automated).....	29
3.1.3 Ensure that the kubelet configuration file has permissions set to 644 or more restrictive (Automated).....	32
3.1.4 Ensure that the kubelet configuration file ownership is set to root:root (Automated)	35
3.2 Kubelet	38
3.2.1 Ensure kubelet anonymous authentication is disabled (Automated).....	39
3.2.2 Ensure kubelet authorization mode is set to Webhook (Automated)	43
3.2.3 Ensure the kubelet X.509 client CA file is configured (Automated).....	47
3.2.4 Ensure the kubelet read-only port is disabled (Manual).....	51
3.2.5 Ensure kubelet makeIPTablesUtilChains is set to true (Automated).....	55
3.2.6 Ensure the kubelet event recording QPS is set to an appropriate value (Automated)	59
3.2.7 Ensure kubelet tlsCertFile and tlsPrivateKeyFile are configured appropriately (Automated).....	63
3.2.8 Ensure the kubelet client certificate rotation is enabled (Automated)	67
4 Policies	71
4.1 RBAC and Service Accounts	72
4.1.1 Ensure that the cluster-admin role is only used where required (Manual)	73
4.1.2 Ensure access to Secrets is minimized and granted only where required (Manual).....	76
4.1.3 Minimize wildcard use in Roles and ClusterRoles (Manual)	80
4.1.4 Minimize access to create pods (Manual).....	84
4.1.5 Ensure that default service accounts are not actively used. (Automated).....	88
4.1.6 Ensure that Service Account Tokens are only mounted where necessary (Automated)	92
4.2 Pod Security Policies	95
4.2.1 Minimize the admission of privileged containers (Automated)	96
4.2.2 Minimize the admission of containers wishing to share the host process ID namespace (Automated).....	100
4.2.3 Minimize the admission of containers wishing to share the host IPC namespace (Automated).....	103
4.2.4 Minimize the admission of containers wishing to share the host network namespace (Automated).....	107
4.2.5 Minimize the admission of containers with allowPrivilegeEscalation (Automated).....	111
4.3 CNI Plugin	115
4.3.1 Ensure the OKE CNI add-on uses an Oracle supported version (Automated)	116
4.3.2 Ensure workload namespaces have required NetworkPolicies defined (Manual).....	119
4.4 Secrets Management	122
4.4.1 Prefer exposing secrets as mounted files rather than environment variables (Automated)	123
4.4.2 Consider external secret storage (Manual)	126
4.5 General Policies	129
4.5.1 Create administrative boundaries between resources using namespaces (Manual)	130
4.5.2 Apply securityContext controls to pods and containers (Manual)	134
4.5.3 The default namespace should not be used (Manual)	138
5 Managed services.....	141
5.1 Image Registry and Image Scanning.....	142
5.1.1 Ensure image vulnerability scanning uses OCIR or a third party provider (Manual).....	143
5.1.2 Restrict OKE and Kubernetes API access to required users (Manual).....	147
5.1.3 Restrict OCIR access for OKE workloads to required image pulls (Manual)	150
5.1.4 Allow only approved container registries for images used by OKE workloads (Manual)	154

5.2 Identity and Access Management (IAM)	157
5.2.1 Use dedicated Kubernetes service accounts for OKE workloads (Automated)	158
5.3 Cloud Key Management Service (Cloud KMS)	161
5.3.1 Use customer managed KMS keys for OKE envelope encryption of Kubernetes secrets (Manual)	162
5.4 Cluster Networking	166
5.4.1 Restrict OKE control plane endpoint access using private endpoints or approved CIDRs (Automated).....	167
5.4.2 Enable private OKE control plane access and disable public IP access (Automated)..	171
5.4.3 Ensure clusters are created with Private Nodes (Manual)	174
5.4.4 Ensure Network Policy is Enabled and set as appropriate (Manual)	178
5.4.5 Use TLS certificates on OKE load balancer listeners for encrypted client connections (Manual)	182
Appendix: Summary Table	185
Appendix: CIS Controls v7 IG 1 Mapped Recommendations	189
Appendix: CIS Controls v7 IG 2 Mapped Recommendations	191
Appendix: CIS Controls v7 IG 3 Mapped Recommendations	193
Appendix: CIS Controls v7 Unmapped Recommendations	195
Appendix: CIS Controls v8 IG 1 Mapped Recommendations	196
Appendix: CIS Controls v8 IG 2 Mapped Recommendations	198
Appendix: CIS Controls v8 IG 3 Mapped Recommendations	201
Appendix: CIS Controls v8 Unmapped Recommendations	204
Appendix: Change History	<i>Error! Bookmark not defined.</i>

Overview

All CIS Benchmarks™ (Benchmarks) focus on technical configuration settings used to maintain and/or increase the security of the addressed technology, and they should be used in **conjunction** with other essential cyber hygiene tasks like:

- Monitoring the base operating system and applications for vulnerabilities and quickly updating with the latest security patches.
- End-point protection (Antivirus software, Endpoint Detection and Response (EDR), etc.).
- Logging and monitoring user and system activity.

In the end, the Benchmarks are designed to be a key **component** of a comprehensive cybersecurity program.

Important Usage Information

All Benchmarks are available free for non-commercial use from the [CIS Website](#). They can be used to manually assess and remediate systems and applications. In lieu of manual assessment and remediation, there are several tools available to assist with assessment:

- [CIS Configuration Assessment Tool \(CIS-CAT® Pro Assessor\)](#)
- [CIS Benchmarks™ Certified 3rd Party Tooling](#)

These tools make the hardening process much more scalable for large numbers of systems and applications.

NOTE: Some tooling focuses only on the Benchmark Recommendations that can be fully automated (skipping ones marked **Manual**). It is important that **ALL** Recommendations (**Automated** and **Manual**) be addressed since all are important for properly securing systems and are typically in scope for audits.

Key Stakeholders

Cybersecurity is a collaborative effort, and cross functional cooperation is imperative within an organization to discuss, test, and deploy Benchmarks in an effective and efficient way. The Benchmarks are developed to be best practice configuration guidelines applicable to a wide range of use cases. In some organizations, exceptions to specific Recommendations will be needed, and this team should work to prioritize the problematic Recommendations based on several factors like risk, time, cost, and labor. These exceptions should be properly categorized and documented for auditing purposes.

Apply the Correct Version of a Benchmark

Benchmarks are developed and tested for a specific set of products and versions and applying an incorrect Benchmark to a system can cause the resulting pass/fail score to be incorrect. This is due to the assessment of settings that do not apply to the target systems. To assure the correct Benchmark is being assessed:

- **Deploy the Benchmark applicable to the way settings are managed in the environment:** An example of this is the Microsoft Windows family of Benchmarks, which have separate Benchmarks for Group Policy, Intune, and Stand-alone systems based upon how system management is deployed. Applying the wrong Benchmark in this case will give invalid results.
- **Use the most recent version of a Benchmark:** This is true for all Benchmarks, but especially true for cloud technologies. Cloud technologies change frequently and using an older version of a Benchmark may have invalid methods for auditing and remediation.

Exceptions

The guidance items in the Benchmarks are called recommendations and not requirements, and exceptions to some of them are expected and acceptable. The Benchmarks strive to be a secure baseline, or starting point, for a specific technology, with known issues identified during Benchmark development are documented in the Impact section of each Recommendation. In addition, organizational, system specific requirements, or local site policy may require changes as well, or an exception to a Recommendation or group of Recommendations (e.g. A Benchmark could Recommend that a Web server not be installed on the system, but if a system's primary purpose is to function as a Webserver, there should be a documented exception to this Recommendation for that specific server).

In the end, exceptions to some Benchmark Recommendations are common and acceptable, and should be handled as follows:

- The reasons for the exception should be reviewed cross-functionally and be well documented for audit purposes.
- A plan should be developed for mitigating, or eliminating, the exception in the future, if applicable.
- If the organization decides to accept the risk of this exception (not work toward mitigation or elimination), this should be documented for audit purposes.

It is the responsibility of the organization to determine their overall security policy, and which settings are applicable to their unique needs based on the overall risk profile for the organization.

Remediation

CIS has developed [Build Kits](#) for many technologies to assist in the automation of hardening systems. Build Kits are designed to correspond to Benchmark's "Remediation" section, which provides the manual remediation steps necessary to make that Recommendation compliant to the Benchmark.

When remediating systems (changing configuration settings on deployed systems as per the Benchmark's Recommendations), please approach this with caution and test thoroughly.

The following is a reasonable remediation approach to follow:

- CIS Build Kits, or internally developed remediation methods should never be applied to production systems without proper testing.
- Proper testing consists of the following:
 - Understand the configuration (including installed applications) of the targeted systems. Various parts of the organization may need different configurations (e.g., software developers vs standard office workers).
 - Read the Impact section of the given Recommendation to help determine if there might be an issue with the targeted systems.
 - Test the configuration changes with representative lab system(s). If issues arise during testing, they can be resolved prior to deploying to any production systems.
 - When testing is complete, initially deploy to a small sub-set of production systems and monitor closely for issues. If there are issues, they can be resolved prior to deploying more broadly.
 - When the initial deployment above is completed successfully, iteratively deploy to additional systems and monitor closely for issues. Repeat this process until the full deployment is complete.

Summary

Using the Benchmarks Certified tools, working as a team with key stakeholders, being selective with exceptions, and being careful with remediation deployment, it is possible to harden large numbers of deployed systems in a cost effective, efficient, and safe manner.

NOTE: As previously stated, the PDF versions of the CIS Benchmarks™ are available for free, non-commercial use on the [CIS Website](#). All other formats of the CIS Benchmarks™ (MS Word, Excel, and [Build Kits](#)) are available for CIS [SecureSuite](#)® members.

CIS-CAT® Pro is also available to CIS [SecureSuite](#)® members.

Target Technology Details

This document provides prescriptive guidance for running Oracle Kubernetes Engine (OKE) vNext following recommended security controls. This benchmark only includes controls which can be modified by an end user of OKE. For information on OKE's performance against the Kubernetes CIS benchmarks, for items which cannot be audited or modified, see the OKE documentation at <https://docs.cloud.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengoverview.htm>

To obtain the latest version of this guide, please visit www.cisecurity.org. If you have questions, comments, or have identified ways to improve this guide, please write us at support@cisecurity.org.

Intended Audience

This document is intended for cluster administrators, security specialists, auditors, and any personnel who plan to develop, deploy, assess, or secure solutions that incorporate Oracle Kubernetes Engine (OKE).

Consensus Guidance

This CIS Benchmark™ was created using a consensus review process comprised of a global community of subject matter experts. The process combines real world experience with data-based information to create technology specific guidance to assist users to secure their environments. Consensus participants provide perspective from a diverse set of backgrounds including consulting, software development, audit and compliance, security research, operations, government, and legal.

Each CIS Benchmark undergoes two phases of consensus review. The first phase occurs during initial Benchmark development. During this phase, subject matter experts convene to discuss, create, and test working drafts of the Benchmark. This discussion occurs until consensus has been reached on Benchmark recommendations. The second phase begins after the Benchmark has been published. During this phase, all feedback provided by the Internet community is reviewed by the consensus team for incorporation in the Benchmark. If you are interested in participating in the consensus process, please visit <https://workbench.cisecurity.org/>.

Typographical Conventions

The following typographical conventions are used throughout this guide:

Convention	Meaning
<code>Stylized Monospace font</code>	Used for blocks of code, command, and script examples. Text should be interpreted exactly as presented.
<code>Monospace font</code>	Used for inline code, commands, UI/Menu selections or examples. Text should be interpreted exactly as presented.
<code><Monospace font in brackets></code>	Text set in angle brackets denote a variable requiring substitution for a real value.
<i>Italic font</i>	Used to reference other relevant settings, CIS Benchmarks and/or Benchmark Communities. Also, used to denote the title of a book, article, or other publication.
Bold font	Additional information or caveats things like Notes , Warnings , or Cautions (usually just the word itself and the rest of the text normal).

Recommendation Definitions

The following defines the various components included in a CIS recommendation as applicable. If any of the components are not applicable it will be noted, or the component will not be included in the recommendation.

Title

Concise description for the recommendation's intended configuration.

Assessment Status

An assessment status is included for every recommendation. The assessment status indicates whether the given recommendation can be automated or requires manual steps to implement. Both statuses are equally important and are determined and supported as defined below:

Automated

Represents recommendations for which assessment of a technical control can be fully automated and validated to a pass/fail state. Recommendations will include the necessary information to implement automation.

Manual

Represents recommendations for which assessment of a technical control cannot be fully automated and requires all or some manual steps to validate that the configured state is set as expected. The expected state can vary depending on the environment.

Profile

A collection of recommendations for securing a technology or a supporting platform. Most benchmarks include at least a Level 1 and Level 2 Profile. Level 2 extends Level 1 recommendations and is not a standalone profile. The Profile Definitions section in the benchmark provides the definitions as they pertain to the recommendations included for the technology.

Description

Detailed information pertaining to the setting with which the recommendation is concerned. In some cases, the description will include the recommended value.

Rationale Statement

Detailed reasoning for the recommendation to provide the user a clear and concise understanding on the importance of the recommendation.

Impact Statement

Any security, functionality, or operational consequences that can result from following the recommendation.

Audit Procedure

Systematic instructions for determining if the target system complies with the recommendation.

Remediation Procedure

Systematic instructions for applying recommendations to the target system to bring it into compliance according to the recommendation.

Default Value

Default value for the given setting in this recommendation, if known. If not known, either not configured or not defined will be applied.

References

Additional documentation relative to the recommendation.

CIS Critical Security Controls® (CIS Controls®)

The mapping between a recommendation and the CIS Controls is organized by CIS Controls version, Safeguard, and Implementation Group (IG). The Benchmark in its entirety addresses the CIS Controls safeguards of (v7) "5.1 - Establish Secure Configurations" and (v8) "4.1 - Establish and Maintain a Secure Configuration Process" so individual recommendations will not be mapped to these safeguards.

Additional Information

Supplementary information that does not correspond to any other field but may be useful to the user.

Profile Definitions

The following configuration profiles are defined by this Benchmark:

- **Level 1**

Worker Node Profile

- **Level 2**

Extends Level 1

Acknowledgements

This Benchmark exemplifies the great things a community of users, vendors, and subject matter experts can accomplish through consensus collaboration. The CIS community thanks the entire consensus team with special recognition to the following individuals who contributed greatly to the creation of this guide:

Author/s

Gilson Melo
Adao Oliveira

Editor

Randall Mowen & Mark Larinde

Contributors

Logan Kleier
Adao Oliveira Junior KCNA, CKA, Oracle Corporation
Sherwood Zern
Josh Hammer
Yogesh Asalkar

Recommendations

1 Control Plane Components

In Oracle OKE, the Kubernetes control plane provides the core cluster management functions and is fully managed by Oracle. The control plane runs in the OKE service tenancy, not on customer managed worker nodes, and includes the components required to process Kubernetes API requests, maintain cluster state, schedule workloads, and coordinate cluster behavior. Key components include kube-apiserver, kube-controller-manager, kube-scheduler, etcd, and cloud-controller-manager.

The Kubernetes API endpoint is the primary interface for administrators, automation, and tools such as kubectl to interact with the cluster. In VCN native OKE clusters, the API endpoint is hosted in a subnet in the customer VCN and can be configured for private or public access, with access controlled through network security groups or security lists. Oracle maintains and operates the managed control plane components, while customers interact with the cluster through supported OKE, OCI, and Kubernetes APIs.

Security is a shared responsibility between Oracle and the OKE customer. Under the [Oracle Cloud Infrastructure Shared Security Model](#)

Security of the cloud – Oracle is responsible for protecting the infrastructure that runs Oracle services in the Oracle Cloud. For OKE, Oracle is responsible for the Kubernetes control plane, which includes the control plane nodes and etcd database. Third-party auditors regularly test and verify the effectiveness of our security as part of the [Oracle compliance programs](#).

Security in the cloud – Your responsibility includes the following areas:

- The security configuration of the data plane, including the configuration of the security groups that allow traffic to pass from the Oracle OKE control plane into the customer Virtual Cloud Network (VCN)
- The configuration of the worker nodes and the containers themselves
- The worker node guest operating system (including updates and security patches)
 - OKE follows the shared responsibility model for CVEs and security patches on managed node groups. Because managed nodes run the OKE-optimized OPIs, OKE is responsible for building patched versions of these OPIs when bugs or issues are reported and we are able to publish a fix. Customers are responsible for deploying these patched OPI versions to your managed node groups.
- Other associated application software:
 - Setting up and managing network controls, such as firewall rules
 - Managing platform-level identity and access management, either with or in addition to IAM

- The sensitivity of your data, your company's requirements, and applicable laws and regulations

2 Control Plane Configuration

This section contains recommendations for Oracle OKE control plane configuration. Customers are able to configure logging for control plane in Oracle OKE.

2.1 Authentication and Authorization

2.1.1 Use OCI IAM or OIDC tokens instead of user client certificates (Manual)

Profile Applicability:

- Level 1

Description:

OKE authenticates cluster users through OCI IAM by default, and enhanced clusters can also be configured to use an external OIDC identity provider for user authentication. User access should rely on OCI IAM or OIDC tokens rather than Kubernetes client certificates, because these mechanisms align user access to managed identity, policy, and RBAC controls.

OKE provides a managed Kubernetes control plane, and control plane authentication settings are modified only through supported OKE APIs and configuration options. Client certificates used by Kubernetes components are not a customer-managed user authentication method

Rationale:

With any authentication mechanism the ability to revoke credentials if they are compromised or no longer required, is a key control. OKE kubeconfig version 2.0.0 uses an OCI CLI command to dynamically generate short lived user-specific authentication tokens. These tokens are not intended to be shared between users, and OIDC can be used when the organization wants users to authenticate through an external identity provider.

Impact:

Using OCI IAM or OIDC for user authentication requires users to have the required OCI CLI, kubeconfig, IAM policy, OIDC identity provider, and Kubernetes RBAC configuration in place. OIDC also requires the organization to maintain the external identity provider configuration and token retrieval process.

Audit:

Review user access to the cluster and ensure that users are not making use of Kubernetes client certificate authentication.

Review approved user kubeconfig files and confirm that user entries use OCI IAM generated tokens or OIDC tokens instead of client certificate fields.

Run the following command to verify the authentication method configured for users in the current kubeconfig file:

```
kubectl config view --raw -o json | jq -r '
  .users[]
  | [
    .name,
    (
      if .user.exec then "exec_token"
      elif .user["auth-provider"]?.name == "oidc" then "oidc_token"
      elif .user.token then "static_token"
      elif (
        .user["client-certificate"]? or
        .user["client-certificate-data"]? or
        .user["client-key"]? or
        .user["client-key-data"]?
      ) then "client_certificate"
      else "unknown"
      end
    )
  ]
  | @tsv
'
```

The expected result is that user entries use `exec_token` for OCI IAM based OKE access or `oidc_token` for OIDC based access. Any user entry showing `client_certificate` should be reviewed and replaced with a supported OCI IAM or OIDC token based access method.

Remediation:

Replace user kubeconfig entries that use client certificates with OCI IAM generated tokens or OIDC token based authentication. For OKE clusters, user authentication should align to the supported OKE access model, where OCI IAM is used by default and OIDC can be configured for external identity provider based authentication. This allows user access to be managed through identity, policy, and Kubernetes RBAC controls rather than long lived certificate material stored in kubeconfig files.

- For OCI IAM based access, regenerate the user kubeconfig file from OKE using kubeconfig token version 2.0.0 so the user entry uses the OCI CLI exec plugin to generate authentication tokens.
- For OIDC based access, configure the OKE cluster for OIDC authentication and update the user kubeconfig entry to use the approved OIDC identity provider token flow.
- Remove or replace any user kubeconfig entries that contain `client-certificate`, `client-certificate-data`, `client-key`, or `client-key-data`, then validate that required users can still authenticate with only their approved RBAC permissions.

Default Value:

By default, OKE clusters use OCI IAM to authenticate users. When a kubeconfig file is created with token version 2.0.0, it contains an OCI CLI command that generates short-lived, cluster-scoped, user-specific authentication tokens. OCI IAM authentication remains enabled even when OIDC authentication is also enabled.

References:

1. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/contengdownloadkubeconfigfile.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengOpenIDConnect-Authentication.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutaccesscontrol.htm>
4. https://docs.oracle.com/iaas/tools/oci-cli/latest/oci_cli_docs/cmdref/ce/cluster/create-kubeconfig.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.8 <u>Uninstall or Disable Unnecessary Services on Enterprise Assets and Software</u> Uninstall or disable unnecessary services on enterprise assets and software, such as an unused file sharing service, web application module, or service function.			
v7	4.3 <u>Ensure the Use of Dedicated Administrative Accounts</u> Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1078, T1098	TA0003, TA0006	

2.2 Logging

2.2.1 Ensure authorized access to OCI Audit logs for OKE activity (Manual)

Profile Applicability:

- Level 1

Description:

OCI Audit logs are separate from OKE managed Kubernetes control plane service logs. OKE integrates with the Oracle Cloud Infrastructure Audit service to record operations performed by Kubernetes Engine and the Kubernetes API server as Audit log events.

Operations performed by Kubernetes Engine and the Kubernetes API server are visible as log events in the Oracle Cloud Infrastructure Audit service.

Rationale:

Logging is a crucial detective control for all systems to detect potential unauthorized access. Authorized access to OCI Audit logs helps security and operations teams review OKE activity, identify security anomalies, support compliance checks, and troubleshoot errors by identifying who did what and when.

Impact:

OKE audit events are available through the Oracle Cloud Infrastructure Audit service, while Kubernetes control plane process logs are available separately as OKE service logs in Oracle Cloud Infrastructure Logging. Access to Audit events requires IAM authorization to read **audit-events**, and access should be limited to approved users or groups.

Audit:

To monitor and manage operations performed by Container Engine for Kubernetes on a particular cluster:

Review IAM policies and confirm that only authorized groups are granted access to read OCI Audit events for the tenancy or approved compartment. The read **audit-events** permission allows access to Audit log events.

To view operations performed by Container Engine for Kubernetes and the Kubernetes API server as log events in the Oracle Cloud Infrastructure Audit service:

Open the **navigation menu**, select **Identity & Security**, and then select **Audit**. Choose the **compartment** you have permission to work in, then search and filter for the operations of interest. Use **ClustersAPI** to view operations performed by Kubernetes Engine, and use **OKE API Server Admin Access** to view operations performed by the Kubernetes API server.

Remediation:

Update OCI IAM policies so only approved security, operations, or audit groups can read OCI Audit events for the required tenancy or compartment. Remove broad or unnecessary Audit access from users or groups that do not require it, and validate that authorized users can view OKE related events in the Audit service using the appropriate compartment and filters.

Default Value:

By default, Kubernetes Engine and Kubernetes API server audit events are sent to the Oracle Cloud Infrastructure Audit service. Audit logs are retained for 365 days, and the retention period is a tenancy level setting that affects all regions and compartments and cannot be changed.

References:

1. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/contengmonitoringoke.htm>
2. https://docs.oracle.com/iaas/Content/ContEng/Tasks/contengobservingclusters_topic.htm
3. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/contengviewingservicelogs.htm>
4. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengpolicyconfig.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.2 <u>Collect Audit Logs</u> Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.			
v7	6.2 <u>Activate audit logging</u> Ensure that local logging has been enabled on all systems and networking devices.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1543	TA0003, TA0004	M1026

3 Worker Nodes

This section consists of security recommendations for the components that run on OKE worker nodes.

3.1 Worker Node Configuration Files

This section covers recommendations for configuration files on the Oracle OKE worker nodes.

3.1.1 Ensure that the kubelet.conf file permissions are set to 644 or more restrictive (Automated)

Profile Applicability:

- Level 1

Description:

If **kubelet** is running, and if it is using a file-based kubelet.conf file, ensure that the proxy kubelet.conf file has permissions of **644** or more restrictive.

Rationale:

The **kubelet** kubelet.conf file controls various parameters of the **kubelet** service in the worker node. You should restrict its file permissions to maintain the integrity of the file. The file should be writable by only the administrators on the system.

It is possible to run **kubelet** with the kubeconfig parameters configured as a Kubernetes ConfigMap instead of a file. In this case, there is no proxy kubelet.conf file.

Impact:

Ensuring that the **kubelet.conf** file permissions are set to 644 or more restrictive significantly strengthens the security posture of the Kubernetes environment by preventing unauthorized modifications. This restricts write access to the **kubelet.conf** file, ensuring only administrators can alter crucial **kubelet** configurations, thereby reducing the risk of malicious alterations that could compromise the cluster's integrity.

However, this configuration may slightly impact usability, as it limits the ability for non-administrative users to make quick adjustments to the **kubelet** settings. Administrators will need to balance security needs with operational flexibility, potentially requiring adjustments to workflows for managing **kubelet** configurations.

Audit:

Method 1

Optional: Run the following command to enter **debug mode** where **NODE_NAME** and **IMAGE** are defined variables:

```
kubect1 debug node/$NODE_NAME -it --image=$IMAGE --profile=general
```

Run the following command on each node to find the appropriate kubeconfig file:

```
ps -efww | grep kubelet | grep -- '--kubeconfig'
```

The command line output should show a path for kubeconfig similar to -- kubeconfig=/etc/kubernetes/kubelet.conf which is the location of the kubeconfig file.

Run this command to obtain the kubeconfig file permissions:

```
stat -c %a /etc/kubernetes/kubelet.conf (or stat -c %a
/host/etc/kubernetes/kubelet.conf if in debug mode)
```

The output of the above command gives you the kubeconfig file's permissions.

Verify that if a file is specified and it exists, the permissions are **644** or more restrictive.

Method 2

Create and Run a Privileged Pod.

You will need to run a pod that is privileged enough to access the host's file system. This can be achieved by deploying a pod that uses the hostPath volume to mount the node's file system into the pod.

Here's an example of a simple pod definition that mounts the root of the host to /host within the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: file-check
spec:
  volumes:
  - name: host-root
    hostPath:
      path: /
      type: Directory
  containers:
  - name: nsenter
    image: busybox
    command: ["sleep", "3600"]
    volumeMounts:
    - name: host-root
      mountPath: /host
    securityContext:
      privileged: true
```

Save this to a file (e.g., file-check-pod.yaml) and create the pod:

```
kubectl apply -f file-check-pod.yaml
```

Once the pod is running, you can exec into it to check file permissions on the node:

```
kubectl exec -it -n kube-public file-check -- sh
```

Now you are in a shell inside the pod, but you can access the node's file system through the /host directory and check the permission level of the file:

```
ls -l /host/etc/kubernetes/kubelet.conf
```

Verify that if a file is specified and it exists, the permissions are **644** or more restrictive.

Remediation:

Run the below command (based on the file location on your system) on the each worker node. For example,

```
chmod 644 <kubelet.conf file>
```

Default Value:

See the OKE documentation for the default value.

References:

1. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengclustersnodes.htm>
2. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/conteng-kpo.htm>
3. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/replace-worker-node-top.htm>
4. <https://docs.oracle.com/en/learn/oci-oke-kubelet-osdaemons/index.html>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.			
v7	14.6 Protect Information through Access Control Lists Protect all information stored on systems with file system, network share, claims, application, or database specific access control lists. These controls will enforce the principle that only authorized individuals should have access to the information based on their need to access the information as a part of their responsibilities.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1083, T1222	TA0005, TA0007	M1022

3.1.2 Ensure that the kubelet.conf file ownership is set to root:root (Automated)

Profile Applicability:

- Level 1

Description:

If **kubelet** is running, ensure that the file ownership of its kubelet.conf file is set to **root:root**.

Rationale:

The kubelet.conf file for **kubelet** controls various parameters for the **kubelet** service in the worker node. You should set its file ownership to maintain the integrity of the file. The file should be owned by **root:root**.

Impact:

root:root ownership is highest level of user:group security

Audit:

Method 1

Optional: Run the following command to enter **debug mode** where **NODE_NAME** and **IMAGE** are defined variables:

```
kubect1 debug node/$NODE_NAME -it --image=$IMAGE --profile=general
```

Run the following command on each node to find the appropriate kubeconfig file:

```
ps -efww | grep kubelet | grep -- '--kubeconfig'
```

The command line output should show a path for kubeconfig similar to **--kubeconfig=/etc/kubernetes/kubelet.conf** which is the location of the kubeconfig file.

Run this command to obtain the kubeconfig file permissions:

```
stat -c %U:%G /etc/kubernetes/kubelet.conf (or stat -c %U:%G /host/etc/kubernetes/kubelet.conf if in debug mode)
```

The output of the above command gives you the kubeconfig file's ownership. Verify that the ownership is set to **root:root**.

Method 2

Create and Run a Privileged Pod.

You will need to run a pod that is privileged enough to access the host's file system. This can be achieved by deploying a pod that uses the hostPath volume to mount the node's file system into the pod.

Here's an example of a simple pod definition that mounts the root of the host to /host within the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: file-check
spec:
  volumes:
  - name: host-root
    hostPath:
      path: /
      type: Directory
  containers:
  - name: nsenter
    image: busybox
    command: ["sleep", "3600"]
    volumeMounts:
    - name: host-root
      mountPath: /host
    securityContext:
      privileged: true
```

Save this to a file (e.g., file-check-pod.yaml) and create the pod:

```
kubectl apply -f file-check-pod.yaml
```

Once the pod is running, you can exec into it to check file ownership on the node:

```
kubectl exec -it -n kube-public file-check -- sh
```

Now you are in a shell inside the pod, but you can access the node's file system through the /host directory and check the ownership of the file:

```
ls -l /host/etc/kubernetes/kubelet.conf
```

The output of the above command gives you the kubeconfig file's ownership. Verify that the ownership is set to **root:root**.

Remediation:

Run the below command (based on the file location on your system) on the each worker node. For example,

```
chown root:root <kubelet.conf file>
```

Default Value:

See the OKE documentation for the default value.

References:

1. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengclustersnodes.htm>
2. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/conteng-kpo.htm>
3. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/replace-worker-node-top.htm>
4. <https://docs.oracle.com/en/learn/oci-oke-kubelet-osdaemons/index.html>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 <u>Restrict Administrator Privileges to Dedicated Administrator Accounts</u> Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.			
v7	5.2 <u>Maintain Secure Images</u> Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.			
v6	5.1 <u>Minimize And Sparingly Use Administrative Privileges</u> Minimize administrative privileges and only use administrative accounts when they are required. Implement focused auditing on the use of administrative privileged functions and monitor for anomalous behavior.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1083, T1222	TA0005, TA0007	M1026

3.1.3 Ensure that the kubelet configuration file has permissions set to 644 or more restrictive (Automated)

Profile Applicability:

- Level 1

Description:

Ensure that if the kubelet refers to a configuration file with the `--config` argument, that file has permissions of 644 or more restrictive.

Rationale:

The kubelet reads various parameters, including security settings, from a config file specified by the `--config` argument. If this file is specified you should restrict its file permissions to maintain the integrity of the file. The file should be writable by only the administrators on the system.

Impact:

None.

Audit:

Method 1

Optional: Run the following command to enter **debug mode** where `NODE_NAME` and `IMAGE` are defined variables:

```
kubectrl debug node/$NODE_NAME -it --image=$IMAGE --profile=general
```

Run the following command on each node to find the appropriate config file:

```
ps -efww | grep kubelet | grep -- '--config'
```

The command line output should show a path for config similar to `--config=/etc/kubernetes/kubelet-config.json` which is the location of the config file.

Run this command to obtain the config file permissions:

```
stat -c %a /etc/kubernetes/kubelet-config.json (or stat -c %a /host/etc/kubernetes/kubelet-config.json if in debug mode)
```

The output of the above command is the Kubelet config file's permissions. Verify that the permissions are **644** or more restrictive.

Method 2

Create and Run a Privileged Pod.

You will need to run a pod that is privileged enough to access the host's file system. This can be achieved by deploying a pod that uses the hostPath volume to mount the node's file system into the pod.

Here's an example of a simple pod definition that mounts the root of the host to /host within the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: file-check
spec:
  volumes:
  - name: host-root
    hostPath:
      path: /
      type: Directory
  containers:
  - name: nsenter
    image: busybox
    command: ["sleep", "3600"]
    volumeMounts:
    - name: host-root
      mountPath: /host
    securityContext:
      privileged: true
```

Save this to a file (e.g., file-check-pod.yaml) and create the pod:

```
kubectl apply -f file-check-pod.yaml
```

Once the pod is running, you can exec into it to check file permissions on the node:

```
kubectl exec -it -n kube-public file-check -- sh
```

Now you are in a shell inside the pod, but you can access the node's file system through the /host directory and check the permission level of the file:

```
ls -l /host/etc/kubernetes/kubelet-config.json
```

Verify that if a file is specified and it exists, the permissions are **644** or more restrictive.

Remediation:

Run the below command (based on the file location on your system) on the each worker node. For example,

```
chmod 644 <kubelet-config.json file>
```

Default Value:

See the OKE documentation for the default value.

References:

1. <https://kubernetes.io/docs/tasks/administer-cluster/kubelet-config-file/>

2. <https://docs.cloud.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengoverview.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.			
v7	14.6 Protect Information through Access Control Lists Protect all information stored on systems with file system, network share, claims, application, or database specific access control lists. These controls will enforce the principle that only authorized individuals should have access to the information based on their need to access the information as a part of their responsibilities.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1083, T1222	TA0005, TA0007	M1022

3.1.4 Ensure that the kubelet configuration file ownership is set to root:root (Automated)

Profile Applicability:

- Level 1

Description:

Ensure that if the kubelet refers to a configuration file with the `--config` argument, that file is owned by root:root.

Rationale:

The kubelet reads various parameters, including security settings, from a config file specified by the `--config` argument. If this file is specified you should restrict its file permissions to maintain the integrity of the file. The file should be writable by only the administrators on the system.

Impact:

None.

Audit:

Method 1

Optional: Run the following command to enter **debug mode** where `NODE_NAME` and `IMAGE` are defined variables:

```
kubectrl debug node/$NODE_NAME -it --image=$IMAGE --profile=general
```

Run the following command on each node to find the appropriate config file:

```
ps -efww | grep kubelet | grep -- '--config'
```

The command line output should show a path for config similar to `--config /etc/kubernetes/kubelet-config.json` which is the location of the config file.

Run this command to obtain the config file permissions:

```
stat -c %U:%G /etc/kubernetes/kubelet-config.json (or stat -c %U:%G /host/etc/kubernetes/kubelet-config.json if in debug mode)
```

The output of the above command is the Kubelet config file's ownership. Verify that the ownership is set to `root:root`

Method 2

Create and Run a Privileged Pod.

You will need to run a pod that is privileged enough to access the host's file system. This can be achieved by deploying a pod that uses the hostPath volume to mount the node's file system into the pod.

Here's an example of a simple pod definition that mounts the root of the host to /host within the pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: file-check
spec:
  volumes:
  - name: host-root
    hostPath:
      path: /
      type: Directory
  containers:
  - name: nsenter
    image: busybox
    command: ["sleep", "3600"]
    volumeMounts:
    - name: host-root
      mountPath: /host
    securityContext:
      privileged: true
```

Save this to a file (e.g., file-check-pod.yaml) and create the pod:

```
kubectl apply -f file-check-pod.yaml
```

Once the pod is running, you can exec into it to check file ownership on the node:

```
kubectl exec -it -n kube-public file-check -- sh
```

Now you are in a shell inside the pod, but you can access the node's file system through the /host directory and check the ownership of the file:

```
ls -l /etc/kubernetes/kubelet-config.json
```

The output of the above command gives you the kubelet-config.json file's ownership. Verify that the ownership is set to **root:root**.

Remediation:

Run the below command (based on the file location on your system) on the each worker node. For example,

```
chown root:root <kubelet-config.json file>
```

Default Value:

See the OKE documentation for the default value.

References:

1. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengclustersnodes.htm>
2. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/conteng-kpo.htm>
3. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/replace-worker-node-top.htm>
4. <https://docs.oracle.com/en/learn/oci-oke-kubelet-osdaemons/index.html>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1083, T1222	TA0005, TA0007	M1026

3.2 Kubelet

This section contains recommendations for kubelet configuration.

Kubelet settings may be configured using arguments on the running kubelet executable, or they may be taken from a Kubelet config file. If both are specified, the executable argument takes precedence.

To find the Kubelet config file, run the following command:

```
ps -ef | grep kubelet | grep config
```

If the **--kubecfg** argument is present, this gives the location of the Kubelet config file. This config file could be in JSON or YAML format depending on your distribution.

3.2.1 Ensure kubelet anonymous authentication is disabled (Automated)

Profile Applicability:

- Level 1

Description:

In Oracle OKE, this control means the kubelet on managed or self managed worker nodes must not accept unauthenticated requests. OKE worker nodes run kubelet to communicate with the cluster control plane, and kubelet behavior can be configured through supported worker node startup or provisioning configuration. This requirement is represented by disabling kubelet anonymous authentication rather than relying only on a process list check for `--anonymous-auth=false`.

Rationale:

The kubelet exposes node local functionality on each applicable worker node, and Kubernetes documents that requests not rejected by other configured authentication methods are otherwise treated as `system:anonymous` in the `system:unauthenticated` group. The KubeletConfiguration schema defines `authentication.anonymous.enabled` as the setting that controls this behavior, so the technically correct review point in OKE is the effective kubelet configuration on each applicable managed or self managed worker node.

Impact:

Disabling kubelet anonymous authentication causes unauthenticated requests to be rejected with 401 Unauthorized instead of being processed as anonymous access, which reduces exposure of sensitive node local kubelet endpoints and forces callers to use authenticated access paths.

Audit:

To audit whether kubelet anonymous authentication is disabled on an Oracle OKE worker node, query the live kubelet configuration through the Kubernetes API server proxy and inspect the configz output. This is preferred over relying only on the kubelet process list because kubelet settings can be applied through startup configuration, configuration files, or command line arguments.

Using the API `configz` endpoint search for the status of `authentication...`
`"anonymous":{"enabled":false}` by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; `HOSTNAME_PORT="localhost-and-port-number"` `NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"` from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &
export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=10.0.10.180 (example node name from "kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq (for a more readable output)
```

Output should show something similar to

"authentication"..."anonymous":{"enabled":false} in the output.

To interpret the result:

- **false** = Pass. Anonymous requests to the kubelet are disabled. Kubernetes states that disabling anonymous access causes unauthenticated requests to receive 401 Unauthorized.
- **true** = Fail. Requests not rejected by another authentication method can be treated as **system:anonymous** in the **system:unauthenticated** group.
- **null or missing** = Investigate further. That would be unexpected for a normal kubelet config response because **authentication.anonymous** is part of the kubelet configuration structure; verify that the request reached the kubelet successfully and that you are querying the intended node.

Remediation:

If the audit returns **true**, the worker node is allowing anonymous requests to the kubelet and should be treated as noncompliant. If the result is **null** or the field is missing, first confirm the configz response is complete, because Kubernetes identifies that endpoint as the view of the kubelet's final effective configuration. Once the result is validated, remediation in OKE should be performed through the supported worker node provisioning or node update workflow, not by directly editing the kubelet config file on the host. OKE supports custom cloud-init scripts for managed nodes, and the corrected setting should be applied by creating, cycling, or replacing worker nodes.

Remediation of new cluster

When creating a new OKE cluster or node pool, the goal is to ensure worker nodes join the cluster with kubelet anonymous authentication already disabled. This is done by defining the setting in the worker node startup or provisioning configuration that OKE applies during node creation.

- Define kubelet anonymous authentication as disabled in the worker node startup or provisioning configuration before nodes join the cluster
- For managed nodes, update the custom cloud-init script so the OKE initialization script passes **--anonymous-auth=false** through **--kubelet-extra-args**, then replace or cycle the worker nodes so the setting is applied at node startup.

- Validate the live effective kubelet configuration after deployment to confirm the setting resolves to **false**.

Remediation of existing running cluster

For an existing OKE cluster, the objective is to change the node pool or provisioning configuration and then replace the current nodes so the corrected kubelet setting is applied consistently across the node pool. This ensures the fix is durable and survives restart, scale out, and node replacement events.

- Update the custom cloud-init script or equivalent node provisioning configuration so the effective kubelet setting disables anonymous authentication.
- Apply the updated configuration to the managed node pool or self managed node build process.
- Replace, cycle, or rebuild the existing worker nodes, then re-audit the replacement nodes using the live kubelet configuration to confirm the effective value is **false**.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on OKE worker nodes are discouraged because they can be overwritten, may not match the live effective kubelet configuration, and can be lost when nodes are replaced, rebuilt, or recycled.

Default Value:

Oracle OKE documentation does not state a single default value for kubelet anonymous authentication across managed and self managed worker nodes. Verify the effective kubelet configuration on each applicable worker node, and define the setting explicitly in the supported node provisioning or startup configuration when required.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingcustomcloudinitscripts.htm>
2. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengclustersnodes.htm>
3. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/replace-worker-node-top.htm>
4. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/contengupgradingworkernodes.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 <u>Configure Data Access Control Lists</u> Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.			
v7	14.6 <u>Protect Information through Access Control Lists</u> Protect all information stored on systems with file system, network share, claims, application, or database specific access control lists. These controls will enforce the principle that only authorized individuals should have access to the information based on their need to access the information as a part of their responsibilities.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1190, T1210	TA0001, TA0008	M1025

3.2.2 Ensure kubelet authorization mode is set to Webhook (Automated)

Profile Applicability:

- Level 1

Description:

In OKE, the available kubelet authorization modes are AlwaysAllow and Webhook. For worker nodes, the recommended setting is **Webhook**, which uses the Kubernetes authorization flow to evaluate access to the kubelet API. In contrast, **AlwaysAllow** is not optimal because it bypasses that authorization decision and permits kubelet requests without a webhook-based access check. For OKE managed nodes, kubelet configuration should be validated from the effective node configuration and maintained through the supported worker node initialization and node pool update process.

Rationale:

Oracle documents that OKE managed nodes can use custom cloud-init initialization scripts and that the OKE default initialization logic should not be modified. Custom logic, including supported kubelet extra arguments, should be added before or after that default logic so worker nodes are built consistently when they join the cluster. In that model, the technically correct target is the effective kubelet configuration on the node, with **authorization.mode** set to **Webhook** instead of an unrestricted mode such as **AlwaysAllow**.

Impact:

Setting kubelet authorization mode to **Webhook** keeps kubelet access aligned with Kubernetes authorization controls on OKE worker nodes. **AlwaysAllow** is not recommended because it bypasses that authorization check and permits kubelet requests without webhook-based access evaluation.

Audit:

To audit whether kubelet authorization is set to Webhook on an OKE worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet configz output for authorization.mode. Kubernetes identifies **configz** as the view of the kubelet's final actuated configuration, and the kubelet config schema defines the valid authorization modes as **AlwaysAllow** and **Webhook**. In OKE, this is the right place to verify the control because kubelet settings should be validated from the effective worker node configuration rather than relying only on visible process flags.

Using the API configz endpoint search for the status of **authorization.*"mode": "Webhook"** by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; `HOSTNAME_PORT="localhost-and-port-number"` `NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"` from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 # example host and port number
export NODE_NAME=10.0.10.25 # example node name from kubectl get nodes

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq # for a more readable output
```

Output should show something similar to `authorization.*"mode":"Webhook"` in the output

To interpret the result:

- **Webhook** = Pass. This is the compliant result. Kubernetes documents that Webhook mode uses the `SubjectAccessReview` API to determine authorization.
- **AlwaysAllow** = Fail. This is noncompliant because requests are allowed without webhook-based authorization.
- **null or missing** = Investigate further. Recheck the raw configz output and confirm the proxy path and node name are correct, because configz is the authoritative view of the final kubelet configuration.

Remediation:

If the audit returns `AlwaysAllow`, the worker node is permitting kubelet requests without webhook-based authorization and should be treated as noncompliant. If the result is null or the field is missing, first confirm the configz response is complete, because Kubernetes identifies that endpoint as the view of the kubelet's final actuated configuration. Once the result is validated, remediation in OKE should be performed through the supported worker node initialization, node pool update, or node replacement workflow, not by directly editing the kubelet config file on the host. Oracle documents that OKE managed nodes can use custom cloud-init initialization scripts, that the OKE default logic should not be modified, and that worker node property changes apply to new worker nodes.

Remediation of new cluster

When creating a new OKE cluster or node group, the goal is to ensure worker nodes join the cluster with kubelet authorization already set to Webhook. This is done by defining the setting in the worker node initialization configuration that OKE applies during node creation.

- Define kubelet authorization mode as `Webhook` in the worker node initialization configuration so the setting is present when nodes first join the cluster.

- Use a custom cloud-init initialization script when kubelet customization is required at cluster or node group creation time. Oracle documents cloud-init as the supported path for custom initialization logic on OKE managed nodes.
- Place the setting in the supported bootstrap path, such as the OKE cloud-init initialization script using the default OKE initialization logic and supported kubelet extra arguments, so it becomes part of the node's durable build configuration.
- Validate the live effective kubelet configuration after deployment to confirm the setting resolves to Webhook.

Remediation of existing running cluster

For an existing OKE cluster, the objective is to change the node initialization or node pool configuration and then replace the current nodes so the corrected kubelet setting is applied consistently across the node group. This ensures the fix is durable and survives restart, scale-out, and node replacement events.

- Update the custom cloud-init initialization script or equivalent worker node initialization configuration so the effective kubelet setting becomes `authorization.mode: Webhook`.
- Apply the updated worker node configuration to the managed node pool so OKE launches replacement nodes with the corrected configuration.
- Replace or roll the existing nodes so the running fleet is rebuilt from the updated node pool configuration. Oracle documents that worker node property changes apply to new worker nodes and that managed nodes can be cycled or replaced so updated node pool properties are applied.
- Re-audit the replacement nodes using the live kubelet configuration to confirm the effective value is `Webhook`.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on OKE worker nodes are discouraged because OKE applies worker node initialization through cloud-init and node pool configuration, while Kubernetes allows effective values to be influenced by multiple configuration sources. As a result, a host-level edit may be overridden, may not match the live actuated configuration, and can be lost when the node is recycled or replaced.

Default Value:

Oracle OKE documentation does not publish an explicit kubelet authorization default value for managed nodes. Verify the effective value from the live kubelet `configz` output. The compliant result is `authorization.mode` set to `Webhook`.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingcustomcloudinitscripts.htm>
2. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/replace-worker-node-top.htm>

3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengupgradingk8sworkernode.htm>
4. [https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengupgradingimageworkernode_topic-Performing an InPlace Worker Node Update By Updating an Existing Node Pool.htm](https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengupgradingimageworkernode_topic-Performing_an_InPlace_Worker_Node_Update_By_Updating_an_Existing_Node_Pool.htm)

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.			
v7	9.2 Ensure Only Approved Ports, Protocols and Services Are Running Ensure that only network ports, protocols, and services listening on a system with validated business needs, are running on each system.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1133	TA0001	M1026

3.2.3 *Ensure the kubelet X.509 client CA file is configured (Automated)*

Profile Applicability:

- Level 1

Description:

This setting in OKE is about configuring the kubelet with a trusted X.509 client CA source so the kubelet can validate client certificates presented to its HTTPS endpoint. It is not the same as validating the kubelet's own serving certificate from the API server side. In OKE, kubelet authentication settings should be evaluated in the effective kubelet configuration applied during managed node initialization.

Rationale:

This setting establishes the trust anchor the kubelet uses for certificate-based client authentication at the node level. For OKE, that trust source should not be assumed from the control plane configuration alone. OKE clusters have PKI root CAs and cluster credentials, but the kubelet client CA file used for node-level X.509 client authentication should still be verified from the live kubelet configuration.

Oracle supports managed node customization through cloud-init initialization scripts, including supported kubelet extra arguments, while preserving the default OKE initialization logic. As a result, if your OKE worker node design depends on X.509 client authentication to the kubelet, the client CA file must be intentionally provisioned through the supported node initialization and node pool update path so the node starts with a consistent trust source rather than relying on manual host edits or an unsupported configuration workflow.

Impact:

If the kubelet X.509 client CA file is not configured, certificate-based clients cannot be validated against a trusted CA when connecting to the kubelet, which can break intended X.509-based access patterns or leave that authentication path unavailable. In OKE, remediation should be made through managed node initialization or node pool configuration and then applied through node replacement, because updated worker node properties are applied when new managed nodes are started or existing managed nodes are cycled and replaced.

Audit:

To audit whether kubelet authentication **X.509** client CA file is configured on an OKE worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet configz output for **authentication.x509.clientCAFile**. Kubernetes identifies configz as the view of the kubelet's final actuated configuration, and the kubelet config schema defines **authentication.x509.clientCAFile** as the setting used for **X.509** client certificate authentication. In OKE, this is the right place to verify the control because kubelet authentication settings should be validated from the effective node configuration rather than only visible process flags.

Using the api configz endpoint search for the status of **authentication...clientCAFile** by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; **HOSTNAME_PORT="localhost-and-port-number"** **NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"** from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 # example host and port number
export NODE_NAME=10.0.10.25 # example node name from kubectl get nodes

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq # for a more readable output
```

Output should show something similar to **authentication":{"x509":{"clientCAFile":"/etc/kubernetes/ca.crt"}}**

To interpret the result:

- A **non-empty** path, such as **/etc/kubernetes/ca.crt** or another valid file path, means the setting is configured. This is the expected result for this control because the kubelet has a CA bundle path defined for validating client certificates.
- An **empty** string means the field is present but not configured. Treat this as noncompliant for this control because the kubelet does not have a CA bundle path configured for X.509 client certificate validation.
- **null** or a **missing** field should be treated as investigate further. Recheck the raw configz response and confirm the proxy path and node name are correct, because **configz** is the authoritative view of the final kubelet configuration. In OKE, if the value still cannot be verified, review the managed node initialization and node pool configuration path because kubelet authentication settings should be applied there.

Remediation:

If the audit shows that the kubelet **X.509** client CA file is not configured, the worker node should be treated as noncompliant. If the result is empty, null, or missing, first confirm the inspection reflects the effective node configuration. In OKE, remediation should be performed through the supported managed node initialization, node pool update, or node replacement workflow, because Oracle supports custom cloud-init initialization scripts for managed nodes and updated node pool properties are applied when replacement managed nodes are started.

Remediation of new cluster

When creating a new OKE cluster or node group, the goal is to ensure each worker node starts with a trusted client CA bundle already present and with the kubelet configured to use it for certificate-based client authentication.

- Select and manage the CA that will anchor trust for kubelet client certificates, consistent with the cluster credential model and the node access design.
- Use a custom cloud-init initialization script for this kubelet authentication customization, since OKE uses cloud-init to initialize managed nodes and supports custom initialization logic.
- Use cloud-init or equivalent managed node initialization customization, when needed, to place the CA bundle on the node and point kubelet to that file path. Oracle documents that custom logic can be added before or after the default OKE initialization logic, but the default logic should not be modified.
- Validate after deployment that the node has the CA file at the expected path and that the kubelet is using the intended authentication configuration.

Remediation of existing running cluster

For an existing OKE cluster, the objective is to update the node initialization or node pool configuration so future nodes include the required client CA bundle and kubelet reference, then replace the current nodes so the change is applied consistently across the node group.

- Prepare an updated custom cloud-init initialization script or equivalent managed node configuration that places the CA bundle on the node and configures kubelet to use it.
- If the managed node group already uses a custom cloud-init initialization script, update the node pool with the revised initialization script so OKE can apply the change to replacement nodes.
- If the managed node group was not created with a custom initialization script, update the existing node pool or create a new node pool with the required cloud-init customization and migrate workloads to it.
- Re-audit the replacement nodes to confirm the client CA file is now configured through the intended node build path.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on OKE worker nodes are discouraged because node behavior is applied through managed node initialization, node pool configuration, and node replacement workflows, so host-level edits can be lost when nodes are recycled or replaced.

Default Value:

No fixed OKE default is documented; verify the live kubelet config.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingcustomcloudinitscripts.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengmodifyingnodepool.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/update-node-pool.htm>
4. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/replace-worker-node-top.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 Encrypt Sensitive Data in Transit Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).			
v7	14.4 Encrypt All Sensitive Information in Transit Encrypt all sensitive information in transit.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1552	TA0006	M1022

3.2.4 Ensure the kubelet read-only port is disabled (Manual)

Profile Applicability:

- Level 1

Description:

In OKE, this setting is intended to keep kubelet access limited to the standard secured management path used for node operations, instead of exposing an additional legacy interface on the worker node. In practice, it should be assessed in the node's effective kubelet configuration because OKE applies managed node behavior through cloud-init initialization, node pool configuration, and worker node replacement workflows rather than relying only on visible runtime flags.

Rationale:

Kubernetes defines `readOnlyPort` as a read-only kubelet service with no authentication or authorization, and specifies that setting `readOnlyPort: 0` disables that service. A nonzero value creates an extra listener that is separate from the main kubelet serving port.

In OKE, Oracle network configuration examples identify `TCP port 10250` as the path used by the Kubernetes API endpoint to communicate with worker nodes, so the preferred design is to keep kubelet access consolidated on that standard endpoint and leave the unauthenticated read-only service disabled. Because OKE uses cloud-init initialization scripts and node pool configuration for managed node customization, this state should be enforced through the node's managed configuration path.

Impact:

Disabling the kubelet `readOnlyPort` removes an unnecessary unauthenticated kubelet service while preserving the standard kubelet access path OKE uses on port 10250.

Audit:

To audit whether the kubelet `read-only` port is disabled on an OKE worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet configz output for `readOnlyPort` set to `0`. Kubernetes identifies configz as the view of the kubelet's final actuated configuration, and the kubelet configuration schema defines `readOnlyPort` as the setting that controls the unauthenticated read-only kubelet service. In OKE, this is the appropriate place to verify the control because kubelet settings should be validated from the effective node configuration rather than being reliably visible only as process flags.

Using the api configz endpoint search for the status of `readOnlyPort` by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; `HOSTNAME_PORT="localhost-and-port-number"` `NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"` from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 # example host and port number
export NODE_NAME=10.0.10.25 # example node name from kubectl get nodes

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq # for a more readable output
```

Output should show something similar to `readOnlyPort: 0` in the output

To interpret the result:

- **0 = Pass.** Kubernetes defines `0` as disabled, which is the expected state for this control.
- A **nonzero** integer such as `10255 = Fail`. That means the kubelet read-only service is enabled and is serving without authentication or authorization.
- **null** or a **missing** field = If `readOnlyPort` is absent from `configz`, it commonly indicates the effective value is `0`, which disables the unauthenticated read-only kubelet service.

Remediation:

If the audit returns a nonzero `readOnlyPort`, the worker node is exposing the kubelet read-only service and should be treated as noncompliant. If the field is missing from `configz`, first confirm whether it is simply omitted because the effective value is `0`, since the kubelet configuration schema defines `readOnlyPort` as the unauthenticated read-only service and states that `0` disables it. Once a noncompliant state is confirmed, remediation in OKE should be performed through the supported managed node initialization, node pool update, or node replacement workflow, not by directly editing the kubelet config file on the host. Oracle documents that OKE managed nodes use cloud-init initialization scripts, that custom logic can be added without modifying the default OKE initialization logic, and that updated worker node properties are applied by starting replacement worker nodes.

Remediation of new cluster

When creating a new OKE cluster or node group, the goal is to ensure worker nodes start with the kubelet read-only service disabled in the effective kubelet configuration. In practice, that means setting the node initialization configuration so `readOnlyPort` resolves to `0` from the time the node joins the cluster.

- Define the kubelet configuration so `readOnlyPort: 0`, which disables the unauthenticated read-only kubelet service.
- Use a custom cloud-init initialization script when kubelet customization is required at cluster or node group creation time, because OKE uses cloud-init to initialize managed nodes and supports custom initialization logic.
- Place the setting in the supported bootstrap path, such as the OKE cloud-init initialization script using kubelet extra arguments or equivalent managed node initialization customization, so it becomes part of the node's durable build configuration.
- Validate after deployment that the live kubelet configuration reflects the intended state and that the read-only service is not enabled on the node.

Remediation of existing running cluster

For an existing OKE cluster, the objective is to update the node initialization or node pool configuration so replacement nodes start with the kubelet read-only service disabled, then roll or replace the current nodes so the corrected state is applied consistently across the node group. This makes the fix durable across restart, scale-out, and node replacement events.

- Update the custom cloud-init initialization script or equivalent managed node configuration so the effective kubelet setting resolves to `readOnlyPort: 0`.
- Apply the updated managed node configuration to the node pool so OKE launches replacement nodes with the corrected configuration.
- Replace or roll the existing nodes so the running fleet is rebuilt from the updated node pool configuration, because Oracle documents that worker node property changes can be applied by replacing the original node pool with a new node pool or by cycling and replacing managed nodes.
- Re-audit the replacement nodes using the live kubelet configuration and, if needed, confirm that no service is listening on the read-only kubelet port.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on OKE worker nodes are discouraged because OKE applies node behavior through managed node initialization, node pool configuration, and node replacement workflows. A host-level edit can therefore be overridden, may not reflect the final actuated kubelet configuration, and can be lost when the node is recycled or replaced.

Default Value:

No fixed OKE default is documented; verify the live kubelet config. If `readOnlyPort` is missing from `configz`, that can still be consistent with a disabled state because zero-value fields may be omitted from serialized config output.

References:

1. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengnetworkconfigexample.htm>

2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingcustomcloudinitscripts.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/update-node-pool.htm>
4. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/replace-worker-node-top.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.8 <u>Uninstall or Disable Unnecessary Services on Enterprise Assets and Software</u> Uninstall or disable unnecessary services on enterprise assets and software, such as an unused file sharing service, web application module, or service function.			
v7	9.2 <u>Ensure Only Approved Ports, Protocols and Services Are Running</u> Ensure that only network ports, protocols, and services listening on a system with validated business needs, are running on each system.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1133	TA0001, TA0003	M1030, M1035

3.2.5 Ensure kubelet makeIPTablesUtilChains is set to true (Automated)

Profile Applicability:

- Level 1

Description:

This control is intended to keep kubelet-managed `iptables` support enabled on worker nodes so the node maintains the expected networking behavior for Kubernetes traffic handling. The recommended state is `makeIPTablesUtilChains: true`, and in OKE this should be enforced through the node's managed initialization and node pool configuration, because OKE applies managed node customization through cloud-init initialization scripts, kubelet extra args, and worker node replacement workflows.

Rationale:

OKE worker nodes rely on Kubernetes-managed `iptables` state for normal network operation. OKE guidance for NodeLocal DNSCache describes kube-proxy adding CoreDNS endpoints and `iptables` rules translating DNS service IP requests. Keeping kubelet `iptables` utility chains enabled helps preserve that expected host networking state and reduces the risk that manual or inconsistent `iptables` handling will break pod communication or service connectivity.

Impact:

Keeping kubelet `iptables` utility chains enabled helps preserve the node-level networking behavior that OKE expects for pod communication and Service routing. OKE networking behavior relies on Kubernetes components such as kube-proxy and `iptables` for service-related packet handling, so disabling this setting can make host packet handling and service connectivity less predictable.

Audit:

To audit whether the kubelet `makeIPTablesUtilChains` is enabled on an OKE worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet configz output for `makeIPTablesUtilChains`. Kubernetes identifies configz as the view of the kubelet's final actuated configuration, and the kubelet configuration schema defines `makeIPTablesUtilChains` with a default of `true`. In OKE, this is the right place to verify the control because kubelet settings should be validated from the effective node configuration rather than relying only on legacy process flags.

Using the api `configz` endpoint search for the status of `makeIPTablesUtilChains` by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; **HOSTNAME_PORT="localhost-and-port-number"** **NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"** from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 # example host and port number
export NODE_NAME=10.0.10.25 # example node name from kubectl get nodes

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq # for a more readable output
```

Output should show something similar to **makeIPTablesUtilChains: true** in the output

To interpret the result:

- **true = Pass**. This is the expected state for the control. Kubernetes defines true as enabling kubelet iptables utility chain creation, and OKE worker node networking relies on Kubernetes-managed node networking behavior for pod communication and service connectivity.
- **false = Fail**. This means kubelet iptables utility chains are disabled and the node should be treated as noncompliant. Since OKE worker node networking relies on node-level networking rules to enable pod communication, disabling related kubelet iptables support is not the preferred state.
- **null or missing** = Investigate further. Recheck the raw configz output and confirm the proxy path and node name are correct, because configz is the authoritative view of the kubelet's final actuated configuration. If the value still cannot be verified, review the node initialization and node pool customization path used by that node group.

Remediation:

If the audit returns false, the worker node is not keeping kubelet iptables utility chains enabled and should be treated as noncompliant. If the result is null or the field is missing, first confirm the configz response is complete, because Kubernetes identifies that endpoint as the view of the kubelet's final actuated configuration, and the kubelet configuration schema defines **makeIPTablesUtilChains** with a default of **true**. Once the result is validated, remediation in OKE should be performed through the supported managed node initialization, node pool update, or node replacement workflow, not by directly editing the kubelet config file on the host. Oracle documents that OKE managed nodes use cloud-init initialization scripts, that kubelet extra args can be configured through that initialization path, and that updated worker node properties are applied by starting replacement worker nodes.

Remediation of new cluster

When creating a new OKE cluster or node pool, the goal is to ensure worker nodes start with kubelet iptables utility chains enabled as part of the node's baseline configuration. In practice, that means setting the node initialization configuration so `makeIPTablesUtilChains` resolves to true when the node joins the cluster. Oracle supports cloud-init initialization scripts for managed node customization, including kubelet extra args.

- Define the kubelet configuration so `makeIPTablesUtilChains: true`, which keeps kubelet iptables utility chain creation enabled.
- Use a custom cloud-init initialization script when kubelet customization is required at cluster or node pool creation time, because OKE uses cloud-init to initialize managed nodes and supports custom initialization logic.
- Place the setting in the supported bootstrap path, such as the OKE cloud-init initialization script using kubelet extra arguments or equivalent managed node initialization customization, so it becomes part of the node's durable build configuration.
- Validate after deployment that the live kubelet configuration reflects the intended state.

Remediation of existing running cluster

For an existing OKE cluster, the objective is to update the node initialization or node pool configuration so replacement nodes start with kubelet iptables utility chains enabled, then roll or replace the current nodes so the corrected state is applied consistently across the node group. This makes the fix durable across restart, scale-out, and node replacement events. Oracle documents that worker node property changes apply to new worker nodes and can be applied by cycling, replacing, or rebuilding managed nodes.

- Update the custom cloud-init initialization script or equivalent managed node configuration so the effective kubelet setting resolves to `makeIPTablesUtilChains: true`.
- Apply the updated managed node configuration to the node pool so OKE launches replacement nodes with the corrected configuration.
- Replace or roll the existing nodes so the running fleet is rebuilt from the updated node pool configuration.
- Re-audit the replacement nodes using the live kubelet configuration to confirm the effective value is true.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on OKE worker nodes are discouraged because OKE applies node behavior through managed node initialization, node pool configuration, and node replacement workflows. A host-level edit can therefore be overridden, may not reflect the final actuated kubelet configuration, and can be lost when the node is recycled or replaced.

Default Value:

The default setting for this control is `makeIPTablesUtilChains: true`. Kubernetes defines this field as causing the kubelet to create the `KUBE-IPTABLES-HINT` chain in iptables as a hint to other components about the system's iptables configuration, and the schema lists the default as true.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingcustomcloudinitscripts.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengmodifyingnodepool.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/update-node-pool.htm>
4. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/replace-worker-node-top.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	2.5 Allowlist Authorized Software Use technical controls, such as application allowlisting, to ensure that only authorized software can execute or be accessed. Reassess bi-annually, or more frequently.			
v7	5.1 Establish Secure Configurations Maintain documented, standard security configuration standards for all authorized operating systems and software.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1190	TA0001	

3.2.6 Ensure the kubelet event recording QPS is set to an appropriate value (Automated)

Profile Applicability:

- Level 1

Description:

In OKE, this control is about tuning how quickly the kubelet records events so worker nodes capture useful operational data without generating unnecessary event volume. Oracle supports kubelet tuning for managed nodes through custom cloud-init initialization scripts and kubelet extra args, while worker node property changes are applied through node pool updates and replacement worker nodes.

Rationale:

The setting `eventRecordQPS` is the kubelet rate limit for event creation. Kubernetes defines `50` as the default and states that `0` removes the limit entirely. In OKE, this value should be managed through the supported node initialization path when customization is required, because Oracle supports custom cloud-init scripts and kubelet extra args for managed nodes.

In practice, that means a value that is too low can suppress event creation during burst conditions such as node instability, container restart loops, or rapid scheduling churn, while a value of `0` can allow unrestricted event generation and increase load on the API path and related control plane components. This setting should therefore be tuned through supported node configuration rather than treated as an arbitrary host-level change.

Impact:

Setting `eventRecordQPS` too low can throttle or suppress relevant kubelet events during burst conditions, which reduces diagnostic visibility. Leaving it at an appropriate bounded value, such as the default of `50`, helps preserve event capture while limiting event traffic. Setting it to `0` removes the rate limit entirely, which can allow unbounded event generation from the node in OKE. This can contribute to excessive event volume on the API path or related services rather than simply improving event capture.

Audit:

To audit whether the kubelet event recording QPS is appropriately set on an OKE worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet `configz` output for `eventRecordQPS`. Kubernetes identifies `configz` as the view of the kubelet's final actuated configuration, and OKE managed node customization should be validated from the effective node configuration when kubelet settings are changed through cloud-init or kubelet extra args.

Using the api configz endpoint search for the status of **eventRecordQPS** by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; **HOSTNAME_PORT="localhost-and-port-number"** **NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"** from the output of **kubectl get nodes**

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 # example host and port number
export NODE_NAME=10.0.10.25 # example node name from kubectl get nodes

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq # for a more readable output
```

Output should show something similar to **eventRecordQPS":50**

To interpret the result:

50 = Using the kubelet schema default unless it has been explicitly overridden. Kubernetes defines **50** as the default value for **eventRecordQPS**.

0 = No rate limit is enforced. This should be reviewed carefully because Kubernetes defines **0** as unlimited event creation, which can increase event volume from the worker node to the API path and related services.

Value greater than 50 = Event creation is rate-limited to that configured value and should be evaluated against the cluster's event volume and diagnostic needs.

null or missing = Investigate further by reviewing the raw **configz** response and the node initialization path, because configz is the authoritative view of the kubelet's final actuated configuration.

Remediation:

If the audit shows **eventRecordQPS** is set too low, the worker node may throttle useful event creation during burst conditions and should be adjusted. If the value is **0**, Kubernetes defines that as no limit enforced, so it should be used only when the additional event volume is intentional and downstream capacity has been considered. If the field is missing or null, first confirm the configz response is complete, because Kubernetes identifies configz as the view of the kubelet's final actuated configuration. In OKE, remediation should be made through the supported managed node initialization, node pool update, or node replacement workflow, not by directly editing the kubelet config file on the host, because Oracle supports custom cloud-init initialization scripts for managed nodes and applies updated worker node properties through new or replacement worker nodes.

Remediation of new cluster

When creating a new OKE cluster or node pool, set **eventRecordQPS** so worker nodes capture useful events without creating unnecessary downstream load. Kubernetes defines the default as **50** and **0** as unlimited, so the starting point is usually the default unless observed event loss justifies a different value. In OKE, apply this through supported managed node initialization customization such as cloud-init and kubelet extra args.

- Define **eventRecordQPS** in the node initialization configuration so the intended rate limit is applied when nodes join the cluster.
- Start from the default of **50** unless measured event loss or burst conditions justify a change.
- Avoid **0** unless unlimited event creation is intentional and monitored, because it removes the kubelet-side rate limit entirely.
- Use a custom cloud-init initialization script or equivalent supported managed node customization path when this setting must be enforced as part of node build or bootstrap behavior.
- Validate after deployment that the live kubelet configuration reflects the intended **eventRecordQPS** value.

Remediation of existing running cluster

For an existing OKE cluster, update the node initialization or node pool configuration so replacement nodes start with the corrected **eventRecordQPS** value, then roll or replace the current nodes so the change is applied consistently across the node group. Oracle documents that worker node property changes apply to new worker nodes and can be applied by cycling, replacing, or rebuilding managed nodes.

- Update the custom cloud-init initialization script or equivalent managed node configuration so the effective kubelet setting resolves to the intended **eventRecordQPS** value.
- Raise the value when relevant events are being throttled, lower it when event volume is excessive, and avoid **0** unless unlimited event creation is justified and downstream impact is understood.
- Apply the new managed node configuration to the node pool so OKE launches replacement nodes with the corrected configuration.
- If the existing node group was not created with the required managed node customization, update the existing node pool or build a new node pool with the required configuration and migrate workloads to it.
- Re-audit the replacement nodes against the live kubelet configuration and confirm the new value provides the intended balance between event capture and downstream load.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on OKE worker nodes are discouraged because the effective kubelet configuration can come from multiple sources, and OKE applies node behavior through managed node initialization, node pool configuration, and node replacement workflows. A host-level edit can therefore be overridden and lost when the node is recycled or replaced.

Default Value:

The default setting is **eventRecordQPS: 50**. Kubernetes defines **eventRecordQPS** as the maximum event creations per second and states that **0** means no limit is enforced.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingcustomcloudinitscripts.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengmodifyingnodepool.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/update-node-pool.htm>
4. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/replace-worker-node-top.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.2 Collect Audit Logs Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.			
v7	6.2 Activate audit logging Ensure that local logging has been enabled on all systems and networking devices.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1543	TA0003, TA0004	M1047

3.2.7 Ensure kubelet `tlsCertFile` and `tlsPrivateKeyFile` are configured appropriately (Automated)

Profile Applicability:

- Level 1

Description:

Setup TLS connection on the Kubelets. For OKE worker nodes, this control validates that the kubelet HTTPS serving endpoint uses an appropriate TLS certificate and private key configuration rather than relying on an unintended or unmanaged serving identity.

Rationale:

Kubelet communication contains sensitive parameters that should remain encrypted in transit. Configure the Kubelets to serve only HTTPS traffic. In OKE, the Kubernetes API endpoint communicates with worker nodes over the kubelet path, and Oracle also documents certificate based access patterns for worker node metrics. The kubelet serving certificate and private key should therefore be validated from the effective node configuration and managed through the supported OKE node initialization path when customization is required.

Impact:

TLS and client certificate authentication must be configured for your Kubernetes cluster deployment. If the kubelet serving certificate or private key is not configured appropriately, node level HTTPS communication can rely on an unintended serving identity, which can weaken trust validation and complicate secure operational access to the kubelet endpoint.

Audit:

To audit whether kubelet `tlsCertFile` and `tlsPrivateKeyFile` are configured appropriately on an OKE worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet configz output for `tlsCertFile` and `tlsPrivateKeyFile`. In OKE, this is the appropriate place to verify the control because kubelet settings should be validated from the effective node configuration rather than relying only on visible process flags.

If using the API `configz` endpoint consider searching for the status of `tlsCertFile` and `tlsPrivateKeyFile` by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; `HOSTNAME_PORT="localhost-and-port-number"` `NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"` from the output of `kubectl get nodes`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 # example host and port number
export NODE_NAME=10.0.10.4 # example node name from kubectl get nodes

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"
```

Output should show configured values for **tlsCertFile** and **tlsPrivateKeyFile**, or another validated OKE managed serving certificate configuration.

To interpret the result:

- **Configured certificate and key paths = Pass.** This means the kubelet has explicit serving certificate and private key paths defined for its HTTPS endpoint.
- **Empty, null, or missing values = Investigate further.** Confirm the raw **configz** output, the node name, and whether OKE is managing kubelet serving certificate behavior through the node initialization path. Treat the node as noncompliant if the kubelet HTTPS serving identity cannot be validated.

Remediation:

If the audit shows that the kubelet serving certificate and private key are not configured appropriately, first confirm the inspection reflects the effective node configuration. In OKE, remediation should be performed through the supported managed node initialization, node pool update, or node replacement workflow, not by directly editing the kubelet config file on the host. Oracle supports custom cloud-init initialization scripts for managed nodes, including kubelet extra args, and updated worker node properties are applied through new or replacement worker nodes.

Remediation of new cluster

When creating a new OKE cluster or node pool, ensure each worker node starts with the intended kubelet serving certificate and private key configuration. If customization is required, place the certificate, private key, and kubelet reference in the supported managed node initialization path.

- Use a custom cloud-init initialization script when kubelet customization is required at cluster or node pool creation time.
- Configure the kubelet serving certificate and private key paths through the supported initialization path, such as kubelet extra args or equivalent managed node customization.
- Validate after deployment that the live kubelet configuration reflects the intended **tlsCertFile** and **tlsPrivateKeyFile** values.

Remediation of existing running cluster

For an existing OKE cluster, update the node initialization or node pool configuration so replacement nodes start with the corrected kubelet TLS serving configuration, then roll or replace the current nodes so the change is applied consistently across the node pool.

- Update the custom cloud-init initialization script or equivalent managed node configuration so the kubelet uses the intended certificate and private key paths.
- Apply the updated managed node configuration to the node pool so OKE launches replacement nodes with the corrected configuration.
- Replace or roll the existing nodes so the running fleet is rebuilt from the updated node pool configuration.
- Re-audit the replacement nodes using the live kubelet configuration to confirm the effective **tlsCertFile** and **tlsPrivateKeyFile** values are configured appropriately.

Note: Direct edits to the kubelet config file on OKE worker nodes are discouraged because OKE applies node behavior through managed node initialization, node pool configuration, and node replacement workflows. A host-level edit can be overridden, may not reflect the final effective kubelet configuration, and can be lost when the node is recycled or replaced.

Default Value:

No fixed OKE default value is documented for **tlsCertFile** and **tlsPrivateKeyFile**. Verify the live kubelet configuration and confirm the kubelet serving certificate and private key are configured appropriately for the OKE worker node design.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingcustomcloudinitscripts.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengcreatingselfmanagednodes.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengintroducingclusteraddons.htm>
4. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/replace-worker-node-top.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 <u>Encrypt Sensitive Data in Transit</u> Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).			
v7	14.3 <u>Disable Workstation to Workstation Communication</u> Disable all workstation to workstation communication to limit an attacker's ability to move laterally and compromise neighboring systems, through technologies such as Private VLANs or microsegmentation.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1078, T1552	TA0001, TA0006	M1035, M1041

3.2.8 Ensure the kubelet client certificate rotation is enabled (Automated)

Profile Applicability:

- Level 1

Description:

In OKE, the setting `rotateCertificates` is about whether the kubelet renews its client certificate before that certificate expires. Oracle OKE documentation shows worker nodes using the kubelet client certificate at `/var/lib/kubelet/pki/kubelet-client-current.pem`, which is trusted by the cluster CA, but Oracle does not publish a fixed OKE default for `rotateCertificates`. In practice, that means true is the recommended operational state for long-lived nodes, while false or a not present setting should be treated as rotation not enabled unless OKE is explicitly managing the certificate lifecycle for that worker node configuration.

In addition, `rotateCertificates` should not be confused with a similar setting `'RotateKubeletServerCertificate'`. Client certificate rotation affects the certificate the kubelet uses when it authenticates to the Kubernetes API server. By contrast, `RotateKubeletServerCertificate` refers to the kubelet's serving certificate, which is the certificate the kubelet presents from its own HTTPS endpoint when another component connects to it. Oracle OKE documentation also distinguishes client certificate usage from CA validation by showing the kubelet client certificate and key used to authenticate to a worker node metrics endpoint, and the cluster root CA used to verify the server certificate presented by that endpoint.

Rationale:

Enabling kubelet client certificate rotation allows the kubelet to automatically generate a new key and submit a new certificate signing request through the Kubernetes API before its current client certificate expires. Oracle OKE documentation shows that required certificates are available on worker nodes after provisioning and identifies `/var/lib/kubelet/pki/kubelet-client-current.pem` as the kubelet client certificate trusted by the cluster CA. Oracle does not document a fixed `rotateCertificates` default for OKE, so the effective kubelet configuration should be verified directly. In practical terms, a setting of true means the kubelet is configured to renew its client certificate, while false or not present should be interpreted conservatively as rotation not enabled unless the platform is explicitly managing certificate renewal for you.

This setting is distinct from `RotateKubeletServerCertificate`. Client certificate rotation protects the certificate the node uses to authenticate to the API server, while server certificate rotation separately governs the certificate the kubelet presents from its HTTPS endpoint to callers.

Impact:

When kubelet client certificate rotation is enabled, long-lived OKE worker nodes are less likely to lose access to the Kubernetes API because the kubelet can renew its client certificate before it expires. When the setting is false or not present, the node depends on replacement or another renewal process to avoid certificate expiry, so the effective OKE worker node configuration should be validated rather than assumed.

Audit:

To audit whether the kubelet client certificate rotation is enabled on an OKE worker node, query the live effective kubelet configuration through the Kubernetes API server proxy and inspect the kubelet configz output for **rotateCertificates**. Kubernetes identifies configz as the view of the kubelet's final actuated configuration, and Oracle OKE documentation shows that worker nodes use kubelet client certificates trusted by the cluster CA. Oracle does not publish a fixed OKE default for this setting, so the effective kubelet configuration should be verified from the node.

Using the API **configz** endpoint search for the status of **rotateCertificates** by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name; **HOSTNAME_PORT="localhost-and-port-number"** **NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"** from the output of **kubect1 get nodes**

```
kubect1 proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 # example host and port number
export NODE_NAME=10.0.10.25 # example node name from kubect1 get nodes

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"

or

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz" |
jq # for a more readable output
```

Output should show something similar to **"rotateCertificates":true** in the output

To interpret the result:

- **true = Pass**. Client certificate rotation is enabled, so the kubelet is configured to renew its client certificate through the Kubernetes certificate lifecycle before expiry.
- **false = Fail**. Client certificate rotation is disabled. Treat this as noncompliant for long-lived OKE worker nodes unless another documented node replacement or certificate lifecycle process is in place.
- **null or missing** = Investigate further, but treat it conservatively as not enabled unless your platform explicitly manages certificate rotation for you. Oracle OKE documentation does not publish a fixed default value for rotateCertificates, so

validate the raw configz response and the node initialization path before accepting the result.

Remediation:

If the audit returns `false`, the worker node is not configured to rotate its kubelet client certificate and should be treated as noncompliant. If the field is not present, interpret that conservatively as rotation not enabled unless your OKE platform explicitly manages certificate rotation for you. Oracle OKE documentation shows worker nodes using kubelet client certificates trusted by the cluster CA and supports managed node customization through cloud-init initialization scripts and kubelet extra args. Oracle also documents that worker node property changes can be applied through new or replacement managed nodes.

Remediation of new cluster

When creating a new OKE cluster or node pool, the goal is to ensure worker nodes start with kubelet client certificate rotation enabled if the nodes may remain in service long enough for certificate expiry to matter. In OKE, this should be applied through the supported managed node initialization path rather than as a host-level edit, because Oracle supports custom cloud-init initialization scripts and kubelet extra args for managed nodes.

- Set `rotateCertificates: true` in the worker node's kubelet configuration.
- Use a custom cloud-init initialization script when this setting must be enforced at node creation.
- Apply the setting through the supported managed node initialization path, not by editing the host after deployment.
- Verify the live kubelet configuration shows rotation enabled.

Remediation of existing running cluster

For an existing OKE cluster, the objective is to update the node initialization or node pool configuration so replacement nodes start with kubelet client certificate rotation enabled, then replace the current nodes so the corrected state is applied consistently across the node pool. Oracle documents that managed nodes can be cycled, terminated, and replaced, and that updated node pool properties apply to replacement worker nodes.

- Update the cloud-init initialization script or equivalent managed node configuration so `rotateCertificates: true` is applied.
- Roll the managed node pool so replacement nodes launch with the corrected setting.
- If the node pool does not use the required managed node customization, update the existing node pool or create a new node pool with the required configuration and migrate workloads.
- Re-audit replacement nodes to confirm client certificate rotation is enabled.

Note: Even though it is tempting to do so, direct edits to the kubelet config file on OKE worker nodes are discouraged because OKE applies durable node behavior through managed node initialization, node pool configuration, and node replacement workflows. A host-level change can be bypassed by the node's initialization configuration and lost when the node is recycled or replaced.

Default Value:

By default, Oracle OKE does not document a fixed value for **rotateCertificates**. Verify the live kubelet config. The recommended secure state for long-lived worker nodes is true.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengobservingworkernodeaddonhealth.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengrotatingcredentials.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingcustomcloudinitscripts.htm>
4. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/replace-worker-node-top.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 Encrypt Sensitive Data in Transit Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).			
v7	14.4 Encrypt All Sensitive Information in Transit Encrypt all sensitive information in transit.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1552	TA0006	M1022

4 Policies

This section contains recommendations for various Kubernetes policies which are important to the security of Oracle OKE customer environment.

4.1 RBAC and Service Accounts

4.1.1 Ensure that the cluster-admin role is only used where required (Manual)

Profile Applicability:

- Level 1

Description:

Verify that cluster administrative access in Oracle OKE is granted only to identities with a documented need for full cluster administration. In OKE, this review should cover Kubernetes **ClusterRoleBinding** assignments to the **cluster-admin ClusterRole**, OCI IAM users or groups mapped through Kubernetes RBAC, and any service accounts bound to cluster administrative permissions. Where administration is limited to a team, application, or namespace boundary, use a narrower authorization model instead of cluster admin access.

Rationale:

This setting enforces least privilege across the most sensitive OKE authorization paths. Oracle documents that OCI IAM controls who can use OKE to manage clusters and obtain access to the cluster control plane, while Kubernetes RBAC controls what authenticated users and service accounts can do with the Kubernetes API inside the cluster. Oracle also documents that Kubernetes RBAC **rolebindings** and **clusterrolebindings** can map roles and **clusterroles** to users or groups, including OCI IAM users and groups by OCID. Together, these OKE behaviors make it important to keep **cluster-admin** use exceptional, tightly bounded, and reviewable.

Impact:

If **cluster-admin** is assigned too broadly in OKE, a compromised OCI IAM user, group mapping, or overprivileged service account can gain effective control over resources across all namespaces, increasing the chance that a single credential exposure or node level compromise becomes a cluster security or availability incident. The risk is greater when the affected workload also has OCI permissions through workload identity, because exposure can extend beyond Kubernetes RBAC into OCI API access.

Audit:

This audit reviews Kubernetes **ClusterRoleBinding** objects in the Oracle OKE cluster to identify principals that are directly granted the **cluster-admin ClusterRole**. It lists each matching binding and its associated users, groups, or service accounts so the reviewer can verify whether full cluster administrative access is justified. Because OKE uses OCI IAM for access to OKE and the cluster control plane, and Kubernetes RBAC for cluster authorization, this check should be used as a partial review of administrative privilege rather than the sole test of cluster-admin exposure.

```
kubectl get clusterrolebindings -o json | jq -r '
  ([ "BINDING", "KIND", "NAMESPACE", "NAME" ] | join(" | ")),
  (
    .items[]
    | select(.roleRef.name=="cluster-admin")
    | .metadata.name as $binding
    | (.subjects // [])[]
    | [$binding, .kind, (.namespace // "-"), .name]
    | join(" | ")
  )
'
```

Example output with headers and piped separation of values. Review each principal listed and ensure that **cluster-admin** privilege is required for it.

```
BINDING | KIND | NAMESPACE | NAME
cluster-admin | Group | - | system:masters
my-admin-binding | User | - | admin-user
ci-cluster-admin | ServiceAccount | cird | deploy-bot
oke-admin | ServiceAccount | kube-system | oke-admin
```

Remediation:

Review each **ClusterRoleBinding** identified by the audit that grants the Kubernetes **cluster-admin ClusterRole**, and determine whether the listed user, group, or service account still requires full cluster administrative access. In Oracle OKE, this review should also account for access granted through OCI IAM policies and Kubernetes RBAC mappings, because Oracle documents that IAM and the Kubernetes RBAC Authorizer work together to authorize Kubernetes operations. Where responsibilities are limited to a namespace, workload, or operational function, replace cluster administration with a narrower authorization model.

- Review each principal returned by the audit and confirm whether full cluster administrative access is still justified for that identity.
- For OCI IAM users or groups mapped through Kubernetes RBAC, remove unnecessary **cluster-admin** bindings and replace them with narrower Kubernetes RBAC permissions or an OCI IAM and RBAC model aligned to the approved operational need.
- For principals granted access through Kubernetes RBAC, replace unnecessary **cluster-admin** bindings with a lower-privilege **Role**, **ClusterRole**, or namespace **RoleBinding** that matches the approved operational need.
- Validate that the replacement access allows the user, group, or service account to perform its approved tasks before removing the original **cluster-admin** binding.
- Remove the unneeded **ClusterRoleBinding** only after the reduced privilege access has been confirmed to work as intended.
- Do not remove **ClusterRoleBindings** required for OKE cluster operations, Oracle managed add-ons, or approved administrative service accounts unless you have validated the operational impact.

- Review privilege OCI IAM groups separately, because OCI IAM controls access to manage OKE clusters and obtain access to the cluster control plane.

Where possible, first assign a lower privilege role, validate required access, and then remove the **ClusterRoleBinding** that grants **cluster-admin**:

```
kubectl delete clusterrolebinding [name]
```

Default Value:

By default, **cluster-admin** is bound to system in Kubernetes RBAC. Oracle OKE also uses OCI IAM to control access to manage clusters and obtain access to the cluster control plane, while Kubernetes RBAC controls what authenticated users and service accounts can do inside the cluster.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutaccesscontrol.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengclustersnodes.htm>
3. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengpolicyconfig.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contenggrantingworkloadaccessstoresources.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.8 Define and Maintain Role-Based Access Control Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1078, T1078.002	TA0001, TA0004	M1026

4.1.2 Ensure access to Secrets is minimized and granted only where required (Manual)

Profile Applicability:

- Level 1

Description:

Verify that access to Kubernetes Secret objects in Oracle OKE is granted only to OCI IAM users or groups, Kubernetes users or groups, and service accounts with a defined operational or workload need. The review should cover OCI IAM access to the cluster, Kubernetes RBAC roles and bindings, OCI IAM groups mapped through Kubernetes RBAC, workload identities, and service account permissions, with particular attention to permissions that allow get, list, or watch access to Secrets. Where access is needed, scope it as narrowly as possible, because OKE uses OCI IAM for cluster access and Kubernetes RBAC for fine grained cluster authorization.

Rationale:

This requirement reduces unnecessary exposure to one of the most sensitive resource types in an OKE cluster. Oracle documents that OCI IAM controls who can use OKE to manage clusters and access the cluster control plane, while Kubernetes RBAC controls what authenticated users and service accounts can do inside the cluster. Oracle also documents that Kubernetes RBAC **roles**, **clusterroles**, **rolebindings**, and **clusterrolebindings** can be used to grant fine grained permissions to users and groups, including OCI IAM users and groups by OCID. Minimizing Secrets access therefore helps prevent broad read access from being granted through overlapping OCI IAM, Kubernetes RBAC, and service account authorization paths.

For workloads, least privilege access should be applied to both Kubernetes service accounts and OCI IAM policies. OKE workload identity identifies a workload by the unique combination of cluster, namespace, and service account, and can be used to grant fine grained access to OCI resources. Excessive Secrets access combined with overprivileged workload identities can then expand a pod or node compromise beyond Kubernetes RBAC into OCI API access.

Impact:

If access to Secrets is granted too broadly in OKE, a compromised OCI IAM principal, Kubernetes subject, or service account can retrieve sensitive cluster data and use it to expand the scope of an incident across namespaces or into OCI services when workload OCI permissions are also available. That increases the likelihood that a single credential exposure, pod compromise, or node level breach becomes a wider security event.

Audit:

Start by reviewing every path that can grant access to Kubernetes Secret objects in the OKE cluster. Start with OCI IAM access to the cluster and Kubernetes RBAC permissions, because OKE uses OCI IAM to control access to OKE and the cluster control plane, while Kubernetes RBAC controls cluster authorization. Then review Kubernetes RBAC, service account permissions, and workload identities, because OKE can combine cluster access, Kubernetes RBAC, and OCI resource access through service account based workload identity.

- Review OCI IAM policies and group membership for principals that can access or manage the OKE cluster, then confirm whether those principals receive Kubernetes RBAC permissions that expose Secrets.
- Review Kubernetes RBAC roles and bindings that grant **get**, **list**, or **watch** on **secrets**, and determine which users, groups, and service accounts actually receive that access. Give extra scrutiny to cluster roles and broadly shared bindings.
- Review workload service accounts that can access Secrets and confirm that each one has a defined workload need. Pay particular attention to broadly deployed workloads and privileged operational service accounts, because unnecessary service account permissions can increase exposure if a pod or node is compromised.
- Review OCI IAM group mappings and Kubernetes RBAC bindings, because OKE supports mapping OCI IAM users and groups to Kubernetes RBAC **rolebindings** and **clusterrolebindings** that can later receive Secrets access.
- Confirm that each access path is tied to a documented requirement to the smallest practical namespace or function, and no broader than necessary. Flag any unnecessary Secrets access through OCI IAM, RBAC, or service accounts for remediation.

Summary: Review OCI IAM access, Kubernetes RBAC, workload identities, and service accounts to verify that Secrets access is approved, minimally scoped, and granted only where required.

Remediation:

Start by reducing every access path that allows reading Kubernetes Secret objects in the OKE cluster. Start with OCI IAM access and Kubernetes RBAC permissions, because OCI IAM controls access to OKE and the cluster control plane, while Kubernetes RBAC controls access to Kubernetes resources such as Secrets. Then reduce unnecessary access in Kubernetes RBAC, service accounts, and workload identity policies, and replace broad permissions with the narrowest namespace or workload scope that still supports the required function.

- Remove or narrow OCI IAM and Kubernetes RBAC permissions that expose Secrets, and prefer namespace access over cluster scope wherever possible.

- Replace broad RBAC permissions on secrets with the smallest practical **Role** or **ClusterRole**, and remove unnecessary **get**, **list**, or **watch** access from users, groups, and service accounts.
- Reduce service account access to Secrets to only the workloads that require it, and tighten any related workload identity OCI IAM permissions to least privilege.
- Review broadly deployed workloads and privileged operational service accounts first, because their tokens and permissions can increase exposure if the workload or node is compromised.
- Remove stale OCI IAM group mappings or Kubernetes RBAC bindings that still place principals into groups with Secrets access.
- Validate that each remaining Secrets access path is tied to a documented operational or workload requirement, and flag any unnecessary access for removal.

Default Value:

By default, OKE uses OCI IAM to control access to OKE and the cluster control plane, while access to Kubernetes Secrets is controlled through Kubernetes RBAC roles, bindings, and service account permissions rather than a single cluster setting.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutaccesscontrol.htm>
2. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengpolicyconfig.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contenggrantingworkloadaccesstoresources.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengmanagingsecrets.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.1 Establish and Maintain a Secure Configuration Process Establish and maintain a secure configuration process for enterprise assets (end-user devices, including portable and mobile, non-computing/IoT devices, and servers) and software (operating systems and applications). Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1552	TA0006	M1026

4.1.3 Minimize wildcard use in Roles and ClusterRoles (Manual)

Profile Applicability:

- Level 1

Description:

Verify that Kubernetes **Role** and **ClusterRole** objects in Oracle OKE do not use wildcard entries such as ***** for verbs, resources, or apiGroups unless a documented administrative or platform requirement exists. The review should also include the **RoleBinding** and **ClusterRoleBinding** objects that assign those roles to users, groups, or service accounts, because OKE uses OCI IAM to control access to OKE and the cluster control plane, while Kubernetes RBAC controls what authenticated users and service accounts can do through the Kubernetes API inside the cluster.

Rationale:

This recommendation supports least privilege authorization in OKE Kubernetes RBAC. Oracle documents that OCI IAM controls who can use OKE to manage clusters and obtain access to the cluster control plane, while Kubernetes RBAC controls what authenticated users and service accounts can do inside the cluster.

Oracle also documents that Kubernetes RBAC **roles**, **clusterroles**, **rolebindings**, and **clusterrolebindings** can be used to grant specific permissions to users and groups, including OCI IAM users and groups by OCID. In that model, wildcard rules make the effective privilege set harder to predict, harder to review, and easier to overextend across current and future resources. The risk is higher for service accounts used by broadly deployed controllers or platform components, because those Kubernetes RBAC permissions can be exposed if the workload, pod, or node is compromised.

Impact:

If wildcard permissions are used broadly in OKE, a user or service account can receive more access than intended across namespaces, resource types, or API actions, and that excess access can combine with other OCI IAM permissions or workload identity access to widen the impact of compromise. When the affected identity is a broadly deployed service account or one linked to OKE workload identity, an excessive RBAC rule can turn a single pod or node issue into unauthorized cluster changes or access to OCI APIs beyond the workload's intended boundary.

Audit:

Begin by reviewing custom Kubernetes **Role** and **ClusterRole** objects in OKE to identify any use of wildcard entries such as ***** for verbs, resources, or apiGroups. In OKE, this review should cover native RBAC objects, OCI IAM users and groups mapped through Kubernetes RBAC, service accounts, workload identities, and Oracle managed platform RBAC objects that should be distinguished from customer managed permissions.

Review custom **Role** and **ClusterRole** definitions for wildcard use in verbs, resources, or apiGroups, and prioritize **ClusterRoles** because they can grant permissions across the cluster.

```
kubectl get roles,clusterroles -A -o json | jq -r '
  ([ "KIND", "NAMESPACE", "NAME", "WILDCARD_FIELDS" ] | join(" | ")),
  (
    .items[]
    | . as $r
    | [
      (if any(.rules[]?; any(.verbs[]?; .=="*")) then "verbs" else empty
end),
      (if any(.rules[]?; any(.resources[]?; .=="*")) then "resources" else
empty end),
      (if any(.rules[]?; any(.apiGroups[]?; .=="*")) then "apiGroups" else
empty end)
    ] as $wild
    | select(($wild | length) > 0)
    | [
      $r.kind,
      ($r.metadata.namespace // "-"),
      $r.metadata.name,
      ($wild | join(", "))
    ]
    | join(" | ")
  )
,'
```

Sample Output:

KIND	NAMESPACE	NAME	WILDCARD_FIELDS
ClusterRole	-	cluster-admin	verbs,resources,apiGroups
ClusterRole	-	system:controller	apiGroups
ClusterRole	-	system:controller	resources,apiGroups
ClusterRole	-	system:controller	resources,apiGroups
ClusterRole	-	system:controller	resources,apiGroups
ClusterRole	-	system:controller	resources,apiGroups
ClusterRole	-	system	resources,apiGroups
ClusterRole	-	system	verbs

- Review the **RoleBinding** and **ClusterRoleBinding** assignments for those roles to identify which users, groups, and service accounts actually receive the wildcard permissions, and separate Oracle managed and Kubernetes system bindings from customer managed ones during the review.

- Review OCI IAM users and groups mapped through Kubernetes RBAC, because OKE supports mapping OCI IAM users and groups to Kubernetes **rolebindings** and **clusterrolebindings** by OCID.
- Review service accounts for broadly deployed controllers, storage drivers, ingress controllers, and other platform components first, because their permissions can affect many workloads or nodes.
- Review workload identities, because OKE can grant OCI resource access to workloads based on the unique combination of cluster, namespace, and service account.
- Confirm that each wildcard rule has a documented operational or platform need. Where it does not, flag it for remediation and replace it with enumerated resources, verbs, API groups, or roles limited to the required namespace or function. OKE uses Kubernetes RBAC for authorization inside the cluster, so wildcard permissions should remain rare and justified.

Summary: Review custom RBAC roles, bindings, OCI IAM mappings, workload identities, and service accounts to verify that wildcard permissions are rare, justified, and no broader than required.

Remediation:

Start by replacing wildcard permissions in custom **Role** and **ClusterRole** objects with explicitly enumerated verbs, resources, and apiGroups that match the approved operational need. In OKE, this remediation should focus on customer managed RBAC objects, the bindings that assign them, OCI IAM users and groups mapped through Kubernetes RBAC, workload identities, and service accounts used by workloads or cluster components. OKE uses OCI IAM for access to OKE and the cluster control plane, and Kubernetes RBAC for authorization inside the cluster, so both access paths should be reviewed when excessive permissions are found.

- Replace wildcard values in custom **Role** and **ClusterRole** rules with the specific verbs, resources, and apiGroups the function actually requires.
- Reduce or remove any **RoleBinding** and **ClusterRoleBinding** that assign those wildcard roles more broadly than necessary, and prefer bindings limited to the required namespace where possible.
- Review OCI IAM users and groups mapped through Kubernetes RBAC, because those users and groups can receive wildcard permissions through **rolebindings** or **clusterrolebindings**.
- Review service accounts for broadly deployed controllers, storage drivers, ingress controllers, and other platform components first, and remove wildcard permissions that are not required for their runtime behavior.
- Distinguish customer managed roles from Oracle managed and Kubernetes system roles before making changes, and avoid modifying Oracle managed or Kubernetes system RBAC objects unless you intentionally want to change how a platform component operates.

- After narrowing the role definition, validate that the user, group, or service account can still perform its approved tasks, then remove the superseded wildcard role or binding.

Default Value:

By default, OKE uses OCI IAM to control access to OKE and the cluster control plane, while Kubernetes RBAC roles and bindings control what authenticated users and service accounts can do inside the cluster. Wildcard permissions are not a single OKE cluster setting and should be reviewed in **Role** and **ClusterRole** rules.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutaccesscontrol.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengclustersnodes.htm>
3. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengpolicyconfig.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contenggrantingworkloadaccesstoresources.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.8 Define and Maintain Role-Based Access Control Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1078, T1078.002	TA0001, TA0004	M1026

4.1.4 Minimize access to create pods (Manual)

Profile Applicability:

- Level 1

Description:

The ability to create Pods in OKE is a sensitive permission that should be limited to approved deployment and operational functions. In OKE, Pod creation is controlled through Kubernetes RBAC after OCI IAM allows access to the cluster control plane. Because Pods run within a namespace, use a service account, and can carry workload permissions, Pod creation should be limited to the smallest practical namespace and workload boundary.

Rationale:

Creating a Pod can allow a principal to run code with the permissions of the service account assigned to that Pod. OKE uses Kubernetes service accounts for workloads, and Kubernetes RBAC controls what authenticated users and service accounts can do with the Kubernetes API inside the cluster. Those behaviors make Pod creation more than a routine write permission, because it can become a path to exercise Kubernetes API permissions indirectly through workload identity.

The risk can extend beyond Kubernetes RBAC into OCI access. With OKE workload identity, a workload is identified by the unique combination of cluster, namespace, and service account, allowing IAM policies to grant that workload access to OCI resources. OKE pod security guidance also identifies higher risk Pod behaviors such as privileged execution and hostPath usage, which means the ability to create Pods can also expose host or node attack paths if security controls do not constrain the Pod spec.

Impact:

If Pod creation is granted too broadly in OKE, a user or workload can launch Pods under available service accounts and gain unintended Kubernetes or OCI access, increasing the risk of privilege escalation and wider cluster impact.

Audit:

Begin by reviewing every path that can grant the ability to create Pods in OKE. Start with OCI IAM access to the cluster and Kubernetes RBAC permissions, because OCI IAM controls access to OKE and the cluster control plane, while Kubernetes RBAC controls authorization inside the cluster. Then review Kubernetes RBAC, service accounts, and Pod security controls, because Pod creation can allow an identity to launch workloads under available service accounts and potentially exercise both Kubernetes and OCI permissions.

- Review OCI IAM policies and group membership for principals that can access or manage the OKE cluster, then confirm whether those principals receive

Kubernetes RBAC permissions that allow create on pods. Check whether access is limited to the required namespace or function.

- Review **Role** and **ClusterRole** assignments that allow create on pods, and also review **deployments**, **daemonsets**, **statefulsets**, **jobs**, and **cronjobs**, because those resources can launch Pods through controllers. Map each role to the principals that receive it.
- Review service accounts used by workloads that can create or run Pods. Check for use of the default service account, and confirm whether token auto mounting is disabled where Kubernetes API access is not needed.
- Review workloads that use OKE workload identity, because a Pod created under a linked service account can obtain OCI access in addition to Kubernetes API access. Confirm the OCI IAM policy is limited to the intended workload.
- Review whether Pod creation can be paired with privileged execution, **hostPath**, or **hostNetwork**, and confirm that Pod Security Admission or policy as code controls restrict those settings where required.
- Review OCI IAM group mappings and Kubernetes RBAC bindings, because OKE supports mapping OCI IAM users and groups to Kubernetes RBAC **rolebindings** and **clusterrolebindings** by OCID.

Summary: Review OCI IAM access, Kubernetes RBAC, service accounts, workload identity, and Pod controls to verify that Pod creation is limited and granted only where required.

Remediation:

Start by reducing every access path that allows Pod creation in OKE. Start with OCI IAM access and Kubernetes RBAC permissions, because OCI IAM controls access to OKE and the cluster control plane, while Kubernetes RBAC controls access to Kubernetes resources such as Pods. Then reduce unnecessary permissions in Kubernetes RBAC, service accounts, and Pod security controls so Pod creation is limited to approved deployment paths and the smallest practical workload boundary.

- Remove or narrow OCI IAM and Kubernetes RBAC permissions that allow Pod creation, and prefer namespace limited access wherever possible.
- Replace broad RBAC permissions on **pods**, **deployments**, **daemonsets**, **statefulsets**, **jobs**, and **cronjobs** with the smallest **Role** or **ClusterRole** that supports the approved function.
- Move workloads off the default service account, use dedicated service accounts, and disable **automountServiceAccountToken** where Kubernetes API access is not needed.
- Tighten OKE workload identity OCI IAM permissions for any service account that can run Pods so the linked OCI access is least privilege and specific to the workload.
- Restrict Pod specs that can increase the impact of Pod creation, including privileged mode, **hostPath**, and similar settings, by using Pod Security Admission or policy as code controls.

- Review DaemonSet and other broadly deployed workloads first, because a service account used across many nodes can widen the impact of an overprivileged Pod creator.
- Remove stale OCI IAM group mappings or Kubernetes RBAC bindings where applicable so Pod creation is not still reachable through unneeded group mappings.

Default Value:

By default, OKE uses OCI IAM to control access to OKE and the cluster control plane, while Kubernetes RBAC controls who can create Pods inside the cluster. Pod creation is not controlled by a single OKE cluster setting and should be reviewed through Kubernetes **Role**, **ClusterRole**, **RoleBinding**, **ClusterRoleBinding**, and service account permissions.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutaccesscontrol.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengclustersnodes.htm>
3. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengpolicyconfig.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contenggrantingworkloadaccesstoresources.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.8 Define and Maintain Role-Based Access Control Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●
v7	5.1 Establish Secure Configurations Maintain documented, standard security configuration standards for all authorized operating systems and software.	●	●	●

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1078, T1078.002	TA0001, TA0004	M1026

4.1.5 Ensure that default service accounts are not actively used. (Automated)

Profile Applicability:

- Level 1

Description:

Default service accounts in OKE should not be used as the active identity for ordinary application or automation workloads. If a Pod does not specify a service account, Kubernetes automatically assigns the namespace default service account, and that account can authenticate to the Kubernetes API through its mounted token. Workloads that need Kubernetes or OCI access should instead use an explicitly assigned, dedicated service account that matches the intended namespace, application, and permission boundary.

Rationale:

This supports least privilege at the workload identity layer by using a dedicated service account for each application, including workloads that need Kubernetes API access or OKE workload identity. This allows RBAC and workload identity to be limited to the specific resources and actions required by each application, instead of sharing one namespace identity across unrelated Pods.

The default service account is especially risky when Pods inherit it implicitly. In OKE, workloads are automatically assigned a service account if one is not specified, and service account tokens are automatically mounted unless token auto mounting is disabled. For workloads that need OCI access, OKE workload identity uses the unique combination of cluster, namespace, and service account to grant access to OCI resources, so relying on the default account makes it harder to isolate OCI permissions cleanly and increases the chance that multiple workloads share the same Kubernetes and OCI identity path.

Impact:

If the default service account is actively used in OKE, multiple Pods in the same namespace can inherit the same Kubernetes identity and any linked OCI permissions, which increases security impact, weakens workload isolation, and reduces audit clarity when a Pod is compromised or misconfigured.

Audit:

Begin by identifying workloads in OKE that implicitly or explicitly use the namespace default service account. This matters because Pods that do not specify `serviceAccountName` are assigned the default service account, workloads that need OCI access should use a dedicated service account for OKE workload identity, and workloads that do not need Kubernetes API access can disable token auto mounting with `automountServiceAccountToken: false`.

Review `Deployments`, `StatefulSets`, `DaemonSets`, `Jobs`, `CronJobs`, and standalone Pods for omitted `serviceAccountName` values or explicit use of default.

```
kubectl get deploy,sts,ds,job,cronjob,pod -A -o json | jq -r '
  (["NAMESPACE","KIND","NAME","SERVICEACCOUNT"] | @tsv),
  (
    .items[]
    | if .kind=="CronJob" then
      [.metadata.namespace, .kind, .metadata.name,
      (.spec.jobTemplate.spec.template.spec.serviceAccountName // "default")]
    elif .kind=="Pod" then
      [.metadata.namespace, .kind, .metadata.name,
      (.spec.serviceAccountName // "default")]
    else
      [.metadata.namespace, .kind, .metadata.name,
      (.spec.template.spec.serviceAccountName // "default")]
    end
    | select(.[3]=="default")
    | @tsv
  )
' | column -ts $'\t'
```

Sample output:

NAMESPACE	KIND	NAME	SERVICEACCOUNT
kube-public	Pod	file-check	default
default	Deployment	web-app	default
apps	StatefulSet	orders-db	default
logging	DaemonSet	fluentd	default
batch	Job	nightly-report	default

Review the default service account in each namespace and confirm it is not bound to unnecessary `RoleBinding` or `ClusterRoleBinding` permissions.

Review workload specs for `automountServiceAccountToken: false` and confirm it is set wherever the workload does not need Kubernetes API access. Use the command below to find the `automountServiceAccountToken` setting:

```
kubectl get serviceaccount $SERVICE_ACCOUNT -n $NAMESPACE -o yaml
```

Review workloads that use OKE workload identity and confirm they use a dedicated service account rather than the namespace default service account.

Flag any workload that still uses the default service account or still auto mounts tokens without a documented need for remediation.

Remediation:

Create dedicated service accounts for workloads that need Kubernetes API access or OKE workload identity, and update those workloads to explicitly set `serviceAccountName` rather than relying on the namespace default service account. For workloads that do not need Kubernetes API access, disable service account token auto mounting and avoid using the default service account as the active workload identity. Also review the default service account in each namespace and ensure it is not bound to unnecessary RBAC permissions.

Modify the default service account in each application namespace to set:

```
automountServiceAccountToken: false
```

Then validate workload manifests, because a Pod `automountServiceAccountToken` setting can override the service account setting, and workloads that actually need Kubernetes API access must use an explicitly assigned service account instead of inheriting default.

Automatic remediation for one namespace:

```
kubectl patch serviceaccount default -n -p '{"automountServiceAccountToken": false}'
```

Default Value:

By default, the default service account allows for its service account token to be mounted in pods in its namespace. OKE uses OCI IAM to control access to OKE and the cluster control plane, while Kubernetes RBAC and service account permissions control what workloads can do inside the cluster. OKE workload identity grants OCI resource access based on the combination of cluster, namespace, and service account.

References:

1. https://docs.oracle.com/iaas/Content/Security/Reference/oke_security.htm
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutaccesscontrol.htm>
3. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengpolicyconfig.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contenggrantingworkloadaccessresources.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.3 Disable Dormant Accounts Delete or disable any dormant accounts after a period of 45 days of inactivity, where supported.			

Controls Version	Control	IG 1	IG 2	IG 3
v7	16.9 <u>Disable Dormant Accounts</u> Automatically disable dormant accounts after a set period of inactivity.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1552	TA0006	M1028

4.1.6 Ensure that Service Account Tokens are only mounted where necessary (Automated)

Profile Applicability:

- Level 1

Description:

Service account tokens in OKE should be mounted only for Pods that require Kubernetes API access or an explicitly configured workload identity flow. Kubernetes supports disabling automatic token mounting at the **Pod** or **ServiceAccount** level, and workloads that need OCI access through OKE workload identity should use a dedicated service account rather than relying on broad default token mounting.

Rationale:

This setting is important because a mounted service account token is an authentication credential. Oracle documents that Kubernetes RBAC controls what authenticated users and service accounts can do inside the cluster, and that a service account binds pods in a workload to Kubernetes role permissions and grants access to Kubernetes resources. OKE also allows this behavior to be controlled either on the **Pod** spec or on the **ServiceAccount**, which supports making token mounting an intentional workload decision instead of a default behavior.

The risk is higher when workload identity is involved. With OKE workload identity, a workload is identified by the unique combination of cluster, namespace, and service account, and IAM policies can grant that workload access to OCI resources. Oracle workload identity examples also require the workload deployment to specify the **serviceAccountName** and set **automountServiceAccountToken** to **true**. If service account tokens are mounted more broadly than needed, unnecessary workloads can receive credentials for Kubernetes API access or an approved OCI workload identity path.

Impact:

If service account tokens are mounted more broadly than necessary in OKE, more Pods than required receive credentials that can be used for Kubernetes API access and, in some cases, OCI API access, increasing the security impact of a compromised or misconfigured workload.

Audit:

Begin by reviewing where service account tokens are mounted in OKE and whether each mounted token supports a defined Kubernetes API or workload identity requirement. In OKE, Pods can receive a service account token automatically, `automountServiceAccountToken` can be disabled at the Pod or `ServiceAccount` level, and workloads that need OCI access through OKE workload identity should use a dedicated service account instead of broad default token mounting.

- Review `Deployments`, `StatefulSets`, `DaemonSets`, `Jobs`, `CronJobs`, and standalone `Pods` for `serviceAccountName` and `automountServiceAccountToken`, and identify workloads that still rely on default token mounting.
- Review the `automountServiceAccountToken` setting on both the Pod and its associated `ServiceAccount`, and treat the Pod setting as authoritative where both are defined.

```
kubectl get serviceaccount $SERVICE_ACCOUNT -n $NAMESPACE -o yaml  
kubectl get pod $POD_NAME -n $NAMESPACE -o yaml
```

Expected Output:

```
automountServiceAccountToken: false
```

- Review workloads that use OKE workload identity and confirm they use a dedicated service account tied to the intended application and permission boundary.
- Review `RoleBindings` and `ClusterRoleBindings` for service accounts that still mount tokens, and confirm the bound RBAC permissions are limited to the API resources and actions the workload actually needs.
- Review `DaemonSets`, storage drivers, ingress controllers, and other broadly deployed Pods first, because service account permissions and projected tokens associated with those workloads can become reachable from any node where they run.
- Review Pods that do not need Kubernetes API access and confirm token mounting is disabled, then flag any unnecessary token mount for remediation.

Remediation:

Apply the following steps to reduce unnecessary service account token mounting in OKE:

- Set `automountServiceAccountToken` to `false` for `Pods` and `ServiceAccounts` that do not require Kubernetes API access. This reduces unnecessary default token auto mounting.
- Update workloads to use a dedicated `serviceAccountName` only where a service account token is operationally required, instead of inheriting token

behavior from the namespace default service account. Each application should use its own dedicated service account.

- For workloads that need OCI access, use OKE workload identity with a dedicated service account rather than broad default token mounting. OKE workload identity uses the combination of cluster, namespace, and service account to grant access to OCI resources for the intended workload.
- Revalidate workloads after the change to confirm that only Pods with a documented Kubernetes API or workload identity need continue to receive service account tokens.

Default Value:

By default, all pods get a service account token mounted in them.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutaccesscontrol.htm>
2. https://docs.oracle.com/iaas/Content/Security/Reference/oke_security.htm
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contenggrantingworkloadaccesstoresources.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengaddingserviceaccttoken.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.1 Establish and Maintain a Secure Configuration Process Establish and maintain a secure configuration process for enterprise assets (end-user devices, including portable and mobile, non-computing/IoT devices, and servers) and software (operating systems and applications). Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.			
v7	5.1 Establish Secure Configurations Maintain documented, standard security configuration standards for all authorized operating systems and software.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1528, T1555	TA0006	M1026

4.2 Pod Security Policies

Pod Security Standards (PSS) are recommendations for securing deployed workloads to reduce the risks of container breakout. There are a number of ways if implementing PSS, including the built-in Pod Security Admission controller, or external policy control systems which integrate with Kubernetes via validating and mutating webhooks.

The previous feature described in this document, pod security policy (preview), was deprecated with version 1.21, and removed as of version 1.25. After pod security policy (preview) is deprecated, you must disable the feature on any existing clusters using the deprecated feature to perform future cluster upgrades and stay within Oracle support.

4.2.1 Minimize the admission of privileged containers (Automated)

Profile Applicability:

- Level 1

Description:

Minimize the admission of privileged containers in OKE by enforcing Pod Security Standards through Pod Security Admission or an equivalent policy engine, and restrict privileged workloads to only the limited system or platform namespaces that truly require them.

Do not generally permit containers to run with `securityContext.privileged: true` in OKE clusters, except for narrowly approved system or platform workloads.

Rationale:

In OKE, a privileged container inherits all Linux capabilities associated with root on the host, which significantly weakens container isolation and increases the chance that a compromised workload can interact with the underlying node in unsafe ways. Oracle documents that pod security controls can prevent pods from running in privileged mode, and that current OKE clusters should use Kubernetes Pod Security Standards and the `PodSecurity` admission controller where Pod Security Policy functionality is required. Using baseline or restricted enforcement helps block privilege escalation patterns during admission rather than relying only on runtime detection after the pod has started.

If you need to run privileged containers, this should be defined in a separate policy and you should carefully check to ensure that only limited service accounts and users are given permission to use that policy.

Impact:

Workloads that depend on privileged mode may be denied admission in namespaces enforcing Pod Security Standards, so certain node level agents, device access workloads, or other specialized platform pods may need redesign, tighter access, or isolated placement.

Pods with `spec.containers[].securityContext.privileged: true`, `spec.initContainers[].securityContext.privileged: true`, or `spec.ephemeralContainers[].securityContext.privileged: true` will not be permitted in namespaces enforcing the baseline or restricted Pod Security Standard.

Audit:

Review the effective Pod Security Admission configuration for every namespace in the OKE cluster and verify that workload namespaces are set to a policy level that disallows privileged containers. For this control, confirm that workload namespaces are not unlabeled, not set to privileged, and not relying only on `warn` or `audit` where blocking is required. The `enforce` label should be used where admission blocking is required.

Then inspect existing pods for admitted use of `spec.containers[].securityContext.privileged`, `spec.initContainers[].securityContext.privileged`, and `spec.ephemeralContainers[].securityContext.privileged`. Under the Pod Security Standards, those fields must be unset or `false` for the baseline and restricted profiles.

```
kubectl get pods -A -o json | jq -r '
.items[]
| .metadata.namespace as $ns
| .metadata.name as $pod
| def scan($type; $list):
  $list[]?
  | select(.securityContext.privileged == true)
  | "\""($ns)/\"($pod)\t\"($type)=\"(.name)\tprivileged=true\"";
scan("container"; .spec.containers),
scan("initContainer"; .spec.initContainers),
scan("ephemeralContainer"; .spec.ephemeralContainers)
,'
```

Expected Output:

kube-system/oci-csi-node-9vxcs	container=oci-csi-node-driver
privileged=true	
kube-system/oci-csi-node-dgzj8	container=oci-csi-node-driver
privileged=true	
kube-system/kube-proxy-dn9w5	container=kube-proxy
privileged=true	
kube-system/kube-proxy-wdm9m	container=kube-proxy
privileged=true	

OR

```
kubectl get pods --all-namespaces -o json | jq '
.items[]
| select(
  .metadata.namespace != "kube-system"
  and .spec.containers[]?.securityContext?.privileged == true
)
| {
  pod: .metadata.name,
  namespace: .metadata.namespace,
  container: .spec.containers[].name
}
,'
```

Review `kube-system` separately, because system and platform components may require elevated settings that should not be applied to normal workload namespaces.

This command uses `jq`, a command line JSON processor, to parse the JSON output from `kubectl get pods` and filter out pods where any container has the `securityContext.privileged` flag set to `true`. Please note that you might need to adjust the command depending on your specific requirements and the structure of your pod specifications.

Remediation:

Configure Pod Security Admission on workload namespaces so privileged containers are denied when new pods are created. For this control, both baseline and restricted disallow privileged containers, but restricted is typically the stronger choice for user workloads. Only enforce blocks noncompliant pods. warn and audit are useful during rollout, but they do not remediate the issue by themselves.

Label each workload namespace to enforce restricted.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/enforce=restricted --overwrite
```

Add **warn** and **audit** labels during transition so violations are surfaced before or alongside enforcement.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/warn=restricted --overwrite  
  
kubectl label ns NAMESPACE pod-security.kubernetes.io/audit=restricted --overwrite
```

- Update workload manifests so no regular container, init container, or ephemeral container uses **securityContext.privileged: true**. Under the Pod Security Standards, those fields must be unset or false for baseline and restricted profiles.
- Redeploy or restart affected workloads after the manifest changes so replacement pods are evaluated under the enforced namespace policy. Pod Security Admission evaluates pods when they are created.
- Review system namespaces separately and do not apply the same labels everywhere without checking platform requirements. System and platform components may require elevated settings that should be reviewed and approved separately from normal workload namespaces.

Default Value:

By default, privileged containers are not blocked unless the appropriate Pod Security Admission labels or equivalent policy controls are applied.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingpsspwithoke.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutk8sversions.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Reference/contengadmissioncontrollers.htm>
4. https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingadmissioncontrollers_topic.htm

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	<u>5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts</u> Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.			
v7	<u>5.1 Establish Secure Configurations</u> Maintain documented, standard security configuration standards for all authorized operating systems and software.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1611	TA0004	M1038, M1048

4.2.2 Minimize the admission of containers wishing to share the host process ID namespace (Automated)

Profile Applicability:

- Level 1

Description:

Minimize the admission of pods that request host process ID namespace sharing by restricting use of `spec.hostPID: true` in OKE workload namespaces. In the OKE Pod Security Standards model, this is enforced through Pod Security Admission or an equivalent policy engine. Because the baseline profile prohibits `hostPID` and restricted inherits that restriction, general user workloads should not be permitted to use host PID sharing.

Do not generally permit containers to be run with the `hostPID` flag set to `true`.

Rationale:

Host PID sharing reduces workload isolation by allowing a pod to share the host process namespace. In OKE, this setting should be controlled at admission time using Pod Security Admission or an equivalent policy engine so workloads that request `spec.hostPID: true` are rejected before they are created. Oracle documents that PodSecurity is a supported admission controller in current OKE Kubernetes versions, and that admission controllers can reject requests during Kubernetes API processing. Applying those profiles through Pod Security Admission allows OKE to reject new pods that request host PID sharing instead of relying only on runtime detection after deployment.

If you need to run pods that share the host PID namespace, this should be defined in a separate policy and you should carefully check to ensure that only limited service accounts and users are given permission to use that policy.

Impact:

Pods that set `spec.hostPID: true` will be denied in namespaces enforcing the baseline or restricted Pod Security Standard, so diagnostic, node management, or other specialized workloads that depend on host PID sharing may need a tightly controlled exception path or separate placement.

Pods defined with `spec.hostPID: true` will not be permitted unless they are run under a specific policy.

Audit:

Review the effective Pod Security Admission configuration for every namespace in the OKE cluster and verify that workload namespaces enforce a profile that disallows `spec.hostPID: true`. Oracle documents PodSecurity as a supported admission controller in current OKE Kubernetes versions, and the enforce label should be used where admission blocking is required.

Do not rely only on a live pod search as the primary audit. A pod query shows workloads that are already running with host PID sharing, but it does not prove that new pods requesting hostPID would be denied. Any namespace exceptions should be treated as explicit configuration choices, not assumed defaults.

Inspect existing pods for admitted use of host PID sharing:

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | select(.spec.hostPID == true)
  | "\(.metadata.namespace)/\(.metadata.name)\thostPID=true"
'
```

This command identifies pods already running with `spec.hostPID: true`, which is important because tightening Pod Security Admission does not remove existing pods that were admitted earlier.

Remediation:

Configure Pod Security Admission on workload namespaces so pods that request `spec.hostPID: true` are denied at admission time. For this control, the baseline and restricted Pod Security Standards disallow `hostPID`, while restricted is typically the stronger choice for user workloads. For remediation, use enforce because only enforce rejects violating pods. warn and audit can help during rollout, but they do not block admission.

Label each workload namespace to enforce restricted.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/enforce=restricted --
overwrite
```

Add warn and audit during transition if you want visibility into violations before or alongside enforcement.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/warn=restricted --
overwrite

kubectl label ns NAMESPACE pod-security.kubernetes.io/audit=restricted --
overwrite
```

- Update workload manifests to remove `spec.hostPID: true` then redeploy the affected workloads so new pods are evaluated under the enforced policy.
- Review any exceptions separately and keep them limited to approved platform namespaces or workloads. Exceptions should be documented and tied to a clear operational requirement.

- Check for already running pods that still use host PID sharing, because changing a namespace to a more restrictive enforced profile does not delete existing pods.

```
kubectl get pods -A -o json | jq -r '
.items[]
| select(.spec.hostPID == true)
| "\(.metadata.namespace)/\(.metadata.name)\thostPID=true"
'
```

Default Value:

By default, **hostPID** pods are not blocked unless the appropriate Pod Security Admission labels or equivalent policy controls are applied.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Reference/contengadmissioncontrollers.htm>
2. https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingadmissioncontrollers_topic.htm
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingpspswithoke.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutk8sversions.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	13.10 Perform Application Layer Filtering Perform application layer filtering. Example implementations include a filtering proxy, application layer firewall, or gateway.			●
v7	12.9 Deploy Application Layer Filtering Proxy Server Ensure that all network traffic to or from the Internet passes through an authenticated application layer proxy that is configured to filter unauthorized connections.			●

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1078, T1078.002	TA0001, TA0004	M1026

4.2.3 Minimize the admission of containers wishing to share the host IPC namespace (Automated)

Profile Applicability:

- Level 1

Description:

Minimize the admission of pods that request `spec.hostIPC: true` in OKE so that host IPC namespace sharing is limited to tightly controlled, explicitly approved platform use cases. In the OKE Pod Security Standards model, this is done through Pod Security Admission or an equivalent policy engine. Because the baseline profile prohibits `hostIPC` and restricted inherits the baseline restrictions, general workload namespaces should not permit host IPC sharing.

Do not generally permit containers to be run with the `hostIPC` flag set to `true`.

Rationale:

Host IPC sharing reduces workload isolation by allowing a pod to share the host interprocess communication namespace. In OKE, this setting should be controlled at admission time using Pod Security Admission or an equivalent policy engine so workloads that request `spec.hostIPC: true` are rejected before they are created. Oracle documents PodSecurity as a supported admission controller in current OKE Kubernetes versions, and admission controllers can reject requests during Kubernetes API processing. Since `hostIPC` is prohibited by the baseline profile and inherited by restricted, applying those profiles to workload namespaces provides an admission control that blocks pod specs requesting host IPC sharing before they are persisted and started.

Impact:

Pods with `spec.hostIPC: true` will be denied in namespaces that enforce the baseline or restricted Pod Security Standard, so specialized workloads that depend on host IPC sharing will need a tightly controlled exception path or separate placement. Also, if a namespace is moved to a more restrictive profile, existing noncompliant pods are not automatically deleted, so only newly created or restarted pods are blocked by the updated enforced policy.

Pods defined with `spec.hostIPC: true` will not be permitted unless they are run under a specific policy.

Audit:

Review the effective Pod Security Admission configuration for every namespace in the OKE cluster and verify that workload namespaces enforce a profile that disallows `spec.hostIPC: true`. The enforce label should be used where admission blocking is required. The baseline profile prohibits `hostIPC`, and restricted inherits the baseline restrictions.

Do not rely only on a live pod search as the primary audit. A pod query can identify workloads that are already running with host IPC sharing, but it does not prove that new pods requesting `hostIPC` would be denied. Pod Security Admission uses namespace labels, and only enforce blocks violating pods. warn and audit surface violations but do not block admission.

Review namespace Pod Security Admission labels:

```
kubectl get ns -o json | jq -r '
  ([ "NAMESPACE", "ENFORCE", "WARN", "AUDIT" ] | @tsv),
  (
    .items[]
    | [
      .metadata.name,
      (.metadata.labels["pod-security.kubernetes.io/enforce"] // "-"),
      (.metadata.labels["pod-security.kubernetes.io/warn"] // "-"),
      (.metadata.labels["pod-security.kubernetes.io/audit"] // "-")
    ]
    | @tsv
  )
' | column -ts $'\t'
```

Inspect running pods for admitted use of host IPC sharing:

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | select(.spec.hostIPC == true)
  | "\""(.metadata.namespace)/\"(.metadata.name)\"thostIPC=true"
  ,
'
```

This identifies pods already running with `spec.hostIPC: true`, which matters because changing a namespace to a more restrictive enforced profile does not delete existing pods that were previously admitted.

Remediation:

configure Pod Security Admission on workload namespaces so pods that request `spec.hostIPC: true` are denied at admission time. For this control, the baseline and restricted Pod Security Standards disallow `hostIPC`, while restricted is typically the stronger choice for user workloads. For remediation, use enforce because only enforce rejects violating pods. warn and audit can help during rollout, but they do not block admission.

Label each workload namespace to enforce restricted.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/enforce=restricted --
overwrite
```

Add warn and audit during transition if you want visibility into violations before or alongside enforcement.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/warn=restricted --
overwrite
```

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/audit=restricted --
overwrite
```

Update workload manifests to remove **spec.hostIPC: true**, then redeploy the affected workloads so new pods are evaluated under the enforced policy.

Review any exceptions separately and keep them limited to approved platform namespaces or workloads. Exceptions should be documented and tied to a clear operational requirement.

Check for already running pods that still use host IPC sharing, because changing a namespace to a more restrictive enforced profile does not delete existing pods.

```
kubectl get pods -A -o json | jq -r '
.items[]
| select(.spec.hostIPC == true)
| "\(.metadata.namespace)/\(.metadata.name)\thostIPC=true"
'
```

Default Value:

By default, **hostIPC** pods are not blocked unless the appropriate Pod Security Admission labels or equivalent policy controls are applied.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Reference/contengadmissioncontrollers.htm>
2. https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingadmissioncontrollers_topic.htm
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingpsspswithoke.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutk8sversions.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.12 <u>Segment Data Processing and Storage Based on Sensitivity</u> Segment data processing and storage based on the sensitivity of the data. Do not process sensitive data on enterprise assets intended for lower sensitivity data.			
v7	14.1 <u>Segment the Network Based on Sensitivity</u> Segment the network based on the label or classification level of the information stored on the servers, locate all sensitive information on separated Virtual Local Area Networks (VLANs).			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1098	TA0003	M1030

4.2.4 Minimize the admission of containers wishing to share the host network namespace (Automated)

Profile Applicability:

- Level 1

Description:

Minimize the admission of pods that request `spec.hostNetwork: true` in OKE so that sharing the host network namespace is limited to tightly controlled, explicitly approved platform scenarios. In OKE, this is done through Pod Security Admission or an equivalent policy engine. Because the baseline profile prohibits `hostNetwork`, and `restricted` inherits the baseline restrictions, general workload namespaces should not permit host network sharing.

Do not generally permit containers to be run with the `hostNetwork` flag set to `true`.

Rationale:

Host network sharing reduces workload isolation by allowing a pod to use the worker node network namespace. In OKE, this setting should be controlled at admission time using Pod Security Admission or an equivalent policy engine so workloads that request `spec.hostNetwork: true` are rejected before they are created. Oracle documents PodSecurity as a supported admission controller in current OKE Kubernetes versions, and admission controllers can reject requests during Kubernetes API processing. Applying Pod Security Standards in enforce mode addresses this at admission time by rejecting pods whose specs request host network sharing before those pods are created.

Impact:

Pods with `spec.hostNetwork: true` will be denied in namespaces that enforce the baseline or restricted Pod Security Standard, so workloads that depend on host networking need a tightly controlled exception path or separate placement.

Pods defined with `spec.hostNetwork: true` will not be permitted unless they are run under a specific policy.

Audit:

Review the effective Pod Security Admission configuration for every namespace in the OKE cluster and verify that workload namespaces enforce a profile that disallows `spec.hostNetwork: true`. The enforce label should be used where admission blocking is required. The baseline profile prohibits `hostNetwork`, and only enforce blocks pods that violate the configured Pod Security Standard.

Review namespace Pod Security Admission labels:

```
kubectl get ns -o json | jq -r '
  ([ "NAMESPACE", "ENFORCE", "WARN", "AUDIT" ] | @tsv),
  (
    .items[]
    | [
      .metadata.name,
      (.metadata.labels["pod-security.kubernetes.io/enforce"] // "-"),
      (.metadata.labels["pod-security.kubernetes.io/warn"] // "-"),
      (.metadata.labels["pod-security.kubernetes.io/audit"] // "-")
    ]
    | @tsv
  )
' | column -ts $'\t'
```

Inspect running pods for admitted use of host network sharing:

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | select(.spec.hostNetwork == true)
  | "\""(.metadata.namespace)/\"(.metadata.name)\thostNetwork=true"
'
```

This pod query identifies pods already running with **spec.hostNetwork: true**, which matters because changing a namespace to a more restrictive enforced profile does not delete existing pods that were previously admitted.

Remediation:

Configure Pod Security Admission on workload namespaces so pods that request **spec.hostNetwork: true** are denied at admission time. Use enforce on workload namespaces with restricted, or at minimum baseline, and reserve any exceptions for narrowly approved platform workloads. After updating namespace enforcement, remove **hostNetwork: true** from affected manifests and redeploy those workloads so new pods are evaluated under the enforced policy. For this control, the baseline and restricted Pod Security Standards disallow **hostNetwork**, while restricted is typically the stronger choice for user workloads.

Label each workload namespace to enforce restricted.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/enforce=restricted --
overwrite
```

Add **warn** and **audit** during rollout if you want visibility into violations, but do not rely on them for remediation because they do not block pod creation.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/warn=restricted --
overwrite

kubectl label ns NAMESPACE pod-security.kubernetes.io/audit=restricted --
overwrite
```

Update manifests to remove **spec.hostNetwork: true**, then redeploy the affected workloads.

Review any required exceptions separately and keep them limited to approved platform namespaces or workloads. Exceptions should be documented and tied to a clear operational requirement.

Check for already running pods that still use host networking, because changing a namespace to a more restrictive profile does not remove existing pods automatically.

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | select(.spec.hostNetwork == true)
  | "\(.metadata.namespace)/\(.metadata.name)\thostNetwork=true"
'
```

Default Value:

By default, **hostNetwork** pods are not blocked unless the appropriate Pod Security Admission labels or equivalent policy controls are applied.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Reference/contengadmissioncontrollers.htm>
2. https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingadmissioncontrollers_topic.htm
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingpsspswithoke.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutk8sversions.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.12 Segment Data Processing and Storage Based on Sensitivity Segment data processing and storage based on the sensitivity of the data. Do not process sensitive data on enterprise assets intended for lower sensitivity data.			
v7	14.1 Segment the Network Based on Sensitivity Segment the network based on the label or classification level of the information stored on the servers, locate all sensitive information on separated Virtual Local Area Networks (VLANs).			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1098	TA0003	M1030

4.2.5 Minimize the admission of containers with *allowPrivilegeEscalation* (Automated)

Profile Applicability:

- Level 1

Description:

Minimize the admission of containers with `securityContext.allowPrivilegeEscalation: true` in OKE so that general workload namespaces do not permit processes to gain more privileges than their parent process. In OKE, this should be enforced through Pod Security Admission or an equivalent policy engine. The restricted Pod Security Standard requires containers to explicitly set `securityContext.allowPrivilegeEscalation=false`, so general workload namespaces should enforce restricted where this control is required.

It's important to note that these rights are still constrained by the overall container sandbox, and this setting does not relate to the use of privileged containers.

Rationale:

Container security context settings can strengthen or weaken workload isolation. In OKE, this setting should be controlled at admission time using Pod Security Admission or an equivalent policy engine so workloads that do not explicitly set `securityContext.allowPrivilegeEscalation=false` are rejected before they are created. Oracle documents PodSecurity as a supported admission controller in current OKE Kubernetes versions, and admission controllers can reject requests during Kubernetes API processing.

Privilege escalation allows a process to change the security context under which it runs, such as through binaries with the SUID or SGID bit. In OKE, this can be controlled at admission time by setting `securityContext.allowPrivilegeEscalation=false` in the pod spec or by using Pod Security Standards or policy as code so noncompliant API requests can be rejected before they are persisted.

Impact:

In namespaces enforcing the restricted Pod Security Standard, containers that do not set `securityContext.allowPrivilegeEscalation=false` can be denied at admission, so workloads that rely on privilege changing behavior will need manifest updates or a tightly controlled exception path. warn and audit modes only report violations, while enforce is the mode that blocks pod creation.

Pods defined with `securityContext.allowPrivilegeEscalation: true` will not be permitted unless they are run under a specific policy.

Audit:

Review the effective Pod Security Admission configuration for every namespace in the OKE cluster and verify that workload namespaces enforce a profile that requires `securityContext.allowPrivilegeEscalation=false` for containers. For this specific control, the restricted profile is the relevant check, and only enforce blocks noncompliant pods. If your cluster uses a policy as code engine instead of Pod Security Admission, review the equivalent validation policy that rejects containers which do not explicitly set `allowPrivilegeEscalation=false`.

Review namespace Pod Security Admission labels:

```
kubectl get ns -o json | jq -r '
  ([ "NAMESPACE", "ENFORCE", "WARN", "AUDIT" ] | @tsv),
  (
    .items[]
    | [
      .metadata.name,
      (.metadata.labels["pod-security.kubernetes.io/enforce"] // "-"),
      (.metadata.labels["pod-security.kubernetes.io/warn"] // "-"),
      (.metadata.labels["pod-security.kubernetes.io/audit"] // "-")
    ]
    | @tsv
  )
' | column -ts $'\t'
```

Inspect running pods for containers, init containers, and ephemeral containers where `allowPrivilegeEscalation` is either `true` or unset, because restricted requires the value to be explicitly `false`:

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | .metadata.namespace as $ns
  | .metadata.name as $pod
  | def scan($type; $list):
    $list[]?
    | select((.securityContext.allowPrivilegeEscalation // "unset") !=
false)
  |
  "\($ns)/\($pod)\t\($type)=\(.name)\tallowPrivilegeEscalation=\(.securityContext.allowPrivilegeEscalation // "unset")";
  scan("container"; .spec.containers),
  scan("initContainer"; .spec.initContainers),
  scan("ephemeralContainer"; .spec.ephemeralContainers)
'
```

This pod query identifies workloads already admitted that would not satisfy the restricted requirement, which matters because tightening namespace enforcement does not remove pods that are already running.

Remediation:

Configure Pod Security Admission on workload namespaces so containers that do not explicitly set `securityContext.allowPrivilegeEscalation=false` are denied at admission time. In OKE, use restricted for workload namespaces, or an equivalent policy engine, because restricted requires `allowPrivilegeEscalation=false`. Use enforce for actual remediation, since `warn` and `audit` only report violations and do not block pod creation.

Label each workload namespace to enforce restricted.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/enforce=restricted --overwrite
```

Add warn and audit during rollout if you want visibility into violations before or alongside enforcement.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/warn=restricted --overwrite  
  
kubectl label ns NAMESPACE pod-security.kubernetes.io/audit=restricted --overwrite
```

Update workload manifests so every regular container, init container, and ephemeral container explicitly sets:

```
securityContext:  
  allowPrivilegeEscalation: false
```

Redeploy the affected workloads so new pods are evaluated under the enforced policy.

Review any exceptions separately and keep them limited to approved platform namespaces or workloads. Exceptions should be documented and tied to a clear operational requirement. Existing pods are not deleted automatically when a namespace is tightened to a more restrictive profile.

Default Value:

By default, containers are not required to set `allowPrivilegeEscalation=false` unless the appropriate Pod Security Admission labels or equivalent policy controls are applied.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Reference/contengadmissioncontrollers.htm>
2. https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingadmissioncontrollers_topic.htm
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingpsspwithoke.htm>
4. https://docs.oracle.com/en-us/iaas/Content/Security/Reference/oke_security.htm

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	<u>5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts</u> Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.			
v7	<u>5.1 Establish Secure Configurations</u> Maintain documented, standard security configuration standards for all authorized operating systems and software.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1611	TA0004	M1038, M1048

4.3 CNI Plugin

4.3.1 Ensure the OKE CNI add-on uses an Oracle supported version (Automated)

Profile Applicability:

- Level 1

Description:

There are a variety of CNI plugins available for Kubernetes. In OKE, the CNI plugin used for pod networking is an essential cluster add-on and should run an Oracle supported version that is compatible with the cluster's Kubernetes version.

Rationale:

OKE uses a CNI plugin to implement pod networking for worker nodes. Oracle documents the OCI VCN-Native Pod Networking CNI plugin and the flannel CNI plugin as CNI plugin add-ons, and states that supported add-on versions depend on the Kubernetes version running on the cluster. Running an Oracle supported CNI add-on version helps preserve pod network connectivity, IP address management, and compatibility with OKE managed add-on behavior.

Impact:

Updating the CNI add-on can restart or recreate CNI pods and may affect pod networking during the update window. For the OCI VCN-Native Pod Networking CNI plugin, Oracle documents version specific update behavior, including cases where updates are applied when worker nodes are rebooted or when CNI pods are explicitly recreated.

Audit:

Review the documentation of the OKE CNI add-on in use by the cluster, and confirm that it is an Oracle supported version for the Kubernetes version running on the cluster.

- List the cluster add-ons deployed on the cluster and identify the installed CNI plugin add-on version.

```
oci ce cluster list-addons --cluster-id $CLUSTER_ID
```

- List the add-on versions supported for the Kubernetes version running on the cluster.

```
oci ce addon-option list --kubernetes-version $KUBERNETES_VERSION
```

- If you know the CNI add-on name shown by the cluster, narrow the supported version check to that add-on.

```
oci ce addon-option list --kubernetes-version $KUBERNETES_VERSION --addon-name $ADDON_NAME
```

- Check the DaemonSets in the kube-system namespace: Many CNI plugins operate as DaemonSets within the **kube-system** namespace. To see what's running:

```
kubectl get daemonsets -n kube-system
```

Look for known OKE CNI components such as the OCI VCN-Native Pod Networking CNI plugin or flannel.

- You can further inspect the configuration of these DaemonSets to understand more about the CNI setup:

```
kubectl get daemonsets -n kube-system -o json \
| jq -r '
  .items[]
  | select(
    (.metadata.name |
test("cni|calico|flannel|weave|cilium|canal|multus|vcn|oci"; "i"))
    or
    ([.spec.template.spec.containers[].image] | join(" ") |
test("cni|calico|flannel|weave|cilium|canal|multus|vcn|oci"; "i"))
    )
  | "\(.metadata.name)\timage=\(.spec.template.spec.containers[].image)"
'
```

Check the CNI Configuration Files: If you have access to the nodes (via SSH), you can check the CNI configuration directly in **/etc/cni/net.d/**. This often requires node level access, which might not be available depending on your permissions and the security setup of your environment.

Remediation:

Update the OKE CNI add on to an Oracle supported version that is compatible with the cluster's Kubernetes version. If automatic add on updates are enabled, confirm the cluster is running a supported Kubernetes version so compatible CNI updates can be applied. If a specific add on version is selected, verify that the selected version is supported for the cluster's Kubernetes version and update it when Oracle publishes a newer supported version.

For the OCI CNI plugin, apply updates using Oracle's documented approach. The recommended approach is to provision new nodes so they receive CNI plugin pods running the latest supported CNI plugin version, then drain and remove nodes running older CNI plugin pods when appropriate.

For existing nodes that must be updated in place, follow Oracle's documented CNI pod recreation process, including any required admission policy changes before deleting and recreating the CNI pods.

Default Value:

By default, OKE initially uses the latest version of the selected CNI plugin for the cluster network type. Supported CNI versions depend on the Kubernetes version running on the cluster and should be verified against Oracle's supported add on version list.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengpodnetworking.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengintroducingclusteraddons.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/configuration-arguments-vcn-native-podnetworking.htm>
4. https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengpodnetworking_topic-OCI_CNI_plugin.htm

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	7.4 Perform Automated Application Patch Management Perform application updates on enterprise assets through automated patch management on a monthly, or more frequent, basis.			
v7	11.4 Install the Latest Stable Version of Any Security-related Updates on All Network Devices Install the latest stable version of any security-related updates on all network devices.			

4.3.2 Ensure workload namespaces have required NetworkPolicies defined (Manual)

Profile Applicability:

- Level 1

Description:

Use network policies to isolate traffic in your cluster network. For OKE, ensure workload namespaces have required **NetworkPolicies** defined and that the selected networking configuration supports **NetworkPolicy** enforcement.

Rationale:

Running different applications on the same Kubernetes cluster creates a risk of one compromised application attacking a neighboring application. Network segmentation is important to ensure that containers can communicate only with those they are supposed to. A network policy is a specification of how selections of pods are allowed to communicate with each other and other network endpoints.

NetworkPolicies apply to namespaces. When a network policy is introduced to a given namespace, all traffic not allowed by the policy is denied. However, if there are no network policies in a namespace all traffic will be allowed into and out of the pods in that namespace. In OKE, **NetworkPolicy** enforcement requires a supported network policy solution. Oracle supports **NetworkPolicy** enforcement with Calico when using the OCI CNI plugin, and does not support using the Calico network policy engine with the flannel CNI plugin.

Impact:

Once network policies are in use within a given namespace, traffic not explicitly allowed by a network policy will be denied. As such it is important to ensure that, when introducing network policies, legitimate traffic is not blocked. **NetworkPolicies** that are defined without a supported enforcement mechanism will not provide the intended traffic isolation.

Audit:

Run the below command and review the **NetworkPolicy** objects created in the cluster. This command will list all namespaces in your OKE cluster that have one or more **NetworkPolicy** objects defined:

```
echo -e "NAMESPACE\tNETWORKPOLICY_COUNT" && \
for ns in $(kubectl get ns -o jsonpath='{.items[*].metadata.name}'); do
    count=$(kubectl get networkpolicy -n "$ns" --no-headers 2>/dev/null | wc -l
| tr -d ' ')
    if [ "$count" -gt 0 ]; then
        echo -e "${ns}\t${count}"
    fi
done
```

OR show namespaces that do not have Network Policies defined

```
echo -e "NAMESPACES_WITHOUT_NETWORKPOLICIES" && \
for ns in $(kubectl get ns -o jsonpath='{.items[*].metadata.name}'); do
    if ! kubectl get networkpolicy -n "$ns" --no-headers 2>/dev/null | grep -q
.; then
        echo "$ns"
    fi
done
```

Ensure that each workload namespace defined in the cluster has at least one Network Policy. Review system and platform namespaces separately, because they may require different network rules. Also confirm that **NetworkPolicy** enforcement is supported by the cluster networking configuration.

Remediation:

Follow the documentation and create **NetworkPolicy** objects as you need them. For OKE, define **NetworkPolicies** for workload namespaces based on the required allowed ingress and egress traffic, and confirm that the cluster uses a supported **NetworkPolicy** enforcement model.

Clusters created with OKE use either the OCI CNI plugin or the flannel CNI plugin for pod networking, depending on the cluster network type and creation path. Oracle supports **NetworkPolicy** enforcement through Calico when using the OCI CNI plugin. Oracle does not support using the Calico network policy engine with the flannel CNI plugin.

If the cluster uses the OCI CNI plugin, install and configure Calico network policy enforcement using Oracle's documented process, then create the required namespace **NetworkPolicies**.

If the cluster uses flannel and **NetworkPolicy** enforcement is required, follow Oracle's documented migration or replacement approach rather than installing Calico alongside flannel. Oracle notes that installing the Calico network policy engine alongside flannel can cause network issues and is not supported.

Default Value:

By default, network policies are not created.

References:

1. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/contengsettingupcalico.htm>

2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengpodnetworking.htm>
3. https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengpodnetworking_topic-OCI_CNI_plugin.htm
4. https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengCompatibilityMatrix_topic.htm

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.4 Implement and Manage a Firewall on Servers Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.			
v7	14.1 Segment the Network Based on Sensitivity Segment the network based on the label or classification level of the information stored on the servers, locate all sensitive information on separated Virtual Local Area Networks (VLANs).			
v7	14.2 Enable Firewall Filtering Between VLANs Enable firewall filtering between VLANs to ensure that only authorized systems are able to communicate with other systems necessary to fulfill their specific responsibilities.			
v6	14.1 Implement Network Segmentation Based On Information Class Segment the network based on the label or classification level of the information stored on the servers. Locate all sensitive information on separated VLANs with firewall filtering to ensure that only authorized individuals are only able to communicate with systems necessary to fulfill their specific responsibilities.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1046	TA0007	M1030, M1042

4.4 Secrets Management

4.4.1 Prefer exposing secrets as mounted files rather than environment variables (Automated)

Profile Applicability:

- Level 1

Description:

Secrets for OKE workloads are better exposed as mounted files than as environment variables. Kubernetes secrets can be consumed either way, and Oracle supports mounting secrets from OCI Vault into Kubernetes Pods through the OCI Secrets Store CSI Driver Provider with the Secrets Store CSI Driver.

Rationale:

Environment variable values can unintentionally appear in logs, while secrets mounted as volumes are instantiated as tmpfs volumes and are automatically removed from the node when the Pod is deleted. File based delivery also aligns with the OCI Secrets Store CSI Driver Provider model, which enables Kubernetes clusters to access secrets stored in OCI Vault and mount them into Pods through the Secrets Store CSI Driver.

Impact:

Workloads that currently read secrets from environment variables may need manifest and application changes to consume secrets from mounted file paths instead, using Kubernetes secret volumes or the OCI Secrets Store CSI Driver Provider integration with OCI Vault.

Audit:

Review workload manifests and identify any `Pod`, `Deployment`, `DaemonSet`, `StatefulSet`, `Job`, or `CronJob` that exposes secrets through environment variables instead of mounted files. Check for both direct secret references through `env[].valueFrom.secretKeyRef` and bulk secret imports through `envFrom[].secretRef`, because either pattern exposes secret data as environment variables. Prefer workloads that consume secrets through mounted Secret volumes or file based delivery from OCI secrets integrations instead.

Run this command to list workloads that reference secrets through either `environment-variable` pattern:

```
kubectl get deploy,ds,sts,job,cronjob,pod -A -o json | jq -r '
.items[]
| .kind as $kind
| .metadata.namespace as $ns
| .metadata.name as $name
| (if $kind == "Pod" then .spec else .spec.template.spec end) as $spec
| select([ $spec.containers[]?, $spec.initContainers[]? ] |
any(.env[]?.valueFrom.secretKeyRef? or .envFrom[]?.secretRef?))
| "\($ns)\t\($kind)/\($name)"
'
```

- The command returns one line for each workload that exposes a Kubernetes Secret through an environment variable pattern.

Sample output:

default	Deployment/web-app
apps	StatefulSet/orders-api
logging	DaemonSet/log-agent
batch	Job/nightly-report
reports	CronJob/monthly-export

Note: For this audit, no output is the preferred result, because it means no matching workloads were found exposing Secrets as environment variables

Remediation:

Update workloads to consume secrets as mounted files instead of environment variables. Replace `env[].valueFrom.secretKeyRef` and `envFrom[].secretRef` usage with Kubernetes Secret volumes or the OCI Secrets Store CSI Driver Provider with the Secrets Store CSI Driver, then update the application to read the secret from the mounted file path. This reduces the chance of secret values appearing in logs and aligns with the OCI Vault based file delivery pattern for OKE workloads.

Replace secret environment variables with mounted secret files and update the application to read from the mounted path.

Default Value:

By default, secrets are not defined

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengmanagingsecrets.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengencryptingdata.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/secret-management/Concepts/manage-secrets.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/secret-management/home.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.1 <u>Establish and Maintain a Secure Configuration Process</u> Establish and maintain a secure configuration process for enterprise assets (end-user devices, including portable and mobile, non-computing/IoT devices, and servers) and software (operating systems and applications). Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●
v7	14.4 <u>Encrypt All Sensitive Information in Transit</u> Encrypt all sensitive information in transit.		●	●
v7	14.8 <u>Encrypt Sensitive Information at Rest</u> Encrypt all sensitive information at rest using a tool that requires a secondary authentication mechanism not integrated into the operating system, in order to access the information.			●

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1552	TA0006	M1026

4.4.2 Consider external secret storage (Manual)

Profile Applicability:

- Level 1

Description:

External secret storage for OKE means keeping application secrets outside native Kubernetes Secret objects and retrieving them from a managed service such as OCI Vault when workloads need them. This approach lets OKE workloads consume secrets through integrations such as the OCI Secrets Store CSI Driver Provider with the Secrets Store CSI Driver, instead of relying only on Kubernetes to store and distribute sensitive values.

Rationale:

Storing sensitive credentials outside Kubernetes reduces dependence on Secret objects stored inside the cluster and shifts lifecycle functions such as encryption, access control, rotation, and audit to a dedicated secrets service. In OKE, Pods can retrieve secrets from OCI Vault by using OCI IAM access through OKE workload identity, and the OCI Secrets Store CSI Driver Provider can mount those values into Pods as files through the Secrets Store CSI Driver. This model supports centralized secret lifecycle management and tighter secret to workload access control than broadly distributing static values through Kubernetes manifests.

Impact:

Moving to external secret storage usually requires manifest and application changes so workloads fetch or mount secrets from OCI Vault instead of reading only native Kubernetes Secret objects, and it may require OKE workload identity permissions plus CSI integration components. In return, secret rotation and access management can be handled centrally in OCI rather than separately inside each cluster.

Audit:

Review the OKE cluster's secrets architecture and verify that required workloads use an external secrets store such as OCI Vault instead of relying only on native Kubernetes Secret objects. The audit should confirm both the integration path and the workload access model, because external secret storage in OKE typically depends on the OCI Secrets Store CSI Driver Provider, the Secrets Store CSI Driver, and OCI IAM access through OKE workload identity.

- Identify workloads that still consume sensitive credentials from native Kubernetes Secrets and determine which of those should be moved to OCI Vault.
- Confirm the external secret integration components are in use, including the OCI Secrets Store CSI Driver Provider and Secrets Store CSI Driver, and verify that workloads reference a `SecretProviderClass` where external secrets are mounted as files.

- Verify workload access is limited through OKE workload identity so each workload can retrieve only the secrets it is authorized to read.
- Check whether rotation, access control, and audit requirements are being handled in the external store rather than duplicated through static secret values embedded in Kubernetes manifests or long lived native Secret objects.

Remediation:

Move sensitive application secrets out of native Kubernetes Secret objects and into an external store such as OCI Vault. Then update OKE workloads to retrieve those secrets through the OCI Secrets Store CSI Driver Provider with the Secrets Store CSI Driver, using OCI IAM access through OKE workload identity. This shifts encryption, access control, rotation, and auditing to OCI instead of leaving those functions primarily in Secret objects stored inside the cluster.

- Create the required secrets in OCI Vault and define workload access with least privilege OCI IAM policies.
- Install and configure the OCI Secrets Store CSI Driver Provider and the Secrets Store CSI Driver for OKE.
- Update workload manifests to mount external secrets through a `SecretProviderClass` instead of reading only native Kubernetes Secret objects.
- Enable rotation and update the application to read secrets from the mounted path so secret lifecycle management is handled centrally in OCI.
- Move secrets to OCI Vault, mount them into Pods, and limit access with OKE workload identity.

Default Value:

By default, no external secret management is configured.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengmanagingsecrets.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/secret-management/Concepts/manage-secrets.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contenggrantingworkloadaccesstoresources.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengencryptingdata.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.1 <u>Establish and Maintain a Secure Configuration Process</u> Establish and maintain a secure configuration process for enterprise assets (end-user devices, including portable and mobile, non-computing/IoT devices, and servers) and software (operating systems and applications). Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.			
v7	14.8 <u>Encrypt Sensitive Information at Rest</u> Encrypt all sensitive information at rest using a tool that requires a secondary authentication mechanism not integrated into the operating system, in order to access the information.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1552	TA0006	M1026

4.5 General Policies

These policies relate to general cluster management topics, like namespace best practices and policies applied to pod objects in the cluster.

4.5.1 Create administrative boundaries between resources using namespaces (Manual)

Profile Applicability:

- Level 1

Description:

Namespaces create administrative boundaries in OKE by dividing a cluster into logical partitions for teams, applications, or tenants. They are a core building block for multi tenancy, and several controls used to separate workloads, including quotas, network policies, and service accounts, apply to namespaces.

Rationale:

Namespaces matter operationally because many access and governance controls in Kubernetes and OKE can be applied by namespace. Kubernetes **Role** and **RoleBinding** objects are used to delegate permissions within a namespace, and OKE uses OCI IAM to control access to OKE and the cluster control plane, while Kubernetes RBAC controls what authenticated users and service accounts can do inside the cluster. That allows platform teams to grant administrative access to only the namespaces a team or workload should manage, rather than granting broad cluster permissions.

The namespace boundary becomes stronger when it is combined with namespace specific resource and traffic controls. **ResourceQuota** and **LimitRange** can constrain CPU, memory, and object consumption within a namespace, while network policies can restrict Pod to Pod communication and support a least privilege model between tenant or application namespaces. A namespace alone is still only a logical grouping construct, so effective administrative separation depends on pairing it with RBAC, OCI IAM access design, quotas, service accounts, and network isolation.

Impact:

Creating administrative boundaries with namespaces usually requires teams to move workloads out of the default namespace and implement namespace specific RBAC, OCI IAM access design, quotas, and network policies. This improves delegation and organization, but namespaces by themselves do not provide full isolation.

Audit:

Verify that namespaces are present and are being used as intentional administrative boundaries for teams, applications, or environments in the OKE cluster. Namespaces are the core logical partition for multi tenancy, and they should be paired with namespace specific access controls and namespace level governance such as quotas, limits, and network policies. A namespace should be considered adequately administered only when it has a clear workload or team purpose, appropriately limited access, and the namespace level controls required for that use case.

- List all namespaces and confirm each has a defined administrative, application, or environment purpose. Namespaces are the base partitioning construct for multi tenancy in OKE.
- Verify that user and workload access is limited to the intended namespaces through Kubernetes **Role** and **RoleBinding** objects and, where used, OCI IAM groups mapped through Kubernetes RBAC, rather than broad cluster grants. OKE uses OCI IAM to control access to OKE and the cluster control plane, and Kubernetes RBAC to control authorization inside the cluster.
- Review whether namespace level controls such as **ResourceQuota**, **LimitRange**, service accounts, and network policies are applied where required to support separation, resource governance, and Pod to Pod traffic restrictions.
- Confirm that broad cluster access is limited to identities that truly require it, because namespace boundaries alone do not prevent cross tenant or cross workload access.

Inventory checks for OKE Clusters:

```
kubectl get namespaces

kubectl get rolebindings,roles -A

kubectl get clusterrolebindings,clusterroles

kubectl get resourcequota,limitrange,networkpolicy -A

oci iam policy list --compartment-id $COMPARTMENT_ID
```

Note: Review OCI IAM policies and group membership for access to OKE and the cluster control plane, then review Kubernetes RBAC objects to confirm what those authenticated users and service accounts can do inside the cluster.

Remediation:

Create separate namespaces for teams, applications, or environments that require their own administrative boundary, and move workloads out of the default namespace into that managed namespace structure. Make the namespace boundary meaningful by limiting access to the intended namespaces, applying namespace level governance such as quotas, limits, service accounts, and network policies, and reducing broad cluster permissions that bypass the intended separation model. When stronger isolation is required for mutually untrusted workloads, use separate clusters rather than relying on namespaces alone.

- Create dedicated namespaces for each team, application, or environment that needs its own administrative boundary.
- Move user workloads out of the **default** namespace and into the appropriate managed namespaces.
- Rebind access with namespace specific **Role** and **RoleBinding** objects and, where used, map OCI IAM users or groups to Kubernetes RBAC permissions that are limited to the intended namespaces.

- Apply **ResourceQuota**, **LimitRange**, service accounts, and network policies to reinforce namespace level governance and traffic separation.
- Remove unused namespaces and reduce unnecessary broad cluster permissions or OCI IAM access that bypass the intended namespace boundary.

Create dedicated namespaces, move workloads out of default, limit access by namespace, and apply quotas, limits, service accounts, and network policies to enforce the administrative boundary.

Default Value:

By default, Kubernetes starts with two initial namespaces:

1. **default** - The default namespace for objects with no other namespace
2. **kube-system** - The namespace for objects created by the Kubernetes system
3. **kube-public** - The namespace for public-readable ConfigMap
4. **kube-node-lease** - The namespace for associated lease object for each node

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengclustersnodes.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutaccesscontrol.htm>
3. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengpolicyconfig.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contenggrantingworkloadaccesstoresources.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.1 Establish and Maintain a Secure Configuration Process Establish and maintain a secure configuration process for enterprise assets (end-user devices, including portable and mobile, non-computing/IoT devices, and servers) and software (operating systems and applications). Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.			
v7	14 Controlled Access Based on the Need to Know Controlled Access Based on the Need to Know			
v6	14 Controlled Access Based on the Need to Know Controlled Access Based on the Need to Know			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1106, T1609	TA0002	M1038

4.5.2 Apply securityContext controls to pods and containers (Manual)

Profile Applicability:

- Level 1

Description:

Apply Security Context to Your Pods and Containers. For OKE workloads, **securityContext** controls should be used with Pod Security Admission or an equivalent admission policy so pods and containers run with only the operating system privileges they require.

Rationale:

A security context defines the operating system security settings (uid, gid, capabilities, SELinux role, etc..) applied to a container. When designing your containers and pods, make sure that you configure the security context for your pods, containers, and volumes. A security context is a property defined in the deployment yaml. It controls the security parameters that will be assigned to the **pod/container/volume**. There are two levels of security **context**: pod level security context, and container level security context. In OKE, **PodSecurity** is a supported admission controller, and Oracle guidance points to Pod Security Standards and the **PodSecurity** admission controller for enforcing pod security behavior at admission time.

Impact:

If you incorrectly apply security contexts, you may have trouble running the pods. Workloads may need manifest updates to run as non root, drop unnecessary capabilities, prevent privilege escalation, or satisfy the Pod Security Standard enforced in the namespace.

Audit:

Review the pod definitions in your cluster and verify that you have security contexts defined as appropriate.

Here is a key compliance check in OKE (Oracle Kubernetes Engine) to ensure all Pods define a **securityContext** at either the pod or container level:

```
echo -e
"NAMESPACE\tPOD\tHAS_POD_SECURITY_CONTEXT\tHAS_CONTAINER_SECURITY_CONTEXT" &&
\
kubectl get pods -A -o json \
| jq -r '
  .items[]
  | .metadata.namespace as $ns
  | .metadata.name as $pod
  | (.spec.securityContext != null) as $hasPodSC
  | ([(.spec.containers[]?.securityContext != null)] | any) as
$hasContainerSC
  | [$ns, $pod, $hasPodSC, $hasContainerSC] | @tsv
'
```

- **HAS_POD_SECURITY_CONTEXT**: checks for a pod level `.spec.securityContext`
- **HAS_CONTAINER_SECURITY_CONTEXT**: verifies if any container in the pod defines `.securityContext`
- **jq any**: returns true if at least one container has a security context
- **-A**: Works across all namespaces

Sample Output:

NAMESPACE	POD	HAS_POD_SECURITY_CONTEXT	HAS_CONTAINER_SECURITY_CONTEXT
default	file-check		true
true			
kube-system	coredns-bc868476b-g8285		true
true			
kube-system	csi-oci-node-hsr28		true
true			
kube-system	kube-dns-autoscaler-8b86f9c9f-fzqsv		true
true			
kube-system	kube-proxy-g949b		true
true			
kube-system	proxymux-client-5p85m		true
true			
kube-system	vcn-native-ip-cni-jwxtl		true
true			
dev	api-service-7dd55b8f65		false
false			

Also review namespace Pod Security Admission labels to confirm workload namespaces enforce the required Pod Security Standard.

```
kubectl get ns -o json | jq -r '
  ([ "NAMESPACE", "ENFORCE", "WARN", "AUDIT" ] | @tsv),
  (
    .items[]
    | [
      .metadata.name,
      (.metadata.labels["pod-security.kubernetes.io/enforce"] // "-"),
      (.metadata.labels["pod-security.kubernetes.io/warn"] // "-"),
      (.metadata.labels["pod-security.kubernetes.io/audit"] // "-")
    ]
    | @tsv
  )
' | column -ts $'\t'
```

Remediation:

Use **securityContext** settings in workload manifests and enforce the required Pod Security Standard with Pod Security Admission or an equivalent admission policy. Avoid using legacy **PodSecurityPolicy** manifests for current OKE clusters.

For normal application workloads, configure pod and container **securityContext** settings that align to the intended security level. Common settings include running as a non root user, disabling privilege escalation, dropping unnecessary Linux capabilities, restricting host namespace use, and setting read only root filesystems where the application supports it.

```
securityContext:
  runAsNonRoot: true
  allowPrivilegeEscalation: false
  capabilities:
    drop:
      - ALL
```

- Label workload namespaces to enforce the required Pod Security Standard.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/enforce=restricted --
overwrite
```

- Add warn and audit labels during rollout if you want visibility into violations before or alongside enforcement.

```
kubectl label ns NAMESPACE pod-security.kubernetes.io/warn=restricted --
overwrite

kubectl label ns NAMESPACE pod-security.kubernetes.io/audit=restricted --
overwrite
```

- Redeploy affected workloads after manifest changes so replacement pods are evaluated under the enforced namespace policy. Review system namespaces separately because platform components can require different security settings.

Default Value:

By default, no security contexts are automatically applied to pods.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Reference/contengadmissioncontrollers.htm>
2. https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingadmissioncontrollers_topic.htm
3. https://docs.oracle.com/en-us/iaas/Content/Security/Reference/oke_security.htm
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengusingpsspswithoke.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.1 Establish and Maintain a Secure Configuration Process Establish and maintain a secure configuration process for enterprise assets (end-user devices, including portable and mobile, non-computing/IoT devices, and servers) and software (operating systems and applications). Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.			
v7	5.1 Establish Secure Configurations Maintain documented, standard security configuration standards for all authorized operating systems and software.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1556, T1611	TA0004, TA0006	M1048

4.5.3 The default namespace should not be used (Manual)

Profile Applicability:

- Level 1

Description:

Avoid placing application workloads in the **default** namespace, and instead deploy them into dedicated namespaces that reflect the intended team, application, or environment boundary. In OKE, namespaces are a primary logical partition for organizing workloads and supporting multi tenancy in a shared cluster.

Rationale:

Least privilege administration in OKE depends on using namespaces as intentional operational boundaries. Kubernetes **Role** and **RoleBinding** objects apply within a namespace, and OKE uses OCI IAM to control access to OKE and the cluster control plane, while Kubernetes RBAC controls what authenticated users and service accounts can do inside the cluster. When workloads remain in **default**, access delegation, ownership boundaries, and namespace level governance become harder to separate cleanly.

Impact:

Using the default namespace for application workloads weakens administrative separation and makes namespace level access control, resource governance, and network segmentation harder to apply consistently.

Audit:

Review the OKE cluster and verify that the **default** namespace is not being used for normal application administration or routine workload placement. Objects that do not explicitly specify a namespace are created in default, so the audit should confirm that teams, applications, and environments are instead using dedicated namespaces that align with namespace level access control, quotas, limits, service accounts, and network policies. Namespaces are a logical administrative boundary in OKE, so the goal is to confirm that user workloads are intentionally placed outside default and governed through namespace level controls.

- Inventory the default namespace and identify any user-managed **Pods**, **Services**, **ConfigMaps**, **Secrets**, **ServiceAccounts**, **Deployments**, **StatefulSets**, **DaemonSets**, **Jobs**, **CronJobs**, **Ingresses**, **Roles**, **RoleBindings**, **ResourceQuotas**, **LimitRanges**, or **NetworkPolicies** that still exist there.

```
kubectl get  
pod,svc,configmap,secret,serviceaccount,deploy,statefulset,daemonset,job,cron  
job,ingress,role,rolebinding,resourcequota,limitrange,networkpolicy -n  
default
```

- Verify that access is not effectively centered on **default**. Review Kubernetes **Role** and **RoleBinding** assignments for that namespace and review OCI IAM policies and group membership for access to OKE and the cluster control plane. OKE uses OCI IAM for cluster access and Kubernetes RBAC for authorization inside the cluster.
- Confirm that user workloads have been placed into dedicated namespaces that reflect team, application, or environment boundaries, and that namespace level controls such as quotas, limits, service accounts, and network policies are applied where required. Those controls are what make a namespace an actual administrative boundary instead of just a label.
- Check whether the namespace design reduces shared administrative access and naming sprawl. OKE supports namespace based organization with Kubernetes RBAC, service accounts, quotas, and network policies used to govern resources inside the cluster.
- Inventory the **default** namespace, identify user workloads still running there, and confirm access and governance are instead aligned to dedicated namespaces.

Remediation:

Move user workloads out of the **default** namespace and place them in dedicated namespaces that reflect the intended team, application, or environment boundary. In OKE, namespaces are the core logical partition for multi tenancy, but they only become meaningful administrative boundaries when they are paired with namespace level access, service accounts, quotas, limits, and network policies. Because namespaced objects are created in default when no namespace is specified, remediation should address both existing workloads and the deployment process that created them there.

- Create dedicated namespaces for each team, application, or environment that needs its own administrative boundary.
- Move user workloads out of the **default** namespace and into the appropriate managed namespaces.
- Update manifests, Helm values, GitOps definitions, and pipeline defaults so namespaced resources explicitly set **metadata.namespace** and are not created in default by omission.
- Rebind access with namespace level Kubernetes **Role** and **RoleBinding** objects and, where OCI IAM users or groups are mapped to Kubernetes RBAC, limit the resulting Kubernetes permissions to the intended namespaces.
- Apply namespace level governance such as **ResourceQuota**, **LimitRange**, dedicated service accounts, and network policies so the namespace boundary supports resource control, identity separation, and Pod to Pod traffic isolation.

- Remove unnecessary user workloads and broad operational access from default, and use separate clusters rather than namespaces alone when stronger isolation is required for less trusted workloads.

Move workloads out of **default**, enforce explicit namespaces in deployment workflows, and bind access and governance controls to the target namespaces.

Default Value:

By default, objects that do not specify a namespace are created in the **default** namespace.

References:

1. https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengbestpractices_topic-Multi-Tenancy-best-practices.htm
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutaccesscontrol.htm>
3. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengpolicyconfig.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengclustersnodes.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	12.2 Establish and Maintain a Secure Network Architecture Establish and maintain a secure network architecture. A secure network architecture must address segmentation, least privilege, and availability, at a minimum.			
v7	12.7 Deploy Network-Based Intrusion Prevention Systems Deploy network-based Intrusion Prevention Systems (IPS) to block malicious network traffic at each of the organization's network boundaries.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1578	TA0005	M1018

5 Managed services

This section consists of security recommendations for the Oracle OKE. These recommendations are applicable for configurations that Oracle OKE customers own and manage.

5.1 Image Registry and Image Scanning

This section contains recommendations relating to container image registries and securing images in those registries, such as Oracle Container Registry (OCR).

5.1.1 Ensure image vulnerability scanning uses OCIR or a third party provider (Manual)

Profile Applicability:

- Level 1

Description:

Container image vulnerability scanning helps strengthen the security of OKE workloads by identifying known software weaknesses in the images those workloads run. This supports a more secure software supply chain and helps reduce the chance that vulnerable images are promoted into Kubernetes environments.

Rationale:

Oracle Container Registry can scan images in a repository for security vulnerabilities published in the Common Vulnerabilities and Exposures database, using the OCI Vulnerability Scanning service. When image scanning is enabled for a repository, newly pushed images are scanned, existing images are scanned based on the configured scan target behavior, and images are automatically rescanned when new vulnerabilities are added to the CVE database.

Vulnerabilities in container image software packages can be exploited to compromise workloads and potentially gain unauthorized access to cluster, node, or connected cloud resources. OCIR and approved third party tools can scan container images for known vulnerabilities.

Impact:

This control improves detection and prioritization of vulnerable container images, but it also adds operational overhead to review findings, rebuild images, and manage registry scan configuration. For OCIR scanning, coverage depends on the repositories included in the Vulnerability Scanning configuration, the container scan target, and whether images used by OKE workloads are stored in repositories covered by the configured scanner.

Audit:

Verify that image vulnerability scanning is enabled for repositories that store images used by OKE workloads. For images stored in OCIR, review the Container Registry image scanner or OCI Vulnerability Scanning container scan targets and scan results. If a third party scanner is used instead, obtain equivalent evidence that images deployed to OKE are scanned and that findings are available for review.

Checks the container scan targets in the compartment. Use it to confirm that OCIR repositories used by OKE workloads are included in a Vulnerability Scanning target:

```
oci vulnerability-scanning container scan target list --compartment-id  
$COMPARTMENT_ID --all
```

- **Expected output:** JSON showing container scan targets. In that object, look for **display-name**, **lifecycle-state**, **compartment-id**, **container-scan-recipe-id**, and **target-registry** to confirm which OCIR repositories are covered.

Checks the container scan recipes in the compartment. Use it to confirm the scan settings applied by the scan target:

```
oci vulnerability-scanning container scan recipe list --compartment-id  
$COMPARTMENT_ID --all
```

- **Expected output:** JSON showing container scan recipes. In that object, look for **display-name**, **lifecycle-state**, **scan-settings**, and **id** to confirm the recipe used by the scan target.

Checks the container image scan results. Use it to confirm scan findings exist for images in covered OCIR repositories:

```
oci vulnerability-scanning container scan result list --compartment-id  
$COMPARTMENT_ID --all
```

- **Expected output:** JSON showing scan reports for container images. Review the image path, risk level, lifecycle state, and scan timing to confirm that images used by OKE workloads are being scanned and findings are available for review.

Remediation:

Enable image vulnerability scanning for OCIR repositories that supply container images to OKE workloads, or use an approved third party image scanning provider that produces equivalent vulnerability findings before image promotion and deployment. For OCIR, configure OCI Vulnerability Scanning with a container scan recipe and target for the repositories that store OKE images.

Identify the OCIR repositories and compartments that provide images to OKE workloads. This establishes which repositories must be included in the scanning coverage.

Configure OCI Vulnerability Scanning for the required repositories. Create a container image scan recipe in the compartment that will manage the scan configuration.

```
oci vulnerability-scanning container scan recipe create --display-name  
$SCAN_RECIPE_NAME --compartment-id $COMPARTMENT_ID --scan-settings  
'{"scanLevel":"STANDARD"}'
```

Generate the target registry JSON template and update it for the OCIR repositories that must be scanned.

```
oci vulnerability-scanning container scan target create --generate-param-  
json-input target-registry > target-registry.json
```

Create a container scan target that applies the scan recipe to the selected OCIR repositories.

```
oci vulnerability-scanning container scan target create --compartment-id  
$COMPARTMENT_ID --container-scan-recipe-id $CONTAINER_SCAN_RECIPE_OCID --  
target-registry file://target-registry.json
```

Review vulnerability findings and rebuild, replace, or prevent deployment of images with unacceptable risk before they are used by OKE workloads. This makes the control operational rather than leaving findings unaddressed.

Default Value:

By default, image scanning is disabled when a new Container Registry repository is created.

References:

1. <https://docs.oracle.com/iaas/Content/Registry/Tasks/registryscanningimagesforvulnerabilities.htm>
2. <https://docs.oracle.com/en-us/iaas/scanning/using/overview.htm>
3. <https://docs.oracle.com/en-us/iaas/scanning/using/managing-container-scan-recipes.htm>
4. <https://docs.oracle.com/en-us/iaas/scanning/using/managing-container-scan-targets.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	7.6 Perform Automated Vulnerability Scans of Externally-Exposed Enterprise Assets Perform automated vulnerability scans of externally-exposed enterprise assets using a SCAP-compliant vulnerability scanning tool. Perform scans on a monthly, or more frequent, basis.			
v7	3 Continuous Vulnerability Management Continuous Vulnerability Management			
v7	3.1 Run Automated Vulnerability Scanning Tools Utilize an up-to-date SCAP-compliant vulnerability scanning tool to automatically scan all systems on the network on a weekly or more frequent basis to identify all potential vulnerabilities on the organization's systems.			
v7	3.2 Perform Authenticated Vulnerability Scanning Perform authenticated vulnerability scanning with agents running locally on each system or with remote scanners that are configured with elevated rights on the system being tested.			

5.1.2 Restrict OKE and Kubernetes API access to required users (Manual)

Profile Applicability:

- Level 1

Description:

Restrict access to OKE and Kubernetes API operations to only authorized OCI IAM users, groups, and Kubernetes service accounts. Access should be limited based on the user's required administrative, operational, or workload function.

Rationale:

Excessive access to OKE or the Kubernetes API can allow unauthorized users or service accounts to create, modify, or delete clusters, node pools, workloads, secrets, and other Kubernetes resources. Oracle documents that OCI IAM controls who can manage OKE resources and access the cluster control plane, while Kubernetes RBAC controls what authenticated users and service accounts can do inside the cluster.

Impact:

Restricting access reduces the chance that a compromised or overprivileged identity can modify cluster configuration, deploy unauthorized workloads, access secrets, or disrupt running applications. Care should be taken not to remove access from required administrators, automation identities, platform service accounts, or approved operational users.

Audit:

For most operations on Kubernetes clusters created and managed by Container Engine for Kubernetes, Oracle Cloud Infrastructure Identity and Access Management (IAM) provides access control. A user's permissions to access clusters comes from the groups to which they belong. The permissions for a group are defined by policies. Policies define what actions members of a group can perform, and in which compartments. Users can then access clusters and perform operations based on the policies set for the groups they are members of.

IAM provides control over:

- whether a user can create or delete clusters
- whether a user can add, remove, or modify node pools
- which Kubernetes object create/delete/view operations a user can perform on all clusters within a compartment or tenancy
- See Policy Configuration for Cluster Creation and Deployment.

In addition to IAM, the Kubernetes RBAC Authorizer can enforce additional access control for users on specific clusters via Kubernetes RBAC roles and clusterroles. A Kubernetes RBAC role is a collection of permissions. For example, a role might include read permission on pods and list permission for pods. A Kubernetes RBAC clusterrole is just like a role, but can be used anywhere in the cluster. A Kubernetes RBAC rolebinding maps a role to a user or set of users, granting that role's permissions to those users for resources in that namespace. Similarly, a Kubernetes RBAC clusterrolebinding maps a clusterrole to a user or set of users, granting that clusterrole's permissions to those users for cluster resources.

Review OCI IAM policies that grant access to OKE resources.

```
oci iam policy list --compartment-id $COMPARTMENT_ID --all
```

Review Kubernetes RBAC roles and bindings that grant access inside the cluster.

```
kubectl get rolebindings,roles -A  
kubectl get clusterrolebindings,clusterroles
```

Confirm that each OCI IAM policy, **RoleBinding**, and **ClusterRoleBinding** is tied to a documented administrative, operational, or workload requirement.

Remediation:

By default, users are not assigned any Kubernetes RBAC roles (or clusterroles) by default. So before attempting to create a new role (or clusterrole), you must be assigned an appropriately privileged role (or clusterrole). A number of such roles and clusterroles are always created by default, including the **cluster-admin** clusterrole (for a full list, see Default Roles and Role Bindings in the Kubernetes documentation). The cluster-admin clusterrole essentially confers super-user privileges. A user granted the cluster-admin clusterrole can perform any operation across all namespaces in a given cluster.

Remove or reduce OCI IAM policies that grant unnecessary OKE permissions, and use Kubernetes RBAC roles and rolebindings to grant only the required Kubernetes API permissions inside the cluster. Avoid granting cluster-admin unless full cluster administration is required. Review high privilege service accounts and automation identities separately before making changes to avoid disrupting required platform operations.

Default Value:

By default, users are not assigned Kubernetes RBAC roles or clusterroles unless those permissions are explicitly granted. OCI IAM policies determine whether users can access and manage OKE resources, while Kubernetes RBAC determines what authenticated users and service accounts can do inside the cluster.

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.8 <u>Define and Maintain Role-Based Access Control</u> Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●
v7	14.6 <u>Protect Information through Access Control Lists</u> Protect all information stored on systems with file system, network share, claims, application, or database specific access control lists. These controls will enforce the principle that only authorized individuals should have access to the information based on their need to access the information as a part of their responsibilities.	●	●	●

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1078, T1078.002	TA0001, TA0004	M1026

5.1.3 Restrict OCIR access for OKE workloads to required image pulls (Manual)

Profile Applicability:

- Level 1

Description:

Restricting OCIR access for OKE workload image pulls helps ensure that the identities used to retrieve workload images can pull required container images without also receiving broader registry visibility, push, delete, or repository management access. In OKE, this applies to the OCI IAM users, groups, cluster policies, and Kubernetes `imagePullSecrets` used to pull images from private OCIR repositories.

Rationale:

The OKE runtime image pull path for private OCIR images requires registry credentials and repository access sufficient to pull image layers and manifests. Oracle documents that images pulled from OCIR during Kubernetes deployment require a Docker registry secret containing OCI credentials, and the workload manifest must reference that secret through `imagePullSecrets`. A pull-only policy is therefore a closer least privilege match for runtime image pulls because it supports image retrieval without also granting repository push, delete, visibility, or management actions.

A broader OCIR policy is unnecessary for workload image pulls because Oracle documents separate policy levels for inspecting repositories, pulling images, pushing images, and managing repositories. For image pull access, Oracle documents the read repos permission as the policy level that allows pulling images from Container Registry. Broader policies such as manage repos can allow push, repository deletion, or repository metadata changes, which are useful for administrators or CI/CD publishers but are not required just to start OKE workloads from private OCIR images. Restricting runtime identities to pull-only permissions therefore reduces unnecessary registry exposure while still preserving the permissions required for image retrieval.

Impact:

This control reduces unnecessary exposure of OCIR repository metadata and administrative actions to runtime identities, but the assigned identity must still retain the permissions required to authenticate and pull images. If those pull permissions are removed or limited incorrectly, OKE workloads will not be able to retrieve private OCIR images, which can prevent pods from starting successfully.

Audit:

Review the OCI IAM policies, OCIR repository access, and Kubernetes imagePullSecrets that apply to the identities used by OKE to pull images, and verify that those identities are limited to required image pull access. Focus on the OCI identity represented in the Docker registry secret, any OCI IAM policy that grants the cluster access to OCIR, and any repository policy or compartment policy that grants access to private OCIR repositories. The expected pull path requires read repos for image layers and manifests, while broader access such as manage repos can allow push, delete, and repository management actions that are not required just to pull images.

Identify the OCIR repositories used by OKE workloads. Review image references in workload manifests and confirm which private repositories are used.

```
kubectl get deploy,ds,sts,job,cronjob,pod -A -o json | jq -r '
.items[]
| .kind as $kind
| .metadata.namespace as $ns
| .metadata.name as $name
| (if $kind == "Pod" then .spec else .spec.template.spec end) as $spec
| $spec.containers[]?
| select(.image | test("ocir|oraclecloud.com"; "i"))
| "\($ns)\t\($kind)/\($name)\timage=\(.image)"
'
```

Identify the **imagePullSecrets** referenced by workloads that pull from private OCIR repositories.

```
kubectl get deploy,ds,sts,job,cronjob,pod -A -o json | jq -r '
.items[]
| .kind as $kind
| .metadata.namespace as $ns
| .metadata.name as $name
| (if $kind == "Pod" then .spec else .spec.template.spec end) as $spec
| ($spec.imagePullSecrets // [])[]
| "\($ns)\t\($kind)/\($name)\timagePullSecret=\(.name)"
'
```

Review the OCI IAM policies attached to the compartment or tenancy. Flag policies that grant more OCIR access than required for runtime image pulls, such as manage repos, broad repository update permissions, or broad repository delete permissions.

```
oci iam policy list --compartment-id $COMPARTMENT_ID --all
```

Review OCIR repositories in the compartment and confirm that workload pull identities are limited to only the repositories they require.

```
oci artifacts container repository list --compartment-id $COMPARTMENT_ID --all
```

Confirm the effective permission set is pull-only. Runtime image pull identities should be limited to repository authentication and image retrieval, not push, delete, or repository management access. If broader permissions are present, mark the identity or policy for remediation.

Remediation:

Enforce pull-only OCIR access for OKE workload image pull identities by limiting the OCI IAM users, groups, cluster policies, and `imagePullSecrets` used to retrieve private images. The runtime pull path should allow only the repository access required to pull image layers and manifests. Broader permissions that allow pushing images, deleting repositories, changing repository visibility, or managing repository metadata are not required just to start workloads.

- Identify the OCI identities, groups, policies, and `imagePullSecrets` used by OKE workloads to pull private OCIR images.
- Remove broader OCIR policies from runtime image pull identities. Avoid granting `manage repos` to identities that only need image pull access.
- Replace broad repository access with `read repos` for the required repositories or compartments. Oracle documents `read repos` as the policy level that allows pulling images from Container Registry.
- Allow group `<image-puller-group>` to read repos in compartment `<compartment-name>`
- Where possible, limit access to the required repositories by repository name condition.
- Allow group `<image-puller-group>` to read repos in compartment `<compartment-name>` where `all { target.repo.name=/app-prefix*/ }`
- Remove or replace policies that add `push`, `delete`, or repository management access unless there is a documented operational need.
- Check workload manifests and Kubernetes `imagePullSecrets` so only approved secrets are used for pulling private OCIR images.

Retest image pulls after the change. Confirm that OKE workloads can still pull required private OCIR images and that the runtime identity no longer has broader repository access than needed.

Default Value:

By default, OKE workloads pulling private OCIR images require a Kubernetes Docker registry secret and OCI credentials with permission to access the repository. OCIR repository access is controlled through OCI IAM policies rather than a single OKE cluster setting.

References:

1. <https://docs.oracle.com/iaas/Content/Registry/Concepts/registripolicyrepoaccess.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengpullingimagesfromocir.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/Registry/Tasks/registripullingimagesfromocir.htm>

4. <https://docs.oracle.com/en-us/iaas/Content/Registry/Concepts/registryoverview.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.			
v7	14 Controlled Access Based on the Need to Know Controlled Access Based on the Need to Know			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1552	TA0006	M1026

5.1.4 Allow only approved container registries for images used by OKE workloads (Manual)

Profile Applicability:

- Level 1

Description:

Allowing only approved container registries for images used by OKE workloads helps reduce supply chain risk by limiting where workloads can source their images. This creates a controlled image origin model in which clusters admit workloads only from trusted registries and can pair that control with OCIR image signature verification before containers are allowed to run.

Rationale:

A practical way to enforce this control in OKE is through admission controls that evaluate Kubernetes API requests before the requested change is persisted. Those policies can validate image source rules, verify signatures, and reject deployments that do not meet the required trust conditions. For images stored in OCIR, OKE supports image verification policies that allow deployment only when images are signed with specified master encryption keys from OCI Vault. This makes the registry restriction more than a naming convention and turns it into an admission control point tied to trusted publishers and approved image sources.

Impact:

This control reduces the chance that workloads pull images from unapproved or untrusted registries, but it adds administrative overhead because the approved registry policy must be maintained for platform add ons, vendor components, and operational exceptions. If the approved registry list is too narrow, admission can deny valid deployments until the required registry is explicitly added. When OCIR image verification is enabled, unsigned images or images signed with an unapproved key can also be denied.

Audit:

Review the image sources currently used by the cluster and the admission controls that govern new deployments, then verify that workloads are limited to approved registries only. The strongest implementation for this control is an admission policy, because admission controllers can reject Kubernetes API requests before objects are persisted. For OCIR images, also review OKE image verification policy configuration where signed image enforcement is used. Include required Oracle managed image sources for platform components and add ons in the approved list.

Compare every running workload image registry hostname to the approved registry list. Any image sourced from an unapproved registry is a finding.

```
kubectl get pods -A -o jsonpath='{range .items[*]}{.metadata.namespace}{"\t"}{.metadata.name}{"\t"}{range .spec.containers[*]}{.image}{" "}{end}{"\n"}{end}'
```

- Confirm the cluster uses an admission control path that evaluates image source before deployment is accepted, rather than relying only on CI/CD checks or documented standards.
- Check the approved registry list. Verify it includes approved OCIR repositories, Oracle managed registries required for platform components, and approved third party registries if any.
- Confirm an approved image is accepted and a test image from an unapproved registry is rejected at admission time.
- Verify any additional allowed registries are documented and intentionally approved, and where image signing is used, confirm the admission controller also validates those trust conditions for approved image sources.

Remediation:

Restrict OKE workloads to approved container registries by enforcing image source controls at admission time, rather than relying only on CI/CD checks or written standards. Enforcement can be implemented with an admission controller so image references are checked before the deployment is persisted. For OCIR images, use OKE image verification policies when signed image enforcement is required, so only images signed with approved OCI Vault keys are admitted. The approved registry list must also include the Oracle managed image sources required for platform components and add ons.

- Include approved OCIR repositories, explicitly approved third party registries, and Oracle managed registries required for OKE platform components and add ons in the approved registry list.
- Configure an admission control path so image registry source is checked before the workload is admitted.
- Apply the policy to workload namespaces while accounting for system namespaces and approved operational exceptions so required platform components are not blocked.
- Configure OKE image verification policies for OCIR images when stronger supply chain controls are required, using approved OCI Vault keys.
- Confirm approved registries are allowed, unapproved registries are denied, and update the approved registry list when new add ons, vendor components, or trusted registries are introduced.

Default Value:

By default, container registries are not restricted by Kubernetes, and OKE image verification policies must be explicitly configured when signed OCIR image enforcement is required.

References:

1. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/contengenforcingsignedimagesfromocir.htm>
2. https://docs.oracle.com/iaas/Content/Registry/Tasks/registrysigningimages_topic.htm
3. <https://docs.oracle.com/en-us/iaas/Content/Registry/Tasks/registrypullingimagesfromocir.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Reference/contengadmissioncontrollers.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v7	5.2 Maintain Secure Images Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.			
v7	5.3 Securely Store Master Images Store the master images and templates on securely configured servers, validated with integrity monitoring tools, to ensure that only authorized changes to the images are possible.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1133, T1195	TA0001, TA0003	M1016, M1042

5.2 Identity and Access Management (IAM)

This section contains recommendations relating to using Oracle Cloud IAM with OKE.

5.2.1 Use dedicated Kubernetes service accounts for OKE workloads (Automated)

Profile Applicability:

- Level 1

Description:

Using dedicated Kubernetes service accounts for OKE workloads helps isolate OCI permissions by application, job, or trust boundary instead of sharing a single identity across multiple workloads. This supports more controlled access to OCI services and reduces the security impact if one workload is compromised or misconfigured.

Rationale:

A workload that needs OCI access should run with a Kubernetes service account that is used with OKE workload identity, and that identity model should be created for each unique permission set an application needs. OKE workload identity uses the unique combination of cluster, namespace, and service account to authorize workload access to OCI resources through OCI IAM policies. Using a dedicated service account per workload or per permission boundary prevents unrelated pods in the same namespace from inheriting the same OCI access and aligns permissions more closely to the actual application function.

Impact:

This control improves permission isolation and usually makes auditing easier, but it increases the number of service accounts, OCI IAM policies, and workload identity configurations that must be created and maintained. If a pod is left on the namespace default service account, or if the wrong service account is referenced in the workload spec, the pod may fail to access required OCI resources or may run with a broader identity than intended.

Audit:

Review all workloads in the cluster and verify that applications use dedicated Kubernetes service accounts rather than relying on the namespace default service account. Confirm that the namespace default service account has only the safe default discovery access unless additional rights are explicitly required, and verify that pods which do not need Kubernetes API access are not automatically mounting service account tokens. For workloads that need OCI access, verify that the referenced service account is configured for OKE workload identity and that the permission set matches the application's actual OCI access requirements. Also confirm that pods cannot bypass the dedicated service account model by inheriting broader node or instance principal based credentials where those are used.

Review each namespace default service account and confirm it has no extra **RoleBindings** or **ClusterRoleBindings** beyond the safe default discovery access unless explicitly required.

Review workload manifests and confirm pods explicitly set **serviceAccountName** when they need Kubernetes API or OCI access, rather than falling back to the namespace default service account.

Verify **automountServiceAccountToken: false** is set for pods or default service accounts where the application does not need Kubernetes API access. Do not treat this as a blanket requirement for every workload.

Check pods for an explicit setting:

```
kubectl get pods -A \
-o custom-
columns='NAMESPACE:.metadata.namespace,POD:.metadata.name,SERVICEACCOUNT:.spec.serviceAccountName,AUTOMOUNT:.spec.automountServiceAccountToken'
```

Check the default service account in each namespace:

```
kubectl get sa -A \
-o custom-
columns='NAMESPACE:.metadata.namespace,NAME:.metadata.name,AUTOMOUNT:.automountServiceAccountToken' | grep ' default '
```

- For workloads that access OCI services, verify the dedicated service account is used with the intended OKE workload identity configuration and that each unique permission set uses its own service account boundary.
- Verify node or instance principal access is restricted where applicable so pods cannot still obtain broader node based OCI access after a service account identity model is configured.

Remediation:

Use dedicated Kubernetes service accounts for OKE workloads and associate OCI permissions to those service accounts instead of relying on the namespace default service account or broad node based OCI permissions. Use OKE workload identity for workloads that need OCI resource access. Create one dedicated service account and OCI IAM policy design for each application or distinct permission boundary, and update workload manifests to reference the intended service account explicitly. Disable automatic service account token mounting for workloads that do not need Kubernetes API access. After migrating workloads to dedicated service accounts, restrict access to node or instance principal credentials where applicable so pods cannot inherit broader node credentials.

- Create a dedicated Kubernetes service account for each application or distinct OCI permission boundary instead of sharing the namespace default service account.

- Associate that service account with least privilege OCI IAM permissions by using OKE workload identity.
- Update workload manifests to set `spec.serviceAccountName` explicitly so the intended service account is always used.
- Set `automountServiceAccountToken: false` for pods or service accounts that do not need Kubernetes API access.
- Restrict node or instance principal access where applicable so workloads use the dedicated service account identity path and cannot also inherit broader node credentials.

Default Value:

By default, Pods use the namespace default service account, and token mounting is automatic unless explicitly disabled. A dedicated service account with OCI permissions exists only if you configure OKE workload identity for that workload.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contenggrantingworkloadaccessstoresources.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengclustersnodes.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengaboutaccesscontrol.htm>
4. https://docs.oracle.com/iaas/Content/Security/Reference/oke_security.htm

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.5 Establish and Maintain an Inventory of Service Accounts Establish and maintain an inventory of service accounts. The inventory, at a minimum, must contain department owner, review date, and purpose. Perform service account reviews to validate that all active accounts are authorized, on a recurring schedule at a minimum quarterly, or more frequently.			
v7	4.3 Ensure the Use of Dedicated Administrative Accounts Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.			

5.3 Cloud Key Management Service (Cloud KMS)

This section contains recommendations relating to using Cloud KMS with OKE.

5.3.1 Use customer managed KMS keys for OKE envelope encryption of Kubernetes secrets (Manual)

Profile Applicability:

- Level 1

Description:

Using customer managed OCI Vault keys for OKE Kubernetes secrets encryption adds a customer managed protection layer for Kubernetes secrets stored through the Kubernetes API. This strengthens data at rest protection while allowing organizations to manage key ownership, rotation, policy, and audit visibility for the key used to protect secret data.

Rationale:

Envelope encryption adds a second encryption layer to Kubernetes secrets stored through the OKE control plane. Secret data is encrypted with a unique data encryption key, and that data encryption key is then encrypted with a master encryption key stored in OCI Vault before the object is persisted in etcd. This means the secret is stored in ciphertext in the backing datastore and is decrypted only through the Kubernetes API server path, which strengthens protection beyond the underlying storage encryption used for etcd.

In the current OKE model, envelope encryption protects Kubernetes secrets in etcd by using a data encryption key and a master encryption key. A customer managed OCI Vault key gives the customer direct control over key selection, rotation, audit visibility, and key lifecycle for the key that protects those data encryption keys.

Impact:

Using a customer managed OCI Vault key increases customer control, but it also adds Vault key cost, key policy administration, and lifecycle responsibility. If you use your own key, do not delete the Vault key used for OKE Kubernetes secrets encryption. Oracle documents that deleting the key makes the Kubernetes secrets stored in etcd permanently inaccessible and can require deleting and recreating the cluster.

Audit:

Review the OKE cluster configuration and verify that Kubernetes secrets are protected with a customer managed OCI Vault key, not only the default Oracle managed encryption for the block volumes used by etcd. For this control, the important check is whether the cluster is associated with a customer managed key for Kubernetes secrets encryption. Also verify that the key is usable by the cluster and has not been disabled, deleted, or stripped of required permissions. A customer managed key association should appear on the cluster, while clusters using the default Oracle managed encryption do not expose a customer key OCID for this purpose.

Review the cluster encryption configuration and confirm a customer managed OCI Vault key OCID is associated to the cluster for Kubernetes secrets protection. If no customer key OCID is associated, the cluster does not meet this control.

```
oci ce cluster get \
  --cluster-id $CLUSTER_ID \
  --query 'data."kms-key-id"' \
  --raw-output
```

In the command output, verify that the encryption configuration includes a nonempty **kms-key-id** value for the customer managed OCI Vault key. If the response is empty, null, or no customer key OCID is present, this control is not satisfied.

Review the selected OCI Vault key and confirm it exists and remains enabled.

```
oci kms management key get \
  --key-id $KEY_ID \
  --endpoint $MANAGEMENT_ENDPOINT
```

Verify the key is still enabled and that the IAM policy allowing the OKE cluster to use the key has not been revoked. A disabled key, revoked policy, or deleted key can place the cluster into a degraded or unrecoverable state.

Where continuous compliance monitoring is used, confirm the cluster has the expected customer managed OCI Vault key OCID configured and that the key remains enabled and accessible to the OKE cluster.

Remediation:

Configure the OKE cluster to use a customer managed OCI Vault key for envelope encryption of Kubernetes secrets so secret data stored through the Kubernetes API is protected with a customer managed master encryption key. This requires an existing master encryption key in OCI Vault, with policy permissions that allow OKE clusters to use the key.

OKE does not allow you to enable customer managed Kubernetes secrets encryption on an existing cluster after creation. Oracle documents that this option is selected when creating a new cluster through the Custom Create workflow or with the OCI CLI by specifying the Vault key OCID.

- After the new cluster is created, the key must remain enabled and accessible. Disabling, deleting, or breaking access to the associated key can place the cluster in a degraded or unrecoverable state.
- Create or identify a customer managed OCI Vault master encryption key that already exists and is allowed for OKE use.
- Create the required IAM policy so the OKE cluster can use the key. Oracle documents either an any-user policy with **request.principal.type = 'cluster'** and the target key OCID, or a dynamic group policy for clusters that need access to the key.
- Create a new OKE cluster with the existing OCI Vault key OCID for Kubernetes secrets encryption. Replace the variables with real values before running the

command. In this command, **CLUSTER_NAME** is the OKE cluster name, **COMPARTMENT_ID** is the target compartment, **VCN_OCID** is the VCN, **KUBERNETES_VERSION** is the selected Kubernetes version, and **KEY_OCID** is the existing OCI Vault key OCID.

```
oci ce cluster create \  
  --name $CLUSTER_NAME \  
  --compartment-id $COMPARTMENT_ID \  
  --vcn-id $VCN_OCID \  
  --kubernetes-version $KUBERNETES_VERSION \  
  --kms-key-id $KEY_OCID
```

- Recheck the cluster configuration and confirm that **data."kms-key-id"** contains the expected customer managed OCI Vault key OCID.
- Keep the associated OCI Vault key enabled and do not delete it while the cluster depends on it.

For an existing OKE cluster that was not created with a customer managed key, create a replacement cluster with the required key configuration and migrate workloads to the new cluster.

Default Value:

By default, Oracle encrypts data in block volumes at rest, including etcd and Kubernetes secrets, using an Oracle managed master encryption key. A customer managed OCI Vault key is used only when you explicitly select that option during OKE cluster creation.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengencryptingdata.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/KeyManagement/Concepts/keyoverview.htm>
3. <https://docs.oracle.com/en-us/iaas/Content/KeyManagement/Tasks/managingkeys.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/Identity/Tasks/managingpolicies.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.11 <u>Encrypt Sensitive Data at Rest</u> Encrypt sensitive data at rest on servers, applications, and databases containing sensitive data. Storage-layer encryption, also known as server-side encryption, meets the minimum requirement of this Safeguard. Additional encryption methods may include application-layer encryption, also known as client-side encryption, where access to the data storage device(s) does not permit access to the plain-text data.			

Controls Version	Control	IG 1	IG 2	IG 3
v7	14.8 Encrypt Sensitive Information at Rest Encrypt all sensitive information at rest using a tool that requires a secondary authentication mechanism not integrated into the operating system, in order to access the information.			●

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1552	TA0006	M1022

5.4 Cluster Networking

This section contains recommendations relating to network security configurations in OKE.

5.4.1 Restrict OKE control plane endpoint access using private endpoints or approved CIDRs (Automated)

Profile Applicability:

- Level 1

Description:

This recommendation is about reducing exposure of the Kubernetes API server without necessarily making it private only. The intended end state is that the OKE Kubernetes API endpoint is placed behind a private endpoint where possible, or if a public IP address is assigned, access is limited to approved source CIDRs through OCI network security rules. This keeps the control plane reachable for approved administration paths while lowering unnecessary internet exposure.

Rationale:

OKE exposes the Kubernetes API through an endpoint hosted in a subnet of the cluster VCN. The Kubernetes API endpoint subnet can be private or public, and if a public subnet is used, the endpoint can optionally be assigned a public IP address. Access to the Kubernetes API endpoint subnet is controlled with network security groups or security lists. If the endpoint is private, access is limited to networks that can reach the VCN, such as connected subnets, peered VCNs, FastConnect, VPN, or bastion based administration paths. If the endpoint has a public IP address, the endpoint can be reached from the internet unless the endpoint subnet's security rules restrict TCP/6443 to approved source CIDRs.

Restricting access to an authorized network can provide additional security benefits for your container cluster, including:

- Better protection from outsider attacks: Restricting the Kubernetes API endpoint to approved CIDRs or private network paths provides an additional layer of security by limiting external access to a specific set of addresses you designate, such as those that originate from your premises. This helps protect access to your cluster in the case of a vulnerability in the cluster's authentication or authorization mechanism.
- Better protection from insider attacks: Restricting endpoint access helps protect your cluster from accidental leaks of cluster access credentials from your company's premises. Leaked credentials used from outside approved network paths or approved CIDR ranges are still denied network access.

Impact:

This recommendation gives you a balanced hardening option because it reduces public exposure while preserving approved remote administration paths. The tradeoff is operational maintenance of network security rules and the risk of accidental lockout if trusted administrator IPs, bastion paths, VPN ranges, FastConnect ranges, or other required source ranges are omitted. It is stricter than a public endpoint model open to the internet, but less restrictive than a private only API design.

Audit:

Check the OKE Kubernetes API endpoint configuration and determine whether the endpoint has a public IP address assigned. Also identify the endpoint subnet and any network security groups attached to the endpoint.

```
oci ce cluster get \
  --cluster-id $CLUSTER_ID \
  --query 'data.{endpointConfig:"endpoint-config",endpoints:endpoints}' \
  --output json
```

Expected Output:

```
{
  "endpointConfig": {
    "is-public-ip-enabled": false,
    "nsg-ids": [
      "ocid1.networksecuritygroup.oc1..."
    ],
    "subnet-id": "ocid1.subnet.oc1..."
  },
  "endpoints": {
    "private-endpoint": "10.0.0.10:6443",
    "public-endpoint": null
  }
}
```

OR

```
{
  "endpointConfig": {
    "is-public-ip-enabled": true,
    "nsg-ids": [],
    "subnet-id": "ocid1.subnet.oc1..."
  },
  "endpoints": {
    "ipv6-endpoint": null,
    "kubernetes": null,
    "private-endpoint": "10.0.0.7:6443",
    "public-endpoint": "132.226.xxx.xxx:6443",
    "vcn-hostname-endpoint": "dfd6airtdg...com:6443"
  }
}
```

- **Note:** Check that **is-public-ip-enabled** is **false** when the Kubernetes API endpoint should be private. If **is-public-ip-enabled** is **true**, review the

endpoint subnet security rules and any endpoint network security groups to confirm TCP/6443 is limited to approved CIDRs and is not open to **0.0.0.0/0**.

If the endpoint subnet is returned, review the subnet configuration and associated security lists.

```
oci network subnet get \  
  --subnet-id $ENDPOINT_SUBNET_OCID \  
  --query 'data.{cidrBlock:"cidr-block",prohibitPublicIpOnVnic:"prohibit-public-ip-on-vnic",securityListIds:"security-list-ids"}' \  
  --output json
```

If endpoint network security groups are returned, review their ingress rules and confirm Kubernetes API access on TCP/6443 is limited to approved source CIDRs.

```
oci network nsg rules list \  
  --nsg-id $NSG_OCID \  
  --direction INGRESS \  
  --all
```

Remediation:

Restrict access to the OKE Kubernetes API endpoint by using a private endpoint where possible, or by limiting public endpoint access through OCI network security groups or security lists. The Kubernetes API endpoint is hosted in a VCN subnet, and Oracle recommends using network security groups or security lists to control access to that subnet.

For new clusters, place the Kubernetes API endpoint in a private subnet when internet access to the API endpoint is not required.

```
oci ce cluster create \  
  --name $CLUSTER_NAME \  
  --compartment-id $COMPARTMENT_ID \  
  --vcn-id $VCN_OCID \  
  --kubernetes-version $KUBERNETES_VERSION \  
  --endpoint-subnet-id $PRIVATE_ENDPOINT_SUBNET_OCID \  
  --endpoint-public-ip-enabled false
```

If the Kubernetes API endpoint must have a public IP address, update the endpoint subnet security list or endpoint network security group so TCP/6443 is allowed only from approved administrator, bastion, VPN, FastConnect, or automation CIDRs. Do not leave Kubernetes API endpoint access open to **0.0.0.0/0** unless there is a documented and approved requirement.

If an existing cluster uses a public Kubernetes API endpoint and a private endpoint is required, create or move to an OKE cluster design that places the API endpoint in a private subnet, then update kubeconfig access paths to use the private endpoint through approved network connectivity such as bastion, VPN, FastConnect, or peered VCN access.

Default Value:

By default, the OKE Kubernetes API endpoint is not assigned a public IP address unless endpoint public IP access is explicitly enabled during cluster creation.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengclustersnodes.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengnetworkconfig.htm>
3. <https://docs.oracle.com/iaas/Content/ContEng/Concepts/contengnetworkconfigexample.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengsettingupbastion.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	9.3 <u>Maintain and Enforce Network-Based URL Filters</u> Enforce and update network-based URL filters to limit an enterprise asset from connecting to potentially malicious or unapproved websites. Example implementations include category-based filtering, reputation-based filtering, or through the use of block lists. Enforce filters for all enterprise assets.			
v7	7.5 <u>Subscribe to URL-Categorization service</u> Subscribe to URL categorization services to ensure that they are up-to-date with the most recent website category definitions available. Uncategorized sites shall be blocked by default.			

5.4.2 Enable private OKE control plane access and disable public IP access (Automated)

Profile Applicability:

- Level 1

Description:

This recommendation is the strict private access version of API endpoint hardening. The intended end state is that the Kubernetes API server is reachable only from inside the cluster VCN or from a connected private network, and there is no internet-reachable public API endpoint at all. This is a pure control plane exposure decision.

Rationale:

In OKE, the Kubernetes API endpoint is hosted in a subnet of the cluster VCN. The endpoint is always assigned a private IP address, and if the endpoint is placed in a public subnet, it can optionally be assigned a public IP address. For private control plane access, place the Kubernetes API endpoint in a private subnet and do not assign a public IP address. Administration from outside the VCN must use a private connectivity path, such as peered VCN access, VPN, FastConnect, or a bastion.

If the Kubernetes API endpoint has no public IP address, internet based kubectl access is not available. Access is governed by the endpoint subnet route rules, security lists, and network security groups. If public IP access is disabled, approved administrators and automation must reach the private endpoint through an allowed network path.

Although Kubernetes API requires an authorized token to perform sensitive actions, a vulnerability could potentially expose the Kubernetes API publicly with unrestricted access. Additionally, an attacker may be able to identify the current cluster and Kubernetes API version and determine whether it is vulnerable to an attack. Unless required, disabling public IP access will help prevent such threats, and require the attacker to be on the VCN or an approved connected network to perform any attack on the Kubernetes API.

Impact:

This recommendation controls for reducing direct exposure of the OKE control plane, but it requires deliberate private connectivity for administrators, automation, and any external systems that need Kubernetes API access. If that private access path is missing or incomplete, cluster operations, troubleshooting, and deployment tooling can be blocked even though the cluster itself is healthy.

Audit:

Check for private endpoint access to the Kubernetes API server

```
oci ce cluster get \
  --cluster-id $CLUSTER_ID \
  --query 'data.{endpointConfig:"endpoint-config",endpoints:endpoints}' \
  --output json
```

Expected Output:

```
{
  "endpointConfig": {
    "is-public-ip-enabled": false,
    "nsg-ids": [
      "ocidl.networksecuritygroup.oc1..."
    ],
    "subnet-id": "ocidl.subnet.oc1..."
  },
  "endpoints": {
    "private-endpoint": "10.0.0.10:6443",
    "public-endpoint": null
  }
}
```

Confirm that **is-public-ip-enabled** is **false**, **private-endpoint** is populated, and **public-endpoint** is **null**. If **is-public-ip-enabled** is **true** or **public-endpoint** contains a value, public IP access is enabled and the cluster does not meet this control.

Remediation:

Use private OKE control plane access by placing the Kubernetes API endpoint in a private subnet and disabling public IP assignment. This keeps Kubernetes API access on the VCN or an approved private network path.

For a new cluster, create the Kubernetes API endpoint in a private subnet and do not assign a public IP address.

```
oci ce cluster create \
  --name $CLUSTER_NAME \
  --compartment-id $COMPARTMENT_ID \
  --vcn-id $VCN_OCID \
  --kubernetes-version $KUBERNETES_VERSION \
  --endpoint-subnet-id $PRIVATE_ENDPOINT_SUBNET_OCID \
  --endpoint-public-ip-enabled false
```

For an existing cluster, update the endpoint configuration so the Kubernetes API endpoint is not assigned a public IP address.

```
oci ce cluster update-endpoint-config \
  --cluster-id $CLUSTER_ID \
  --is-public-ip-enabled false
```

Note: After changing whether a public IP address is assigned to the Kubernetes API endpoint, update route rules and security rules appropriately. Validate administrator, automation, and node access through the private endpoint before removing any temporary access path.

Default Value:

By default, the OCI CLI does not assign a public IP address to the Kubernetes API endpoint unless endpoint public IP access is explicitly enabled during cluster creation.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengclustersnodes.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengnetworkconfig.htm>
3. https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengcreatingclusterusingoke_topic-Using_the_Console_to_create_a_Custom_Cluster_with_Explicitly_Defined_Settings.htm
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengsettingupbastion.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	9.3 Maintain and Enforce Network-Based URL Filters Enforce and update network-based URL filters to limit an enterprise asset from connecting to potentially malicious or unapproved websites. Example implementations include category-based filtering, reputation-based filtering, or through the use of block lists. Enforce filters for all enterprise assets.			
v7	7.4 Maintain and Enforce Network-Based URL Filters Enforce network-based URL filters that limit a system's ability to connect to websites not approved by the organization. This filtering shall be enforced for each of the organization's systems, whether they are physically at an organization's facilities or not.			

5.4.3 *Ensure clusters are created with Private Nodes (Manual)*

Profile Applicability:

- Level 1

Description:

This recommendation is about the data plane, not the control plane. It means worker nodes are placed in private subnets and are not intended to have direct public internet exposure. In the common pattern, nodes run in private subnets while internet-facing ingress resources are placed in public subnets. This changes where compute runs and how node traffic exits the VCN, but it does not by itself determine whether the Kubernetes API endpoint is public, private, or both.

Rationale:

OKE worker nodes are created with public or private IP addresses according to whether public or private subnets are specified when defining node pools. Placing nodes in private subnets reduces direct reachability of the worker instances and gives tighter control over egress paths. At the same time, those nodes still need access to required services such as OCIR for image pulls, and that access must be provided through a NAT gateway, service gateway, or other required private network path. In a fully private cluster design with no outbound internet access, the Kubernetes API endpoint and supporting OCI services must remain reachable through private connectivity, route rules, and security rules that support node registration, image pulls, and required workload operations.

Impact:

This reduces direct internet exposure of the worker nodes and usually improves traffic control around the data plane, but it increases dependency on NAT design, service gateway access, route tables, security lists, network security groups, and private management methods such as bastion or private administrative access patterns. It also does not automatically make the control plane private. A cluster can have private nodes and still expose a public API endpoint unless the endpoint settings are hardened separately.

Audit:

Review the subnets used by each OKE node pool and verify that worker nodes are deployed in private subnets, not public subnets. For this control, the deciding evidence is node subnet placement, public IP behavior, and subnet routing, not the control plane endpoint settings. A compliant private-node design places nodes in subnets that prohibit public IP assignment on VNICs and do not have a default route to an Internet Gateway. If outbound access is required, the design should use a NAT gateway, service gateway, or other required private network path instead.

Review each node pool and identify the subnet IDs used by the worker nodes. If the node pool is placed in public subnets, this control is not satisfied.

```
oci ce node-pool list \
  --compartment-id $COMPARTMENT_ID \
  --cluster-id $CLUSTER_ID \
  --query 'data[*].{name:name,id:id}' \
  --output table
```

Review the subnet IDs used by a node pool.

```
oci ce node-pool get \
  --node-pool-id $NODE_POOL_ID \
  --query 'data.{name:name,subnetId:"subnet-id",placementSubnets:"node-
config-details"."placement-configs"[*].\"subnet-id\"}' \
  --output json
```

- **Expected output:** JSON containing the node pool name and the subnet ID or placement subnet IDs used by worker nodes. These are the subnets you must inspect next. Private node compliance is not determined by the node pool alone, but by whether these subnets are actually private.

Review each worker node subnet and verify that public IP assignment is prohibited.

```
oci network subnet get \
  --subnet-id $NODE_SUBNET_OCID \
  --query 'data.{displayName:"display-name",prohibitPublicIpOnVnic:"prohibit-
public-ip-on-vnic",routeTableId:"route-table-id\"}' \
  --output json
```

- **Expected output:** `prohibitPublicIpOnVnic` should be true for worker node subnets used by private nodes.
- Verify the node subnets do not allow public IP assignment. A false value for `prohibit-public-ip-on-vnic` indicates that VNICs in the subnet can receive public IP addresses, which is not the intended private node design.

Verify the route table associated with each node subnet does not contain a default route such as `0.0.0.0/0` to an Internet Gateway. Private subnets can still have outbound access through a NAT gateway or access to OCI services through a service gateway, but they should not route directly to an Internet Gateway.

```
oci network route-table get \
  --rt-id $ROUTE_TABLE_OCID \
  --query 'data.\"route-rules\"' \
  --output json
```

- Verify the subnets identified for the node pool are intended private subnets in the cluster VCN design and that nodes do not require direct public IP reachability for administration or workload traffic.

Verify outbound dependencies for private nodes are satisfied through NAT, service gateway, or required private network paths so nodes can still pull images and reach supporting services without direct public exposure.

Remediation:

For existing worker nodes that run in public subnets, remediate the nodes by replacing them with a new node pool that uses only private subnets, then migrating workloads and retiring the original node pool. The target private subnets should prohibit public IP assignment on VNICs and should not have a default route to an Internet Gateway. If outbound access is required for image pulls or OCI API calls, provide it through a NAT gateway, service gateway, or the required private network path instead.

- Create a new node pool that uses only private subnet IDs. Do not reuse public subnets for the replacement worker nodes.
- Ensure the replacement subnets prohibit public IP assignment on VNICs, so new worker instances are launched without public IPs.
- Ensure the route tables for those subnets do not send **0.0.0.0/0** directly to an Internet Gateway. If internet egress is needed, route it through a NAT gateway or use a service gateway for supported OCI service access. OCIR image pulls and other required service access must remain available through the approved private egress design.
- If custom node pool configuration or instance settings are used, confirm the network configuration does not assign public IP addresses or otherwise override the private subnet design.

Default Value:

By default, worker nodes are created with public or private IP addresses according to whether public or private subnets are specified when defining the node pool.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengnetworkconfig.htm>
2. https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengaddingnodepools_topic.htm
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/replace-worker-node-top.htm>
4. https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengpodnetworking_topic-OCI_CNI_plugin.htm

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.1 <u>Establish and Maintain a Secure Configuration Process</u> Establish and maintain a secure configuration process for enterprise assets (end-user devices, including portable and mobile, non-computing/IoT devices, and servers) and software (operating systems and applications). Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.			
v7	5.1 <u>Establish Secure Configurations</u> Maintain documented, standard security configuration standards for all authorized operating systems and software.			
v7	12 <u>Boundary Defense</u> Boundary Defense			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1556, T1611	TA0004, TA0006	M1048

5.4.4 Ensure Network Policy is Enabled and set as appropriate (Manual)

Profile Applicability:

- Level 1

Description:

Implementing Kubernetes **NetworkPolicies** in an Oracle Kubernetes Engine (OKE) cluster is a critical security measure for controlling and segregating Pod to Pod communication. By default, all pods in a Kubernetes cluster can freely communicate with each other, which increases the risk of lateral movement if a single workload is compromised. Enforcing network policies allows administrators to explicitly define which pods or namespaces can communicate, ensuring that workloads are properly isolated based on function, sensitivity, or compliance requirements. This segregation helps contain potential security breaches, limits unauthorized data access between applications, and supports the principle of least privilege in network communication, strengthening the overall security posture and reducing the attack surface within the OKE environment.

Configure **NetworkPolicy** enforcement and define **NetworkPolicy** resources to restrict Pod to Pod traffic within a cluster and segregate workloads.

Rationale:

By default, all pod to pod traffic within a cluster is allowed. **NetworkPolicy** creates a pod level firewall that can be used to restrict traffic between sources. Pod traffic is restricted by having a **NetworkPolicy** that selects it through the use of labels. Once there is any **NetworkPolicy** in a namespace selecting a particular pod, that pod will reject any connections that are not allowed by any **NetworkPolicy**. Other pods in the namespace that are not selected by any **NetworkPolicy** will continue to accept all traffic.

NetworkPolicies are managed via the Kubernetes **NetworkPolicy** API and enforced by a network plugin, so simply creating the resource without a compatible network plugin to implement it will have no effect. OKE supports NetworkPolicy enforcement through integration with the Calico network policy engine when using the OCI CNI plugin. OKE does not support using the Calico network policy engine with the flannel CNI plugin.

Impact:

NetworkPolicy enforcement requires a supported enforcement solution, such as Calico with the OCI CNI plugin, and **NetworkPolicy** resources must be designed carefully so required application, DNS, monitoring, ingress, and egress traffic is not blocked.

If the cluster uses flannel and **NetworkPolicy** enforcement is required, Oracle documents that Calico should not be installed alongside flannel because that configuration is not supported and can cause network issues.

Enabling enforcement or changing network policies can affect application connectivity. Test policies in lower environments and validate required traffic before applying restrictive policies to production namespaces.

Audit:

Below is a command line statement to check whether **NetworkPolicies** are configured in your Oracle Kubernetes Engine (OKE) cluster:

```
echo -e "NAMESPACE\tNETWORK_POLICY_COUNT" && \
for ns in $(kubectl get ns -o jsonpath='{.items[*].metadata.name}'); do
  count=$(kubectl get networkpolicy -n "$ns" --no-headers 2>/dev/null | wc -l
  | tr -d ' ')
  echo -e "${ns}\t${count}"
done
```

Sample Output

NAMESPACE	NETWORK_POLICY_COUNT
default	2
kube-node-lease	3
kube-public	5
kube-system	0

A count of 0 means no NetworkPolicy objects exist in that namespace and pods that are not selected by any NetworkPolicy continue to accept all traffic.

A count greater than 0 means network policies are defined and applied, helping isolate and protect workloads.

Also confirm that a supported **NetworkPolicy** enforcement solution is installed and running. For OKE, verify Calico is present when **NetworkPolicy** enforcement is required.

```
kubectl get pods -A -o json | jq -r '
  .items[]
  | select(
    (.metadata.name | test("calico"; "i"))
    or
    ([.spec.containers[]?.image] | join(" ") | test("calico"; "i"))
  )
  | "\"(.metadata.namespace)\t\(.metadata.name)\t\(.status.phase)\""
```

Confirm the cluster CNI configuration supports the intended enforcement model.

```
oci ce cluster list-addons --cluster-id $CLUSTER_ID
```

Review the output for the CNI plugin add on in use. If the cluster uses the OCI CNI plugin, Calico can be used for **NetworkPolicy** enforcement. If the cluster uses flannel, do not install the Calico network policy engine alongside flannel.

Remediation:

For OKE, do not use an `isNetworkPolicyEnabled` cluster flag. Configure `NetworkPolicy` enforcement by using a supported enforcement solution and then create the required Kubernetes NetworkPolicy resources.

If the cluster uses the OCI CNI plugin, install and configure the Calico network policy engine using Oracle's documented process, then apply `NetworkPolicy` resources to workload namespaces.

If the cluster uses flannel and `NetworkPolicy` enforcement is required, follow Oracle's documented approach to replace flannel with Calico for new nodes, or create a replacement cluster design that supports the required policy enforcement. Do not install Calico alongside flannel.

- Define namespace level NetworkPolicies after enforcement is available, and apply a restrictive default policy to each workload namespace where required.
- Add allow rules for required traffic and create specific NetworkPolicy resources to allow communication only between approved pods or namespaces.

Apply a Default Deny All Policy. Add specific policies for authorized communication by defining NetworkPolicy manifests that explicitly allow required traffic between trusted workloads, services, or namespaces.

Default Value:

By default, NetworkPolicy resources are not created, and NetworkPolicy resources have no effect unless a compatible enforcement solution is installed.

References:

1. <https://docs.oracle.com/iaas/Content/ContEng/Tasks/contengsettingupcalico.htm>
2. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengpodnetworking.htm>
3. https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengpodnetworking_topic-OCI_CNI_plugin.htm
4. https://docs.oracle.com/en-us/iaas/Content/ContEng/Concepts/contengCompatibilitymatrix_topic.htm

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.4 <u>Implement and Manage a Firewall on Servers</u> Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.			
v7	9.2 <u>Ensure Only Approved Ports, Protocols and Services Are Running</u> Ensure that only network ports, protocols, and services listening on a system with validated business needs, are running on each system.			
v7	9.4 <u>Apply Host-based Firewalls or Port Filtering</u> Apply host-based firewalls or port filtering tools on end systems, with a default-deny rule that drops all traffic except those services and ports that are explicitly allowed.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1046	TA0007	M1030, M1042

5.4.5 Use TLS certificates on OKE load balancer listeners for encrypted client connections (Manual)

Profile Applicability:

- Level 1

Description:

Using TLS certificates on OKE load balancer listeners protects client to load balancer traffic in transit by presenting a trusted server certificate and negotiating encrypted HTTPS or TLS connections at the listener. In OKE, this commonly means configuring TLS on an OCI Load Balancer created for Kubernetes Service resources, or configuring HTTPS/TLS on OCI native ingress controller listeners so externally exposed application entry points do not rely on unencrypted HTTP.

Rationale:

OKE supports TLS configuration on the primary load balancer paths used for exposed workloads. For Kubernetes Services of type LoadBalancer, OCI Load Balancer listeners can terminate SSL/TLS using Kubernetes secrets or OCI Certificates service based certificate configuration. For OCI native ingress controller resources, HTTPS/TLS listeners can be configured using either a Kubernetes secret that allows the ingress controller to obtain a certificate from OCI Certificates, or a certificate OCID that references a certificate already created in OCI Certificates.

This matters because the listener is the point where the client session is negotiated, the certificate is presented, and encrypted traffic is established before requests are forwarded to targets. OKE also supports backend TLS where the design requires encryption between the load balancer and worker nodes or pods, so requiring certificates on client facing load balancer listeners is the baseline control for securing the external client connection path.

Impact:

This control improves protection against eavesdropping and tampering on client facing connections, but it introduces certificate lifecycle and listener configuration requirements. The certificate must match the intended domain names, the correct listener ports must be configured for TLS, and expired or missing certificates can break client access even when the workload itself is healthy. It also does not by itself guarantee backend encryption beyond the load balancer, because traffic from the load balancer to the backend may still be unencrypted unless a separate backend TLS design is implemented.

Audit:

Review all OKE Service resources of type **LoadBalancer** and all Ingress resources that expose client traffic, and verify that client facing listeners use HTTPS or TLS with a valid certificate. For OCI Load Balancer backed Service resources, listener encryption is configured with annotations such as **service.beta.kubernetes.io/oci-load-balancer-ssl-ports** and **service.beta.kubernetes.io/oci-load-balancer-tls-secret**, or with OCI Certificates based configuration where used. For OCI native ingress controller resources, HTTPS/TLS listeners require a Kubernetes TLS secret or the **oci-native-ingress.oraclecloud.com/certificate-ocid** annotation. A TCP listener on port 443 is a different design because it passes encrypted traffic through instead of terminating TLS at the load balancer.

- Identify all internet facing or client facing **LoadBalancer** Services and Ingress resources.
- Verify OCI Load Balancer services use **service.beta.kubernetes.io/oci-load-balancer-ssl-ports** and a valid TLS certificate source for listener encryption.
- Verify OCI native ingress controller listeners are configured for HTTPS/TLS and have the required Kubernetes secret or certificate OCID association.
- Verify the certificate is valid for the intended hostname and that the listener design matches the intended TLS termination point.

Remediation:

Configure every client facing OKE load balancer listener to use HTTPS or TLS with a valid certificate so client to load balancer traffic is encrypted in transit. For OCI Load Balancer backed Service resources, configure SSL/TLS listener ports and associate a TLS certificate using Kubernetes secrets or OCI Certificates based configuration. For OCI native ingress controller resources, configure HTTPS/TLS listeners and associate the required Kubernetes secret or certificate OCID. If the design requires encryption beyond the load balancer, implement backend TLS separately because listener TLS alone protects only the client facing connection.

- Add a valid certificate to each client facing OCI Load Balancer or OCI native ingress controller listener that should serve encrypted traffic.
- For OCI Load Balancer services, set **service.beta.kubernetes.io/oci-load-balancer-ssl-ports** to the required listener ports and configure the appropriate TLS certificate source.
- For OCI native ingress controller resources, configure HTTPS listeners and associate the required certificate source with the listener configuration.
- Replace or renew invalid certificates and implement backend TLS separately where backend encryption is required.

Default Value:

By default, OKE does not automatically attach TLS certificates to client facing load balancer listeners. TLS is used only when you explicitly configure an HTTPS or TLS listener and associate a certificate, while OCI Load Balancer services and OCI native ingress controller resources can otherwise expose non TLS listener behavior unless the required certificate configuration is set.

References:

1. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengcreatingloadbalancers-subtopic.htm>
2. https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengcreatingloadbalancer_topic-Summaryofannotations.htm
3. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengsettingupnativeingresscontroller-configuring.htm>
4. <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengsettingupnativeingresscontroller.htm>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 Encrypt Sensitive Data in Transit Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).			
v7	14.4 Encrypt All Sensitive Information in Transit Encrypt all sensitive information in transit.			

MITRE ATT&CK Mappings:

Techniques / Sub-techniques	Tactics	Mitigations
T1609	TA0002	M1035

Appendix: Summary Table

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
1	Control Plane Components		
2	Control Plane Configuration		
2.1	Authentication and Authorization		
2.1.1	Use OCI IAM or OIDC tokens instead of user client certificates (Manual)	☐	☐
2.2	Logging		
2.2.1	Ensure authorized access to OCI Audit logs for OKE activity (Manual)	☐	☐
3	Worker Nodes		
3.1	Worker Node Configuration Files		
3.1.1	Ensure that the kubelet.conf file permissions are set to 644 or more restrictive (Automated)	☐	☐
3.1.2	Ensure that the kubelet.conf file ownership is set to root:root (Automated)	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive (Automated)	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root (Automated)	☐	☐
3.2	Kubelet		
3.2.1	Ensure kubelet anonymous authentication is disabled (Automated)	☐	☐
3.2.2	Ensure kubelet authorization mode is set to Webhook (Automated)	☐	☐

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
3.2.3	Ensure the kubelet X.509 client CA file is configured (Automated)	☐	☐
3.2.4	Ensure the kubelet read-only port is disabled (Manual)	☐	☐
3.2.5	Ensure kubelet makeIPTablesUtilChains is set to true (Automated)	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value (Automated)	☐	☐
3.2.7	Ensure kubelet tlsCertFile and tlsPrivateKeyFile are configured appropriately (Automated)	☐	☐
3.2.8	Ensure the kubelet client certificate rotation is enabled (Automated)	☐	☐
4	Policies		
4.1	RBAC and Service Accounts		
4.1.1	Ensure that the cluster-admin role is only used where required (Manual)	☐	☐
4.1.2	Ensure access to Secrets is minimized and granted only where required (Manual)	☐	☐
4.1.3	Minimize wildcard use in Roles and ClusterRoles (Manual)	☐	☐
4.1.4	Minimize access to create pods (Manual)	☐	☐
4.1.5	Ensure that default service accounts are not actively used. (Automated)	☐	☐
4.1.6	Ensure that Service Account Tokens are only mounted where necessary (Automated)	☐	☐
4.2	Pod Security Policies		
4.2.1	Minimize the admission of privileged containers (Automated)	☐	☐

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace (Automated)	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace (Automated)	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace (Automated)	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation (Automated)	☐	☐
4.3	CNI Plugin		
4.3.1	Ensure the OKE CNI add-on uses an Oracle supported version (Automated)	☐	☐
4.3.2	Ensure workload namespaces have required NetworkPolicies defined (Manual)	☐	☐
4.4	Secrets Management		
4.4.1	Prefer exposing secrets as mounted files rather than environment variables (Automated)	☐	☐
4.4.2	Consider external secret storage (Manual)	☐	☐
4.5	General Policies		
4.5.1	Create administrative boundaries between resources using namespaces (Manual)	☐	☐
4.5.2	Apply securityContext controls to pods and containers (Manual)	☐	☐
4.5.3	The default namespace should not be used (Manual)	☐	☐
5	Managed services		
5.1	Image Registry and Image Scanning		

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
5.1.1	Ensure image vulnerability scanning uses OCIR or a third party provider (Manual)	☐	☐
5.1.2	Restrict OKE and Kubernetes API access to required users (Manual)	☐	☐
5.1.3	Restrict OCIR access for OKE workloads to required image pulls (Manual)	☐	☐
5.1.4	Allow only approved container registries for images used by OKE workloads (Manual)	☐	☐
5.2	Identity and Access Management (IAM)		
5.2.1	Use dedicated Kubernetes service accounts for OKE workloads (Automated)	☐	☐
5.3	Cloud Key Management Service (Cloud KMS)		
5.3.1	Use customer managed KMS keys for OKE envelope encryption of Kubernetes secrets (Manual)	☐	☐
5.4	Cluster Networking		
5.4.1	Restrict OKE control plane endpoint access using private endpoints or approved CIDRs (Automated)	☐	☐
5.4.2	Enable private OKE control plane access and disable public IP access (Automated)	☐	☐
5.4.3	Ensure clusters are created with Private Nodes (Manual)	☐	☐
5.4.4	Ensure Network Policy is Enabled and set as appropriate (Manual)	☐	☐
5.4.5	Use TLS certificates on OKE load balancer listeners for encrypted client connections (Manual)	☐	☐

Appendix: CIS Controls v7 IG 1 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Use OCI IAM or OIDC tokens instead of user client certificates	☐	☐
2.2.1	Ensure authorized access to OCI Audit logs for OKE activity	☐	☐
3.1.1	Ensure that the kubelet.conf file permissions are set to 644 or more restrictive	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.2.1	Ensure kubelet anonymous authentication is disabled	☐	☐
3.2.5	Ensure kubelet makeIPTablesUtilChains is set to true	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value	☐	☐
4.1.4	Minimize access to create pods	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.1.6	Ensure that Service Account Tokens are only mounted where necessary	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.3.1	Ensure the OKE CNI add-on uses an Oracle supported version	☐	☐
4.5.2	Apply securityContext controls to pods and containers	☐	☐
5.1.2	Restrict OKE and Kubernetes API access to required users	☐	☐
5.2.1	Use dedicated Kubernetes service accounts for OKE workloads	☐	☐
5.4.3	Ensure clusters are created with Private Nodes	☐	☐
5.4.4	Ensure Network Policy is Enabled and set as appropriate	☐	☐

Appendix: CIS Controls v7 IG 2 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Use OCI IAM or OIDC tokens instead of user client certificates	☐	☐
2.2.1	Ensure authorized access to OCI Audit logs for OKE activity	☐	☐
3.1.1	Ensure that the kubelet.conf file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubelet.conf file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.2.1	Ensure kubelet anonymous authentication is disabled	☐	☐
3.2.2	Ensure kubelet authorization mode is set to Webhook	☐	☐
3.2.3	Ensure the kubelet X.509 client CA file is configured	☐	☐
3.2.4	Ensure the kubelet read-only port is disabled	☐	☐
3.2.5	Ensure kubelet makeIPTablesUtilChains is set to true	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value	☐	☐
3.2.7	Ensure kubelet tlsCertFile and tlsPrivateKeyFile are configured appropriately	☐	☐
3.2.8	Ensure the kubelet client certificate rotation is enabled	☐	☐
4.1.4	Minimize access to create pods	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.1.6	Ensure that Service Account Tokens are only mounted where necessary	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.3.1	Ensure the OKE CNI add-on uses an Oracle supported version	☐	☐
4.3.2	Ensure workload namespaces have required NetworkPolicies defined	☐	☐
4.4.1	Prefer exposing secrets as mounted files rather than environment variables	☐	☐
4.5.2	Apply securityContext controls to pods and containers	☐	☐
5.1.1	Ensure image vulnerability scanning uses OCIR or a third party provider	☐	☐
5.1.2	Restrict OKE and Kubernetes API access to required users	☐	☐
5.1.4	Allow only approved container registries for images used by OKE workloads	☐	☐
5.2.1	Use dedicated Kubernetes service accounts for OKE workloads	☐	☐
5.4.1	Restrict OKE control plane endpoint access using private endpoints or approved CIDRs	☐	☐
5.4.2	Enable private OKE control plane access and disable public IP access	☐	☐
5.4.3	Ensure clusters are created with Private Nodes	☐	☐
5.4.4	Ensure Network Policy is Enabled and set as appropriate	☐	☐
5.4.5	Use TLS certificates on OKE load balancer listeners for encrypted client connections	☐	☐

Appendix: CIS Controls v7 IG 3 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Use OCI IAM or OIDC tokens instead of user client certificates	☐	☐
2.2.1	Ensure authorized access to OCI Audit logs for OKE activity	☐	☐
3.1.1	Ensure that the kubelet.conf file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubelet.conf file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.2.1	Ensure kubelet anonymous authentication is disabled	☐	☐
3.2.2	Ensure kubelet authorization mode is set to Webhook	☐	☐
3.2.3	Ensure the kubelet X.509 client CA file is configured	☐	☐
3.2.4	Ensure the kubelet read-only port is disabled	☐	☐
3.2.5	Ensure kubelet makeIPTablesUtilChains is set to true	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value	☐	☐
3.2.7	Ensure kubelet tlsCertFile and tlsPrivateKeyFile are configured appropriately	☐	☐
3.2.8	Ensure the kubelet client certificate rotation is enabled	☐	☐
4.1.4	Minimize access to create pods	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.1.6	Ensure that Service Account Tokens are only mounted where necessary	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.3.1	Ensure the OKE CNI add-on uses an Oracle supported version	☐	☐
4.3.2	Ensure workload namespaces have required NetworkPolicies defined	☐	☐
4.4.1	Prefer exposing secrets as mounted files rather than environment variables	☐	☐
4.4.2	Consider external secret storage	☐	☐
4.5.2	Apply securityContext controls to pods and containers	☐	☐
4.5.3	The default namespace should not be used	☐	☐
5.1.1	Ensure image vulnerability scanning uses OCIR or a third party provider	☐	☐
5.1.2	Restrict OKE and Kubernetes API access to required users	☐	☐
5.1.4	Allow only approved container registries for images used by OKE workloads	☐	☐
5.2.1	Use dedicated Kubernetes service accounts for OKE workloads	☐	☐
5.3.1	Use customer managed KMS keys for OKE envelope encryption of Kubernetes secrets	☐	☐
5.4.1	Restrict OKE control plane endpoint access using private endpoints or approved CIDRs	☐	☐
5.4.2	Enable private OKE control plane access and disable public IP access	☐	☐
5.4.3	Ensure clusters are created with Private Nodes	☐	☐
5.4.4	Ensure Network Policy is Enabled and set as appropriate	☐	☐
5.4.5	Use TLS certificates on OKE load balancer listeners for encrypted client connections	☐	☐

Appendix: CIS Controls v7 Unmapped Recommendations

Recommendation		Set Correctly	
		Yes	No
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐
4.1.2	Ensure access to Secrets is minimized and granted only where required	☐	☐
4.1.3	Minimize wildcard use in Roles and ClusterRoles	☐	☐

Appendix: CIS Controls v8 IG 1 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.2.1	Ensure authorized access to OCI Audit logs for OKE activity	☐	☐
3.1.1	Ensure that the kubelet.conf file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubelet.conf file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
3.2.1	Ensure kubelet anonymous authentication is disabled	☐	☐
3.2.2	Ensure kubelet authorization mode is set to Webhook	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value	☐	☐
4.1.2	Ensure access to Secrets is minimized and granted only where required	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.1.6	Ensure that Service Account Tokens are only mounted where necessary	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.3.1	Ensure the OKE CNI add-on uses an Oracle supported version	☐	☐
4.3.2	Ensure workload namespaces have required NetworkPolicies defined	☐	☐
4.4.1	Prefer exposing secrets as mounted files rather than environment variables	☐	☐
4.4.2	Consider external secret storage	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.5.1	Create administrative boundaries between resources using namespaces	☐	☐
4.5.2	Apply securityContext controls to pods and containers	☐	☐
5.1.3	Restrict OCIR access for OKE workloads to required image pulls	☐	☐
5.4.3	Ensure clusters are created with Private Nodes	☐	☐
5.4.4	Ensure Network Policy is Enabled and set as appropriate	☐	☐

Appendix: CIS Controls v8 IG 2 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Use OCI IAM or OIDC tokens instead of user client certificates	☐	☐
2.2.1	Ensure authorized access to OCI Audit logs for OKE activity	☐	☐
3.1.1	Ensure that the kubelet.conf file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubelet.conf file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
3.2.1	Ensure kubelet anonymous authentication is disabled	☐	☐
3.2.2	Ensure kubelet authorization mode is set to Webhook	☐	☐
3.2.3	Ensure the kubelet X.509 client CA file is configured	☐	☐
3.2.4	Ensure the kubelet read-only port is disabled	☐	☐
3.2.5	Ensure kubelet makeIPTablesUtilChains is set to true	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value	☐	☐
3.2.7	Ensure kubelet tlsCertFile and tlsPrivateKeyFile are configured appropriately	☐	☐
3.2.8	Ensure the kubelet client certificate rotation is enabled	☐	☐
4.1.2	Ensure access to Secrets is minimized and granted only where required	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.1.6	Ensure that Service Account Tokens are only mounted where necessary	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.3.1	Ensure the OKE CNI add-on uses an Oracle supported version	☐	☐
4.3.2	Ensure workload namespaces have required NetworkPolicies defined	☐	☐
4.4.1	Prefer exposing secrets as mounted files rather than environment variables	☐	☐
4.4.2	Consider external secret storage	☐	☐
4.5.1	Create administrative boundaries between resources using namespaces	☐	☐
4.5.2	Apply securityContext controls to pods and containers	☐	☐
4.5.3	The default namespace should not be used	☐	☐
5.1.1	Ensure image vulnerability scanning uses OCIR or a third party provider	☐	☐
5.1.3	Restrict OCIR access for OKE workloads to required image pulls	☐	☐
5.2.1	Use dedicated Kubernetes service accounts for OKE workloads	☐	☐
5.3.1	Use customer managed KMS keys for OKE envelope encryption of Kubernetes secrets	☐	☐
5.4.1	Restrict OKE control plane endpoint access using private endpoints or approved CIDRs	☐	☐
5.4.2	Enable private OKE control plane access and disable public IP access	☐	☐
5.4.3	Ensure clusters are created with Private Nodes	☐	☐
5.4.4	Ensure Network Policy is Enabled and set as appropriate	☐	☐
5.4.5	Use TLS certificates on OKE load balancer listeners for encrypted client connections	☐	☐

Appendix: CIS Controls v8 IG 3 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Use OCI IAM or OIDC tokens instead of user client certificates	☐	☐
2.2.1	Ensure authorized access to OCI Audit logs for OKE activity	☐	☐
3.1.1	Ensure that the kubelet.conf file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubelet.conf file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
3.2.1	Ensure kubelet anonymous authentication is disabled	☐	☐
3.2.2	Ensure kubelet authorization mode is set to Webhook	☐	☐
3.2.3	Ensure the kubelet X.509 client CA file is configured	☐	☐
3.2.4	Ensure the kubelet read-only port is disabled	☐	☐
3.2.5	Ensure kubelet makeIPTablesUtilChains is set to true	☐	☐
3.2.6	Ensure the kubelet event recording QPS is set to an appropriate value	☐	☐
3.2.7	Ensure kubelet tlsCertFile and tlsPrivateKeyFile are configured appropriately	☐	☐
3.2.8	Ensure the kubelet client certificate rotation is enabled	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐
4.1.2	Ensure access to Secrets is minimized and granted only where required	☐	☐
4.1.3	Minimize wildcard use in Roles and ClusterRoles	☐	☐
4.1.4	Minimize access to create pods	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.1.6	Ensure that Service Account Tokens are only mounted where necessary	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.3.1	Ensure the OKE CNI add-on uses an Oracle supported version	☐	☐
4.3.2	Ensure workload namespaces have required NetworkPolicies defined	☐	☐
4.4.1	Prefer exposing secrets as mounted files rather than environment variables	☐	☐
4.4.2	Consider external secret storage	☐	☐
4.5.1	Create administrative boundaries between resources using namespaces	☐	☐
4.5.2	Apply securityContext controls to pods and containers	☐	☐
4.5.3	The default namespace should not be used	☐	☐
5.1.1	Ensure image vulnerability scanning uses OCIR or a third party provider	☐	☐
5.1.2	Restrict OKE and Kubernetes API access to required users	☐	☐
5.1.3	Restrict OCIR access for OKE workloads to required image pulls	☐	☐
5.2.1	Use dedicated Kubernetes service accounts for OKE workloads	☐	☐
5.3.1	Use customer managed KMS keys for OKE envelope encryption of Kubernetes secrets	☐	☐
5.4.1	Restrict OKE control plane endpoint access using private endpoints or approved CIDRs	☐	☐

Recommendation		Set Correctly	
		Yes	No
5.4.2	Enable private OKE control plane access and disable public IP access	☐	☐
5.4.3	Ensure clusters are created with Private Nodes	☐	☐
5.4.4	Ensure Network Policy is Enabled and set as appropriate	☐	☐
5.4.5	Use TLS certificates on OKE load balancer listeners for encrypted client connections	☐	☐

Appendix: CIS Controls v8 Unmapped Recommendations

Recommendation		Set Correctly	
		Yes	No
5.1.4	Allow only approved container registries for images used by OKE workloads	☐	☐

Appendix: Change History

Date	Version	Changes for this version
June 30, 2026	2.0.0	Removed Recommendation 5.5.1 – the recommendation is no longer relevant – it was determined that it is redundant and covered in the RBAC and Service section
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 5.4.5
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 5.4.4
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 5.4.3
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 5.4.2
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 5.4.1
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 5.3.1
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 5.2.1
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 5.1.4
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 5.1.3
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 5.1.2
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 5.1.1
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.5.3

Date	Version	Changes for this version
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.5.2
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.5.1
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.4.2
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.4.1
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.3.2
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.3.1
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.2.5
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.2.4
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.2.3
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.2.2
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.2.1
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.1.6
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.1.5
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.1.4
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.1.3

Date	Version	Changes for this version
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.1.2
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 4.1.1
June 30, 2026	2.0.0	Removed Recommendation 3.2.10 – the recommendation is no longer relevant – setting has been deprecated.
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 3.2.8
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 3.2.7
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 3.2.6
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 3.2.5
June 30, 2026	2.0.0	Removed Recommendation 3.2.5 – the recommendation is no longer relevant – setting has been deprecated.
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 3.2.4
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 3.2.3
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 3.2.2
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 3.2.1
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 3.1.4
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 3.1.3
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 3.1.2

Date	Version	Changes for this version
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 3.1.1
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 2.2.1
June 30, 2026	2.0.0	Updated description, audit and remediation sections of Recommendation 2.1.1
November 1, 2025	1.8.0	Added and tested support for the latest Kubernetes Cluster version 1.34
October 15 th , 2025	1.8.0	Added and tested support for the Kubernetes Cluster version 1.33
October 12, 2025	1.8.0	Updated recommendation 5.10.2 audit statement
October 12, 2025	1.8.0	Updated recommendation 5.10.1 audit statement
October 12, 2025	1.8.0	Updated recommendation 5.9.2 audit statement
October 12, 2025	1.8.0	Updated recommendation 5.9.1 audit statement
October 12, 2025	1.8.0	Updated recommendation 5.8.3 audit statement
October 12, 2025	1.8.0	Updated recommendation 5.8.1 audit statement
October 12, 2025	1.8.0	Updated recommendation 5.5.6 audit statement
October 12, 2025	1.8.0	Updated recommendation 5.5.5 audit statement
October 12, 2025	1.8.0	Updated recommendation 5.5.4 audit statement
October 12, 2025	1.8.0	Updated recommendation 5.5.3 audit statement

Date	Version	Changes for this version
October 12, 2025	1.8.0	Updated recommendation 5.5.2 audit statement
October 12, 2025	1.8.0	Updated recommendation 5.5.1 audit statement
October 12, 2025	1.8.0	Updated recommendation 5.4.1 audit statement
October 12, 2025	1.8.0	Updated recommendation 5.3.1 audit statement
October 12, 2025	1.8.0	Updated recommendation 5.2.2 audit statement
October 10 th , 2025	1.8.0	Updated recommendation 5.2.1 audit statement
October 7 th , 2025	1.8.0	Updated recommendation 4.6.4 audit statement
October 7 th , 2025	1.8.0	Updated recommendation 4.3.2 audit statement
October 7 th , 2025	1.8.0	Updated recommendation 4.1.4 audit statement
October 5, 2025	1.8.0	Updated recommendation 4.1.5 audit statement
October 5, 2025	1.8.0	Updated recommendation 4.1.3 audit statement
October 5, 2025	1.8.0	Updated recommendation 4.1.2 audit statement
October 5, 2025	1.8.0	Updated recommendation 4.1.1 audit statement
October 1, 2025	1.8.0	Updated recommendation 3.1.4 audit statement
5/1/2025	1.7.0	Tested AAC for OKE clusters built on Kubernetes v1.31, 1.30, 1.29
5/1/2025	1.7.0	Added Support for OKE clusters built on Kubernetes v1.31

Date	Version	Changes for this version
4/22/2025	1.7.0	3.1.3 recommendation edited and AAC test script modified
4/20/2025	1.7.0	3.1.2 recommendation edited to reflect config.json
4/20/2025	1.7.0	3.1.1 recommendation edited to reflect new path and file name for kubelet-config.json
4/14/2025	1.7.0	3.1.1 Edited AAC test script
4/14/2025	1.7.0	2.1.1 changed from an automated test to manual
11/01/2024	1.6.0	Tested AAC for OKE clusters built on Kubernetes v1.30, 1.29, 1.28
11/01/2024	1.6.0	Added Support for OKE clusters built on Kubernetes v1.30, 1.29, 1.28